

# ECE/CS 757 Advanced Computer Architecture II

## Spring 2025

### Interconnects (Part 6)

**Instructor:** Prof. Joshua San Miguel

Some slides adopted from Natalie Enright Jerger

# Last Lecture

- Packet-based flow control
- Flit-based flow control
- Virtual channels
- Deadlocks

# This Lecture

- Buffer backpressure
- Router microarchitecture
- GPUs

# Buffer Backpressure

Need mechanism to prevent overflow of router buffers

- Cannot drop packets; NoCs are lossless
- Upstream routers need to know buffer availability at downstream routers
- Must be conservative; can't overestimate available buffer space
- Significantly impacts network throughput

# Buffer Backpressure – Credit-Based Flow Control

Upstream router keeps track of credit counts from downstream router, which indicates available buffer space

Upstream router:

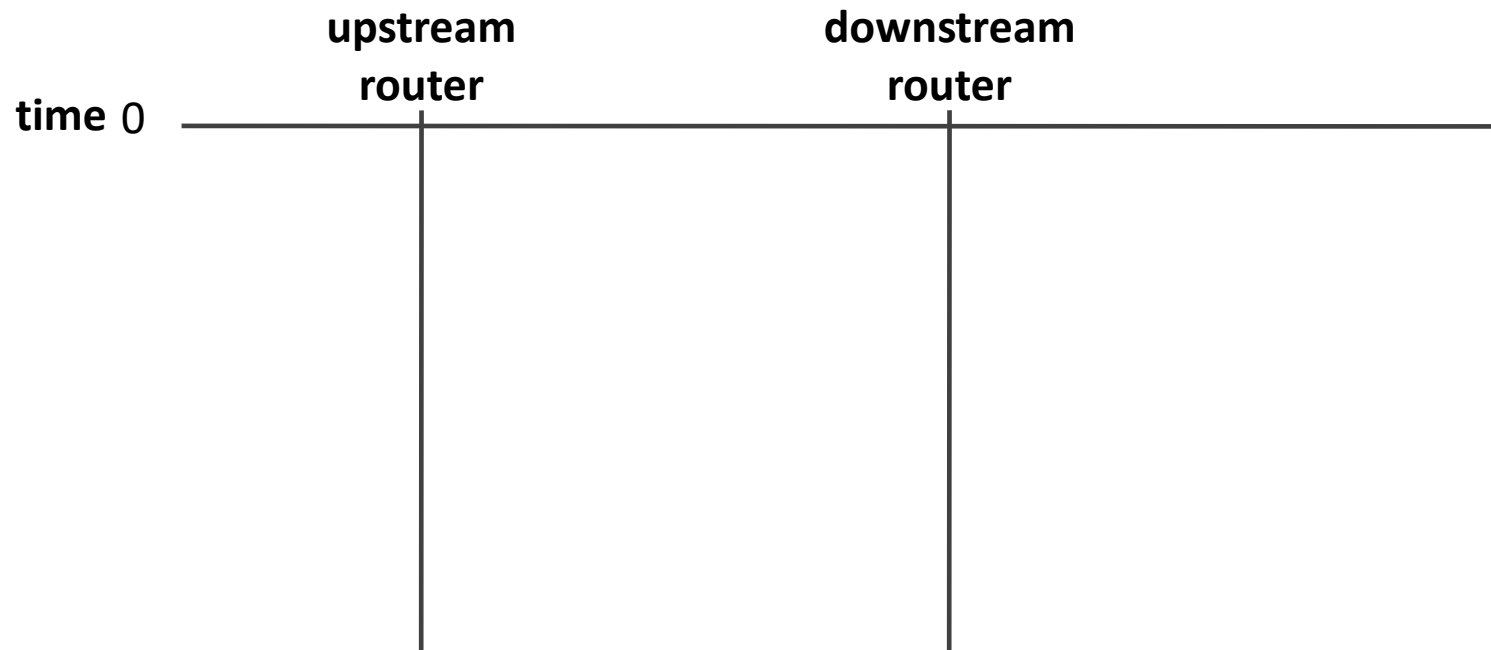
- When flit forwarded, decrement credit count
- If credit count 0, downstream buffer is full; stop sending flits

Downstream router:

- When flit forwarded and buffer freed, send credit to upstream router

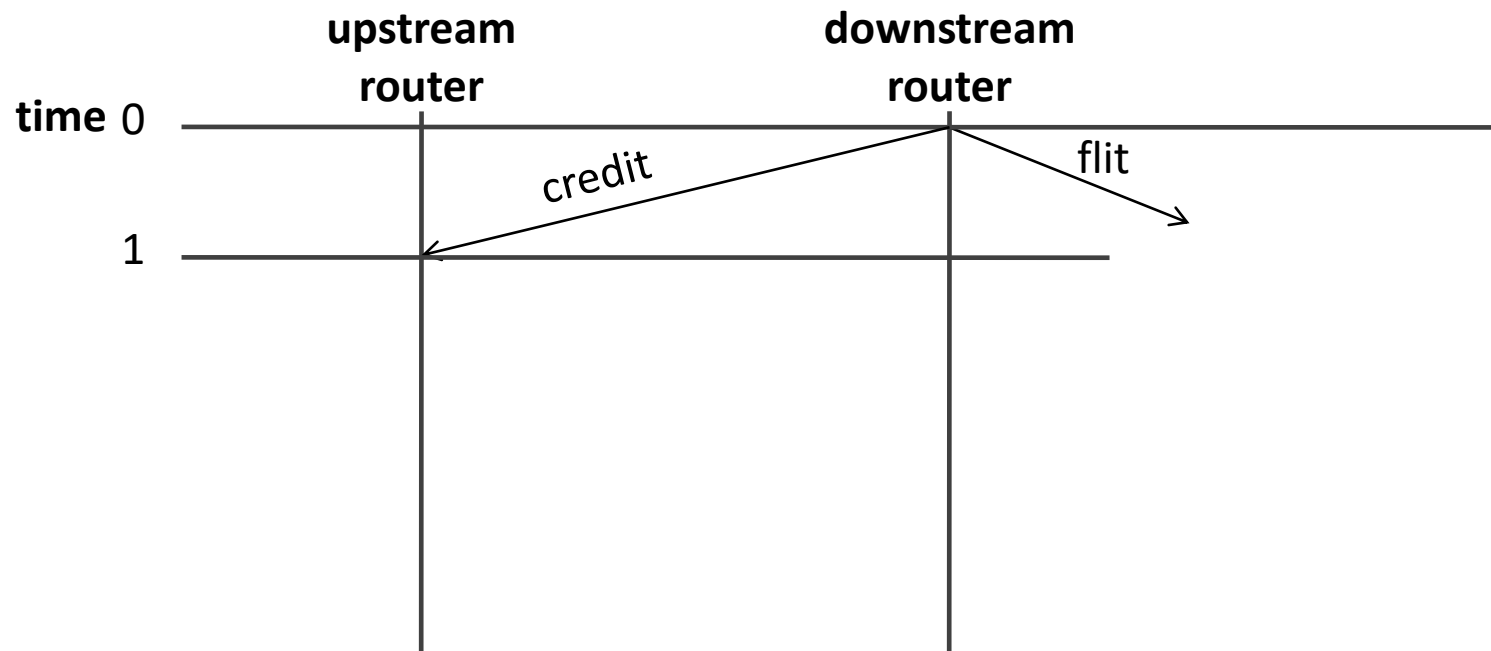
# Buffer Backpressure – Credit-Based Flow Control

e.g., Assume 3-cycle flit processing pipeline and 1-cycle link delay (data/credits)



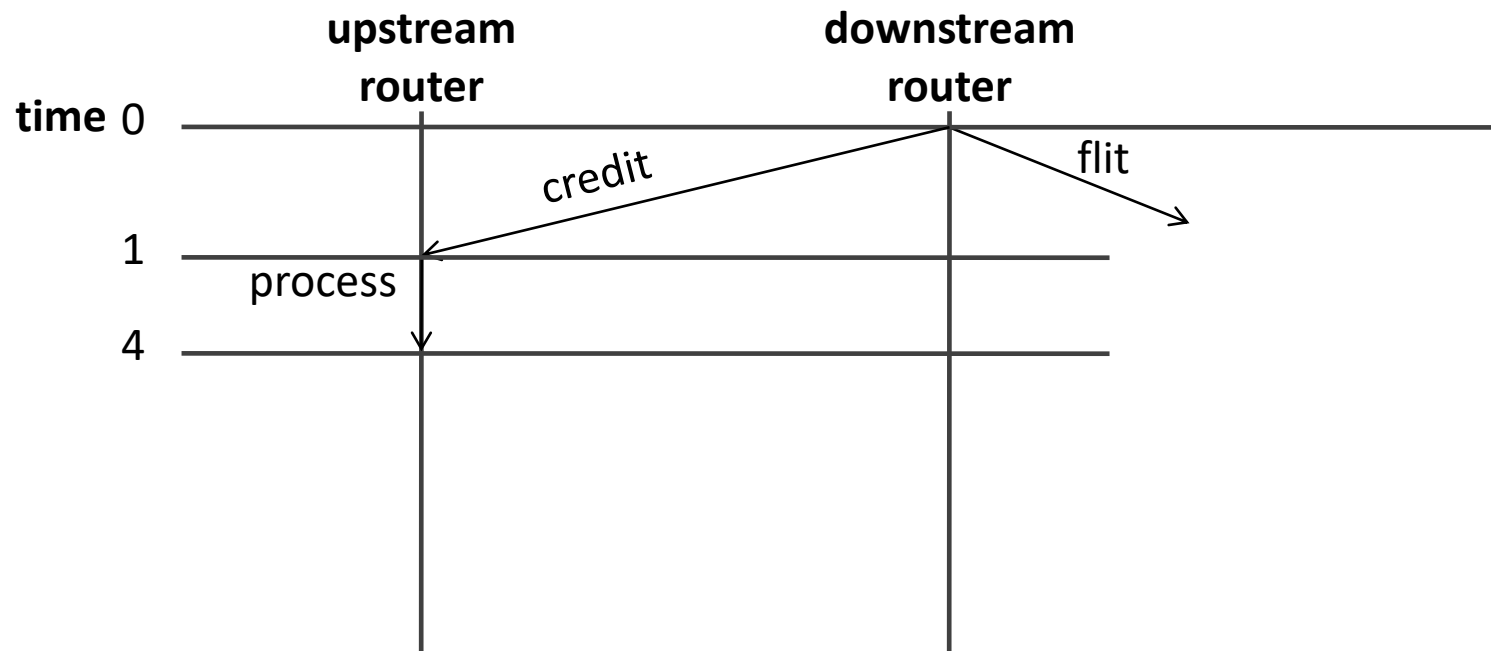
# Buffer Backpressure – Credit-Based Flow Control

e.g., Assume 3-cycle flit processing pipeline and 1-cycle link delay (data/credits)



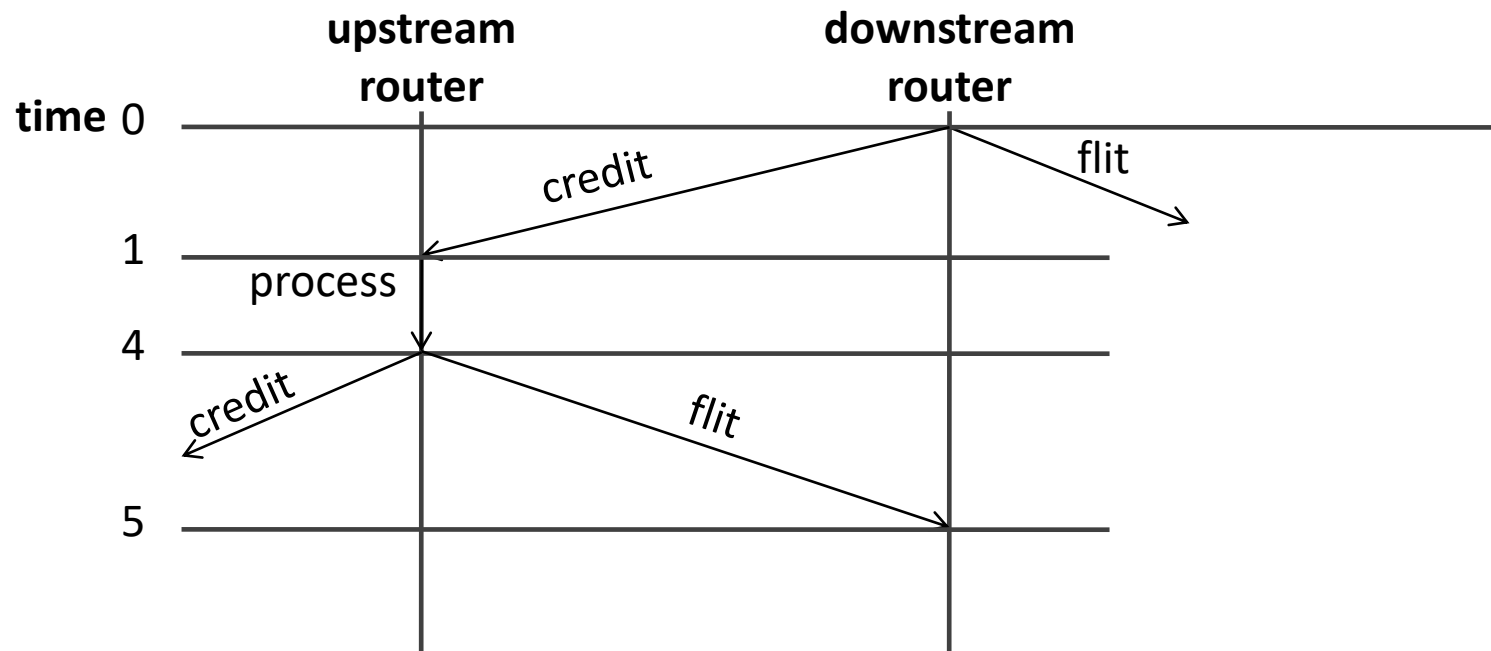
# Buffer Backpressure – Credit-Based Flow Control

e.g., Assume 3-cycle flit processing pipeline and 1-cycle link delay (data/credits)



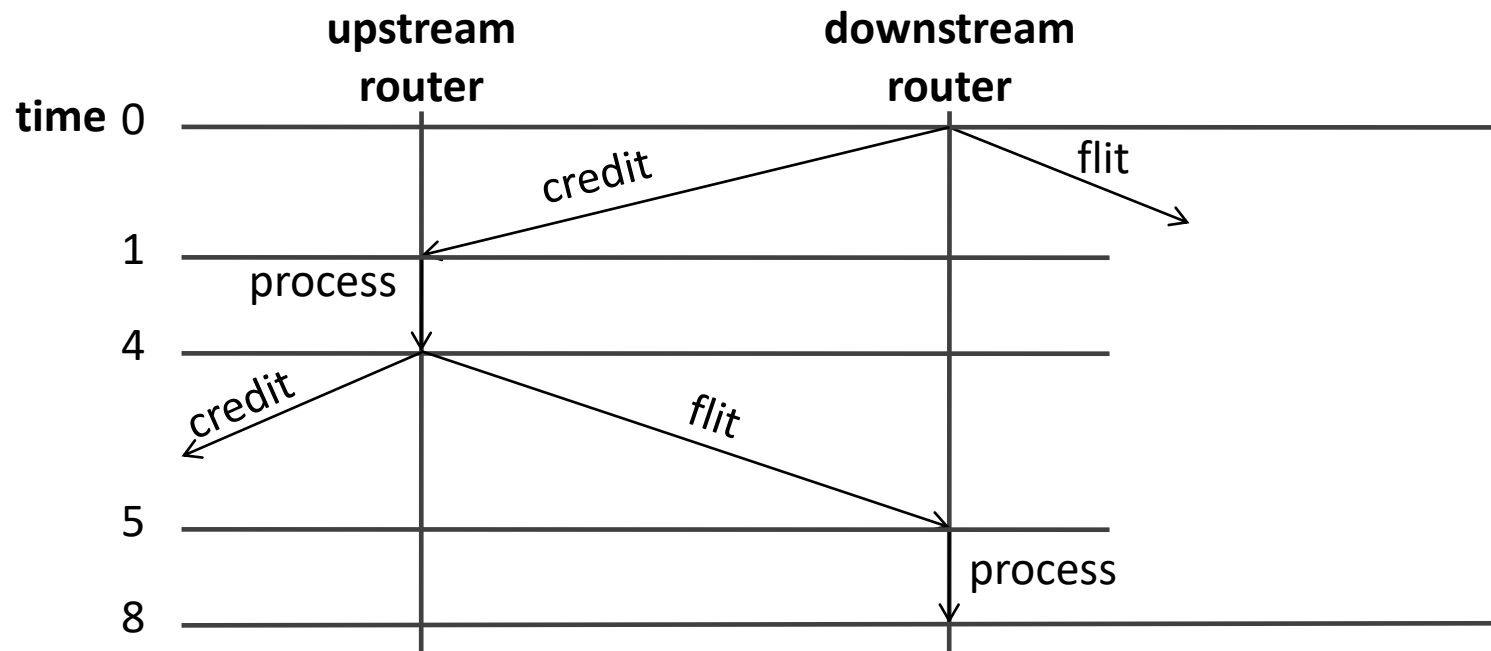
# Buffer Backpressure – Credit-Based Flow Control

e.g., Assume 3-cycle flit processing pipeline and 1-cycle link delay (data/credits)



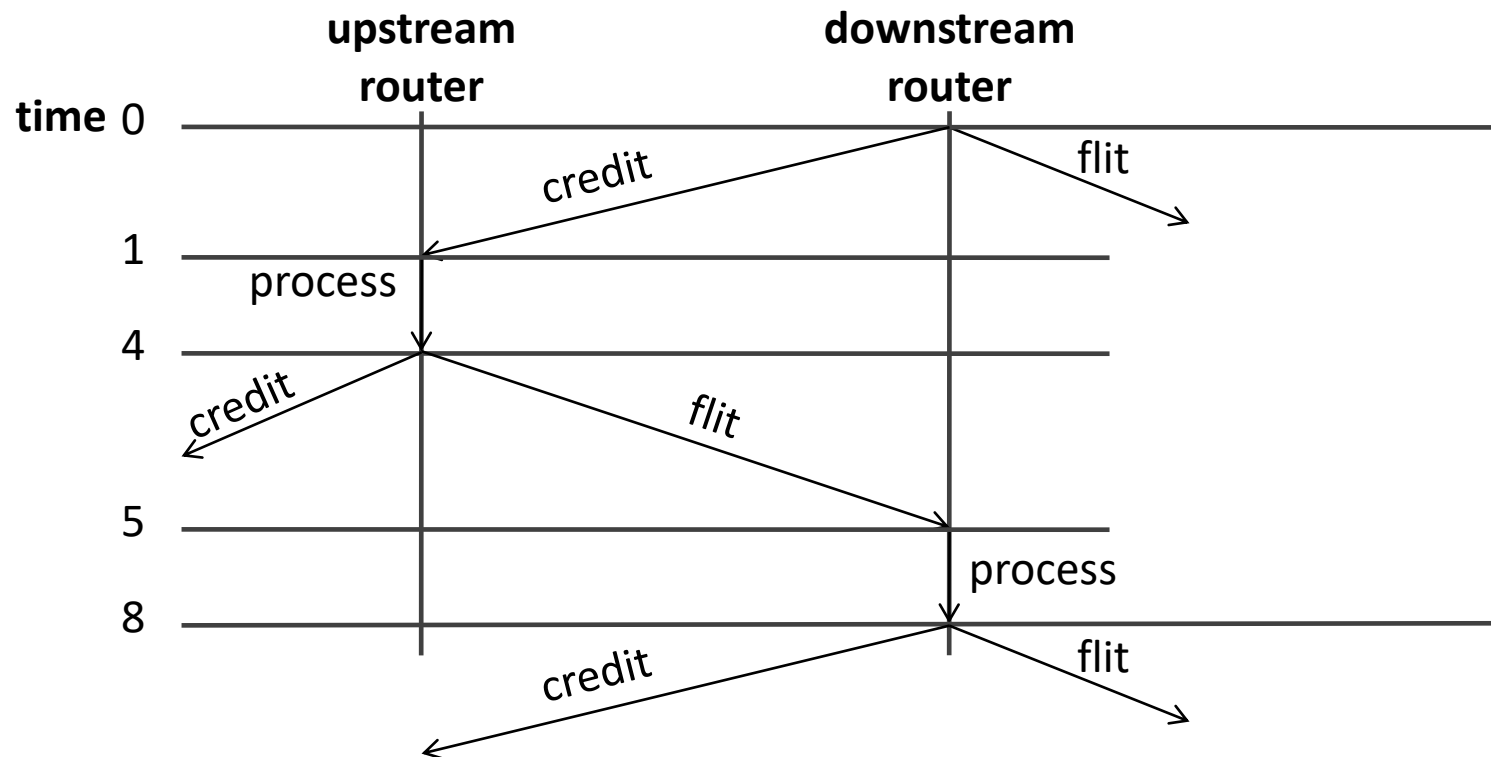
# Buffer Backpressure – Credit-Based Flow Control

e.g., Assume 3-cycle flit processing pipeline and 1-cycle link delay (data/credits)



# Buffer Backpressure – Credit-Based Flow Control

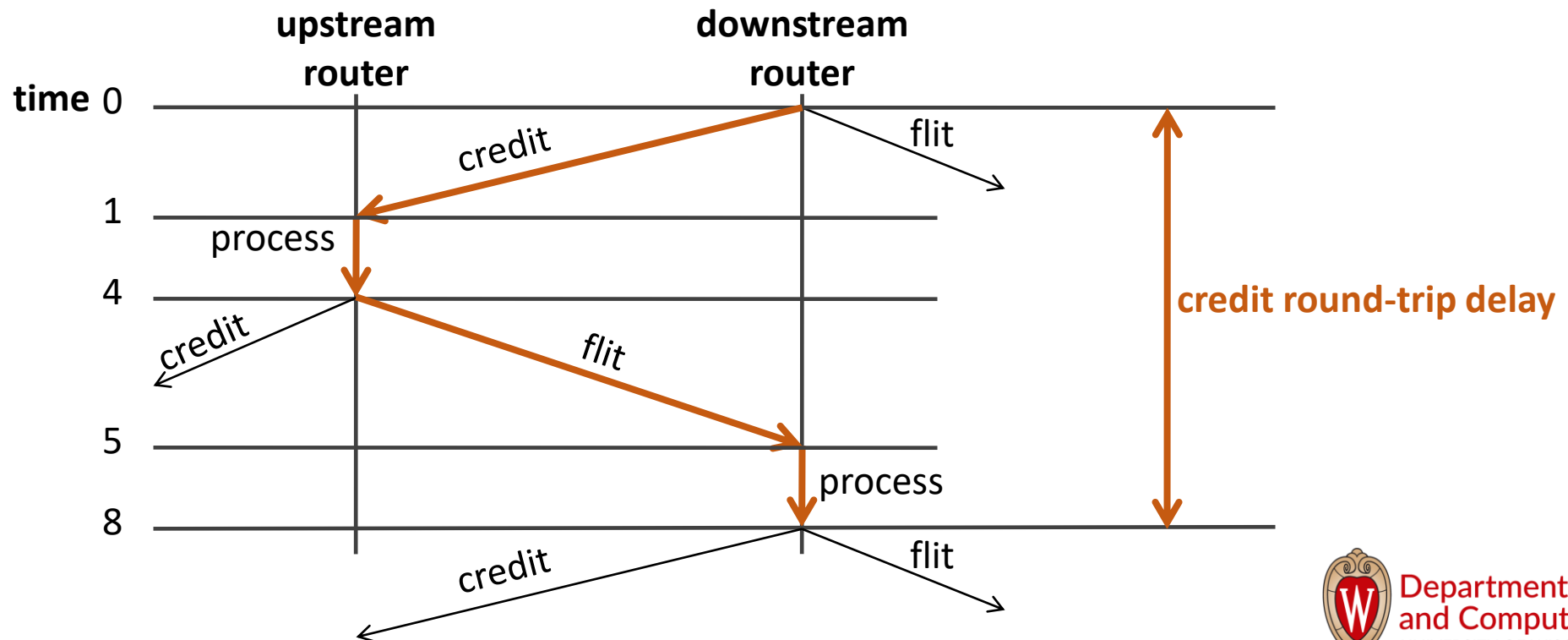
e.g., Assume 3-cycle flit processing pipeline and 1-cycle link delay (data/credits)



# Buffer Backpressure – Credit-Based Flow Control

e.g., Assume 3-cycle flit processing pipeline and 1-cycle link delay (data/credits)

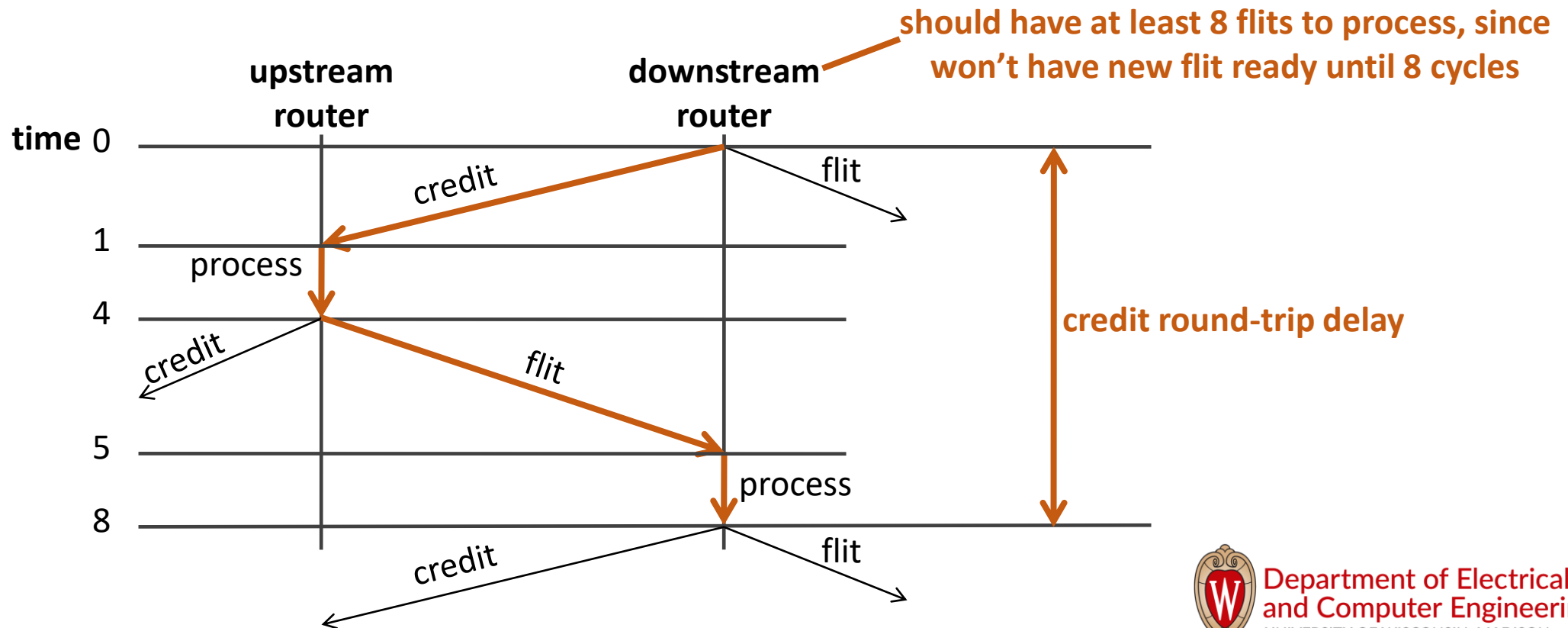
- **Credit round-trip delay:** time between credits being sent upstream for a given buffer slot
- Avoid idle cycles; buffer size should match credit round-trip delay



# Buffer Backpressure – Credit-Based Flow Control

e.g., Assume 3-cycle flit processing pipeline and 1-cycle link delay (data/credits)

- **Credit round-trip delay:** time between credits being sent upstream for a given buffer slot
- Avoid idle cycles; buffer size should match credit round-trip delay (e.g., 8 flits per buffer)



# Buffer Backpressure – On-Off Flow Control

Reduce upstream signaling; unlike credit-based flow control which requires a credit sent upstream for every flit

Off signal:

- Sent when number of free buffer slots falls below some threshold  $F_{\text{off}}$

On signal:

- Sent when number of free buffer slots rises above some threshold  $F_{\text{on}}$

# Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

- Set thresholds to trade off between amount of signaling and buffer utilization

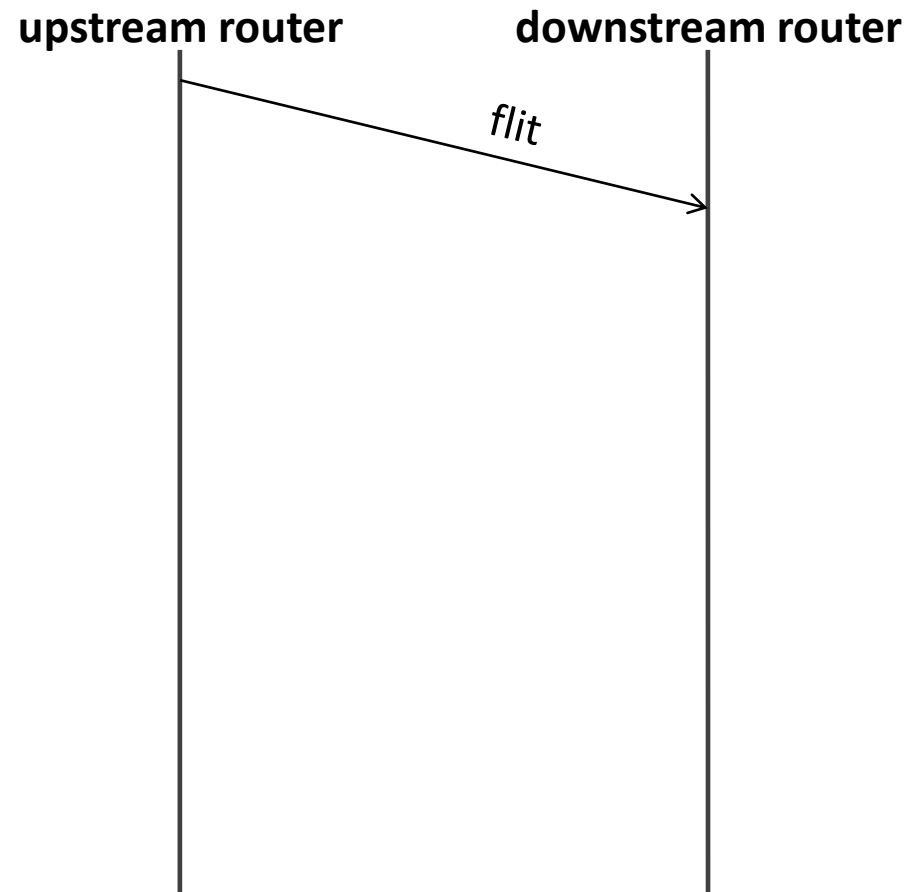
upstream router

downstream router

# Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

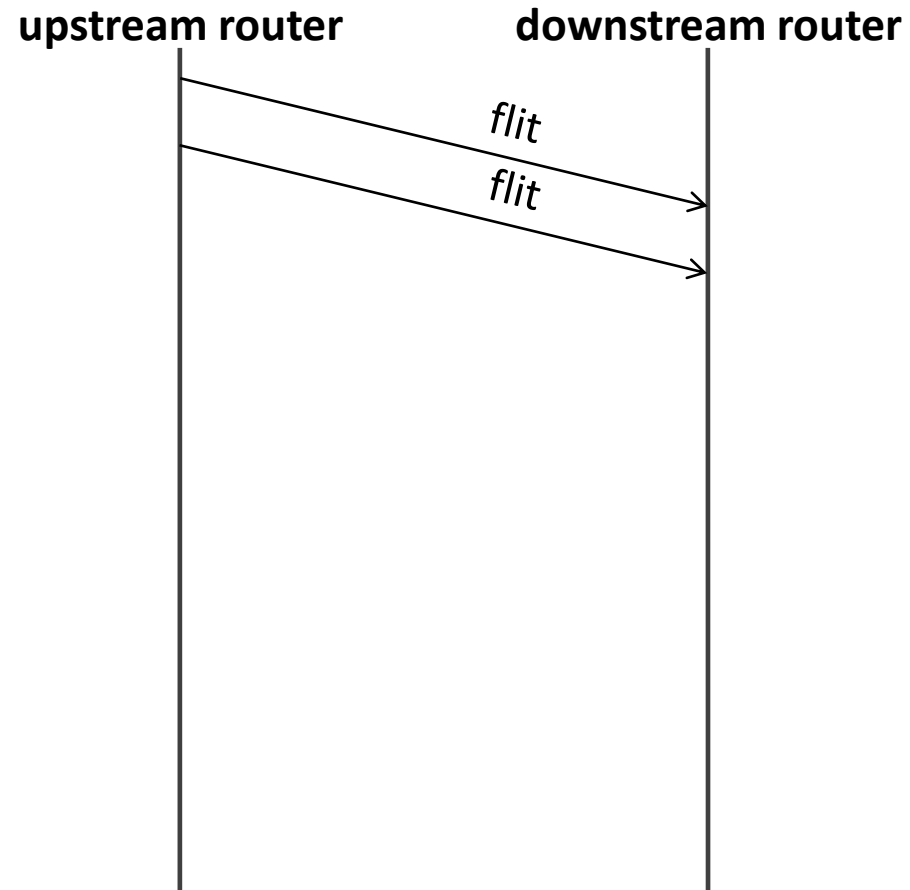
- Set thresholds to trade off between amount of signaling and buffer utilization



# Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

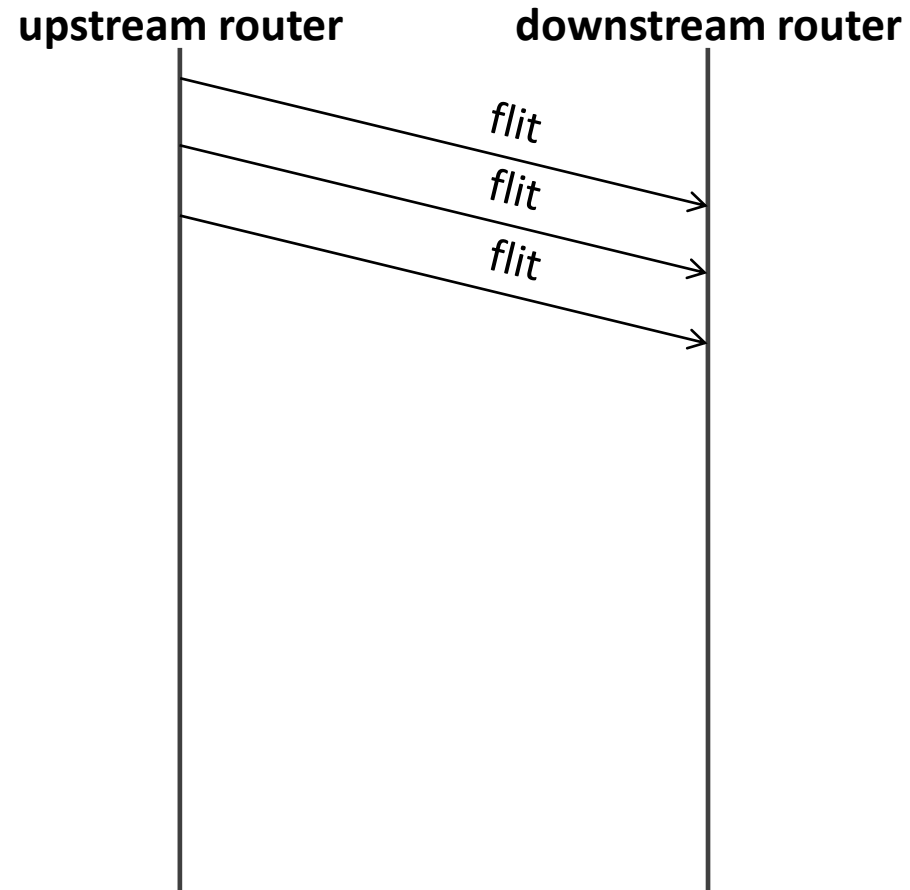
- Set thresholds to trade off between amount of signaling and buffer utilization



# Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

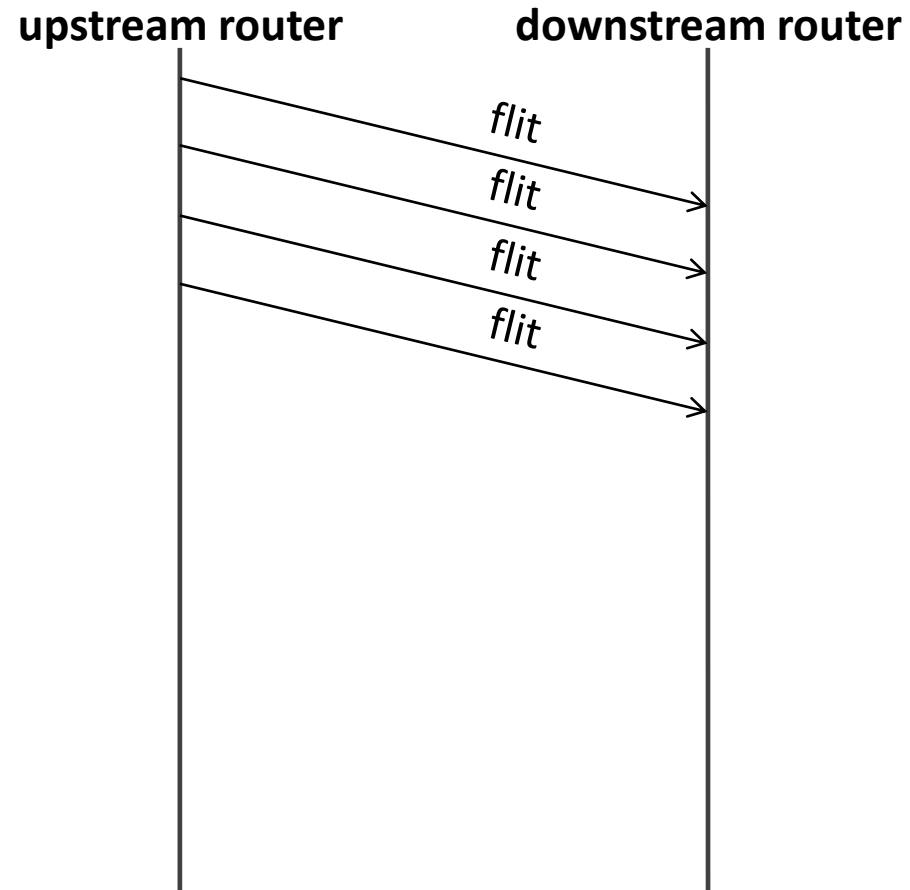
- Set thresholds to trade off between amount of signaling and buffer utilization



# Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

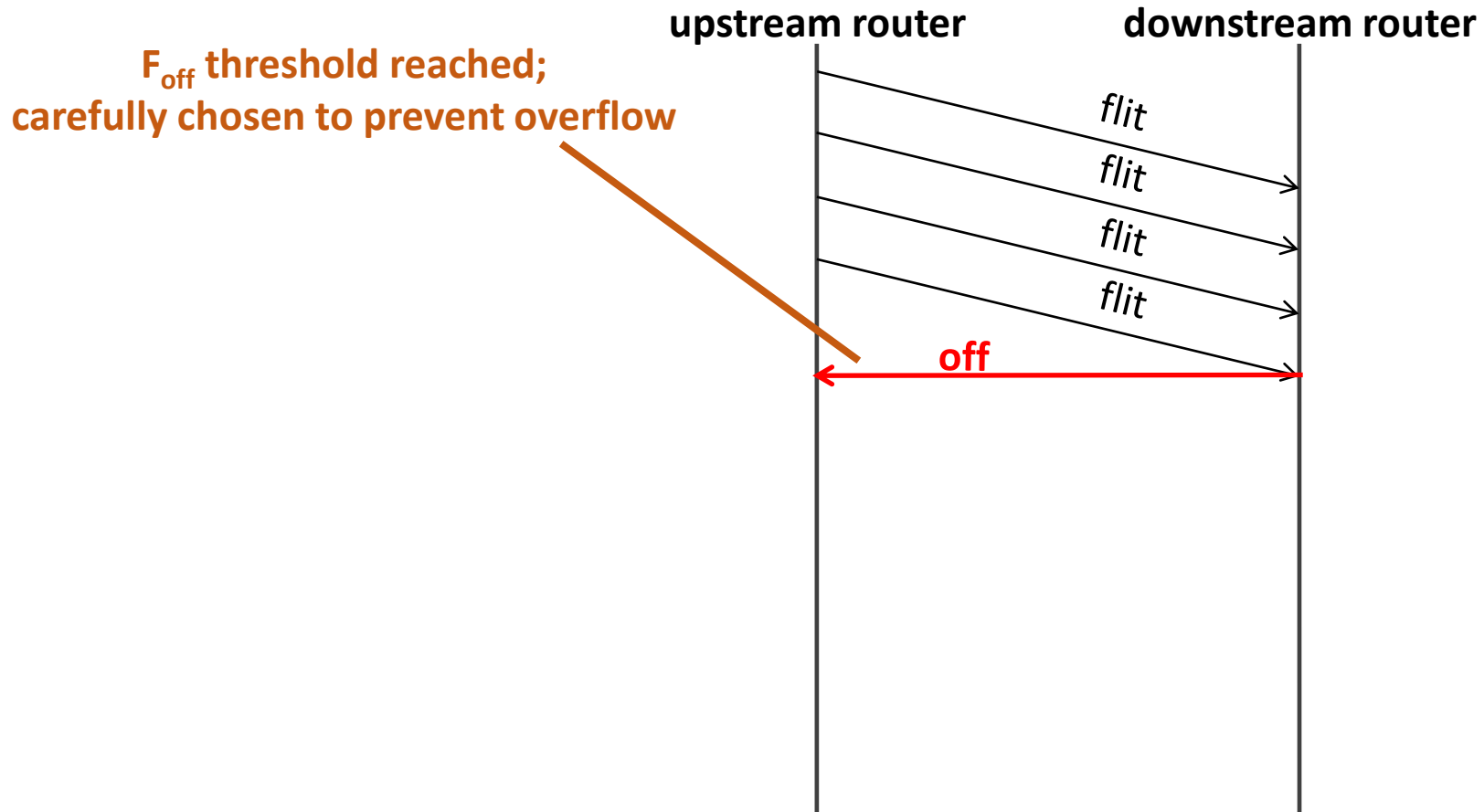
- Set thresholds to trade off between amount of signaling and buffer utilization



# Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

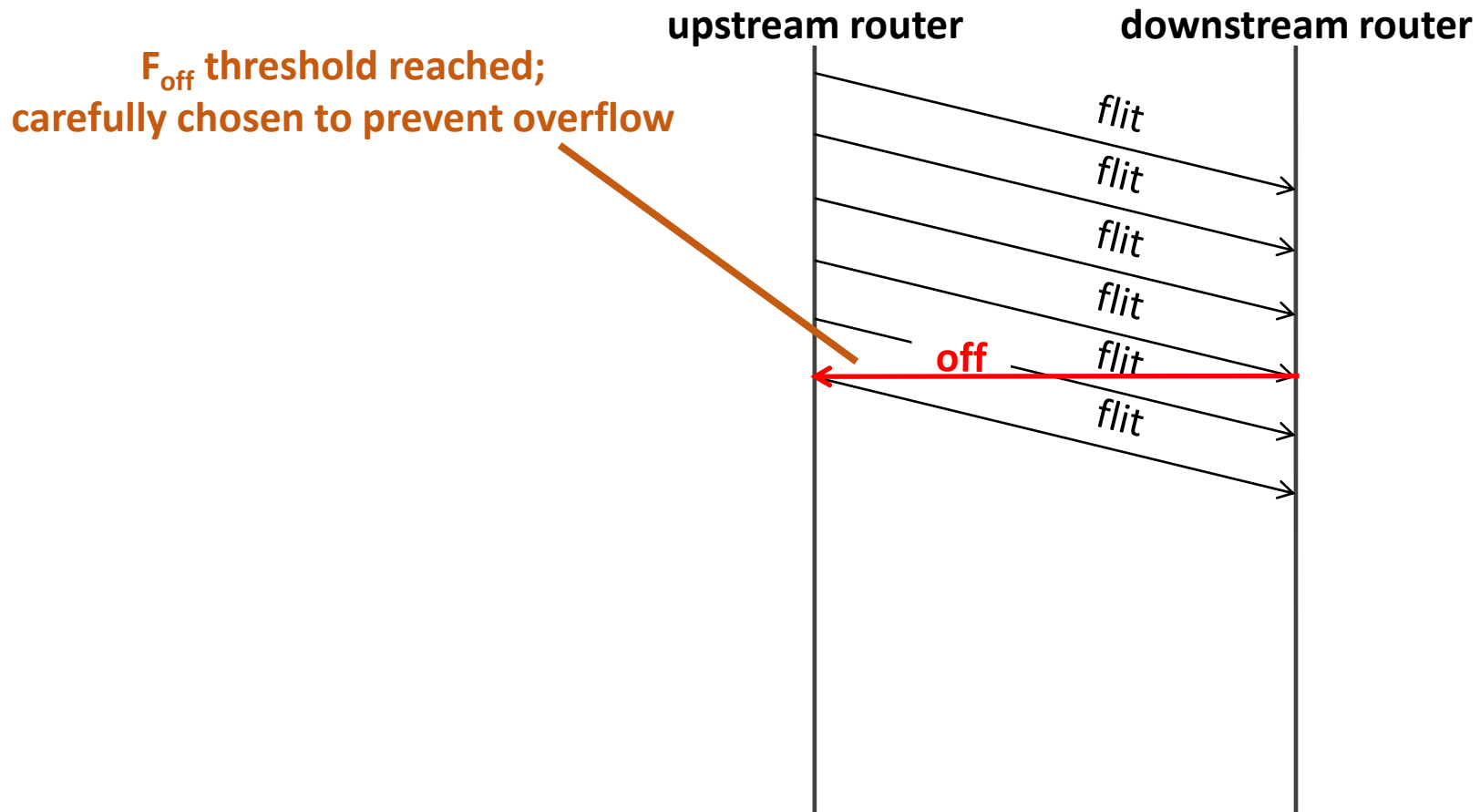
- Set thresholds to trade off between amount of signaling and buffer utilization



# Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

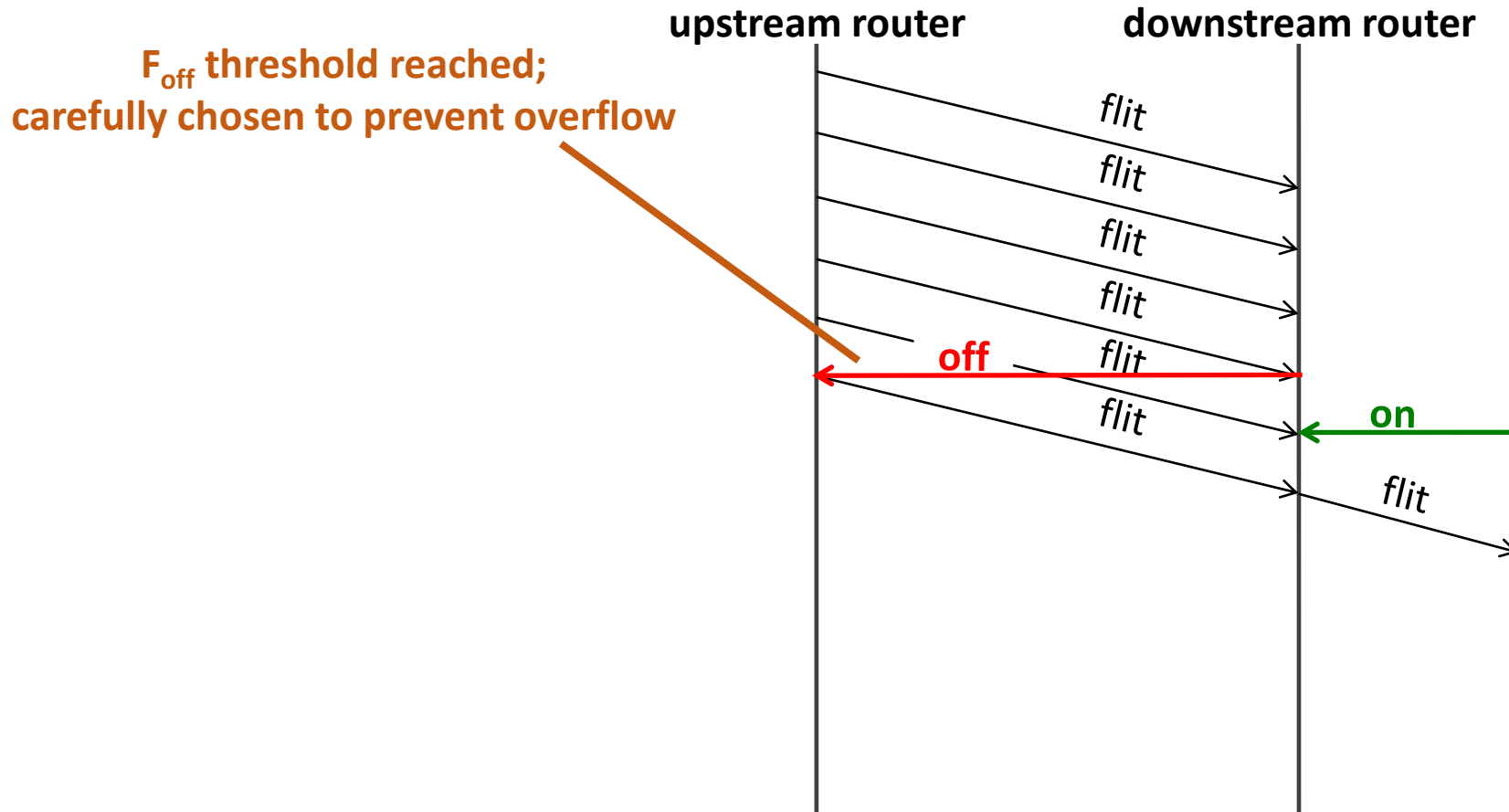
- Set thresholds to trade off between amount of signaling and buffer utilization



# Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

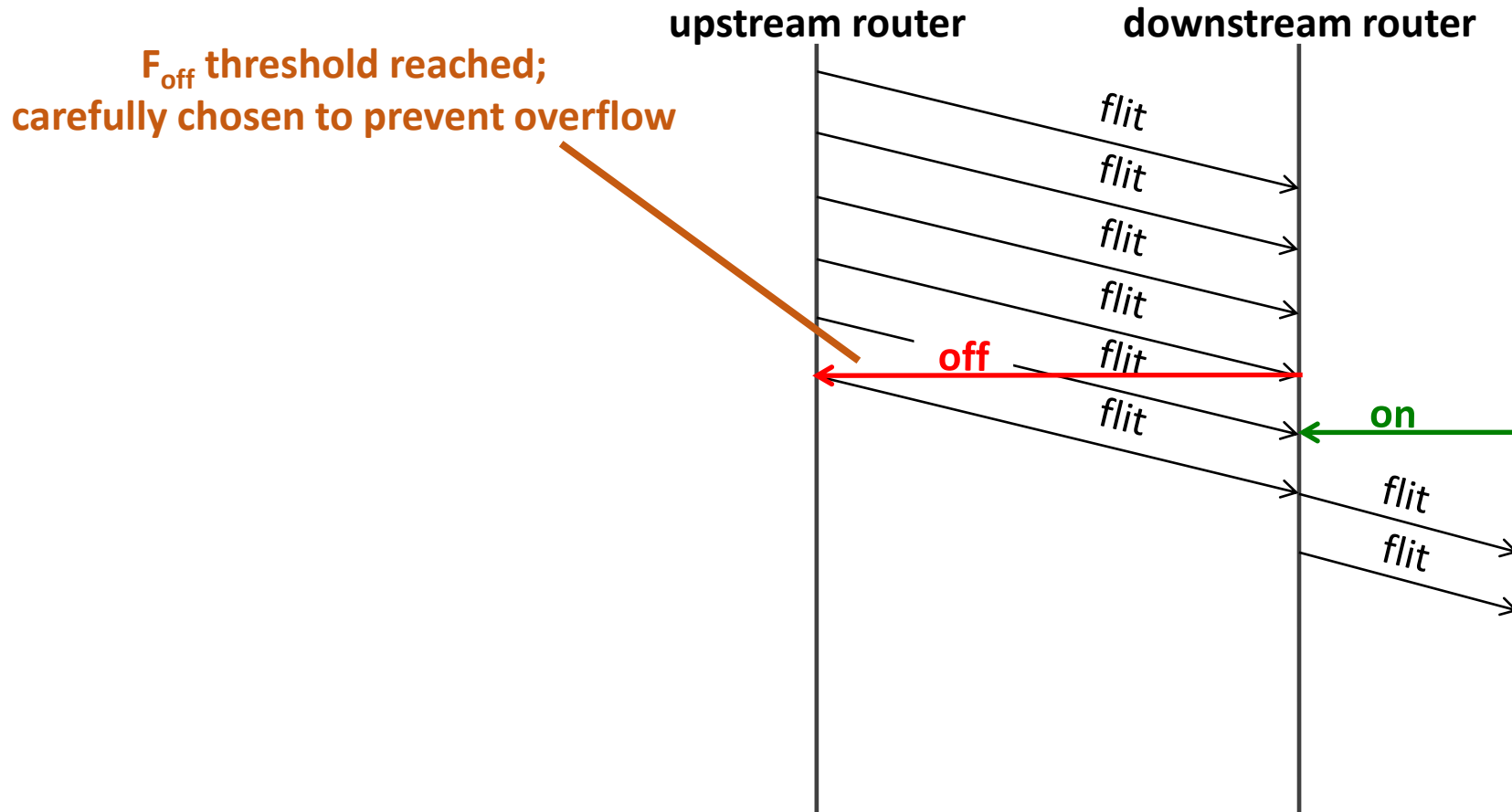
- Set thresholds to trade off between amount of signaling and buffer utilization



# Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

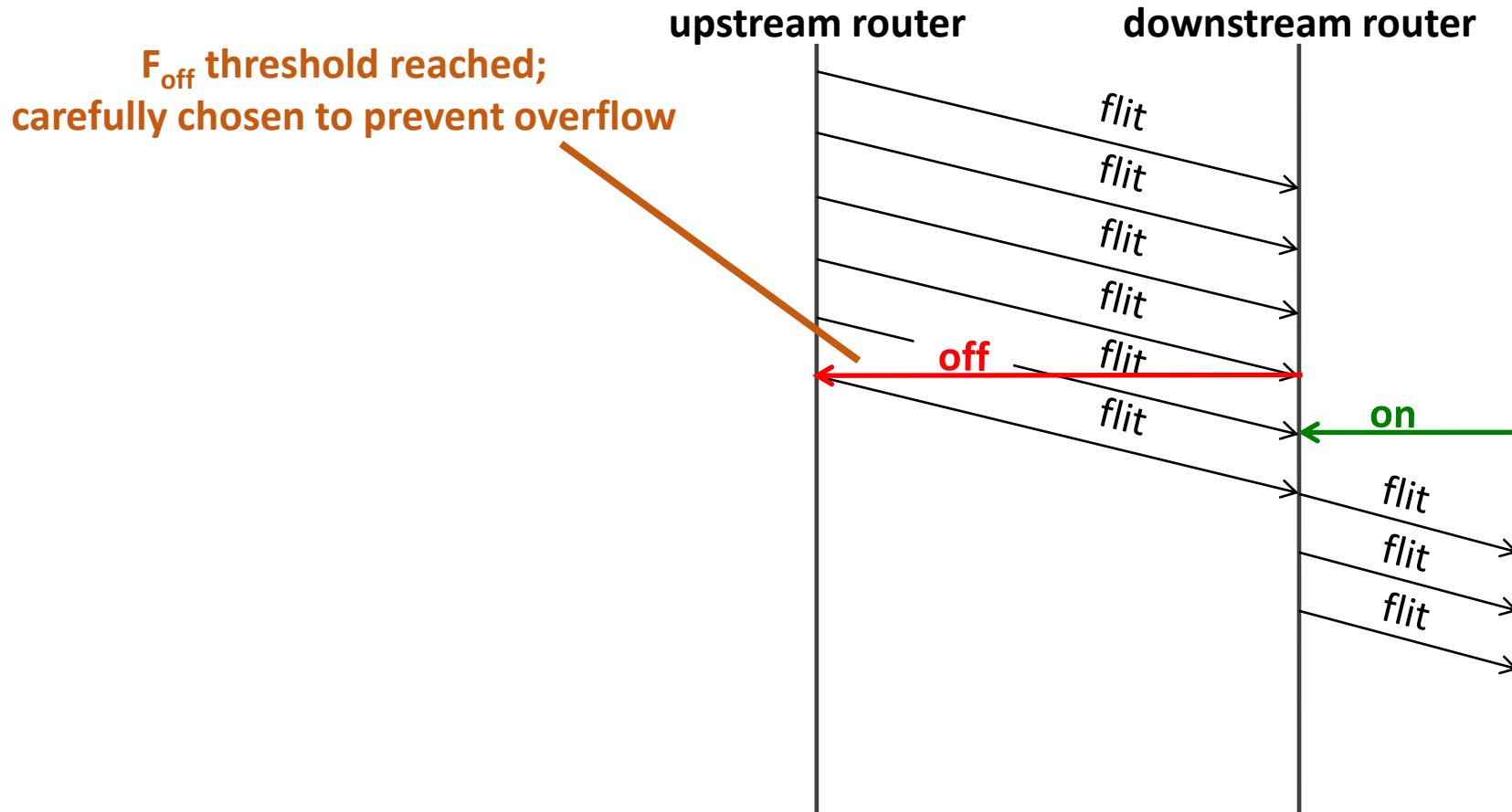
- Set thresholds to trade off between amount of signaling and buffer utilization



# Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

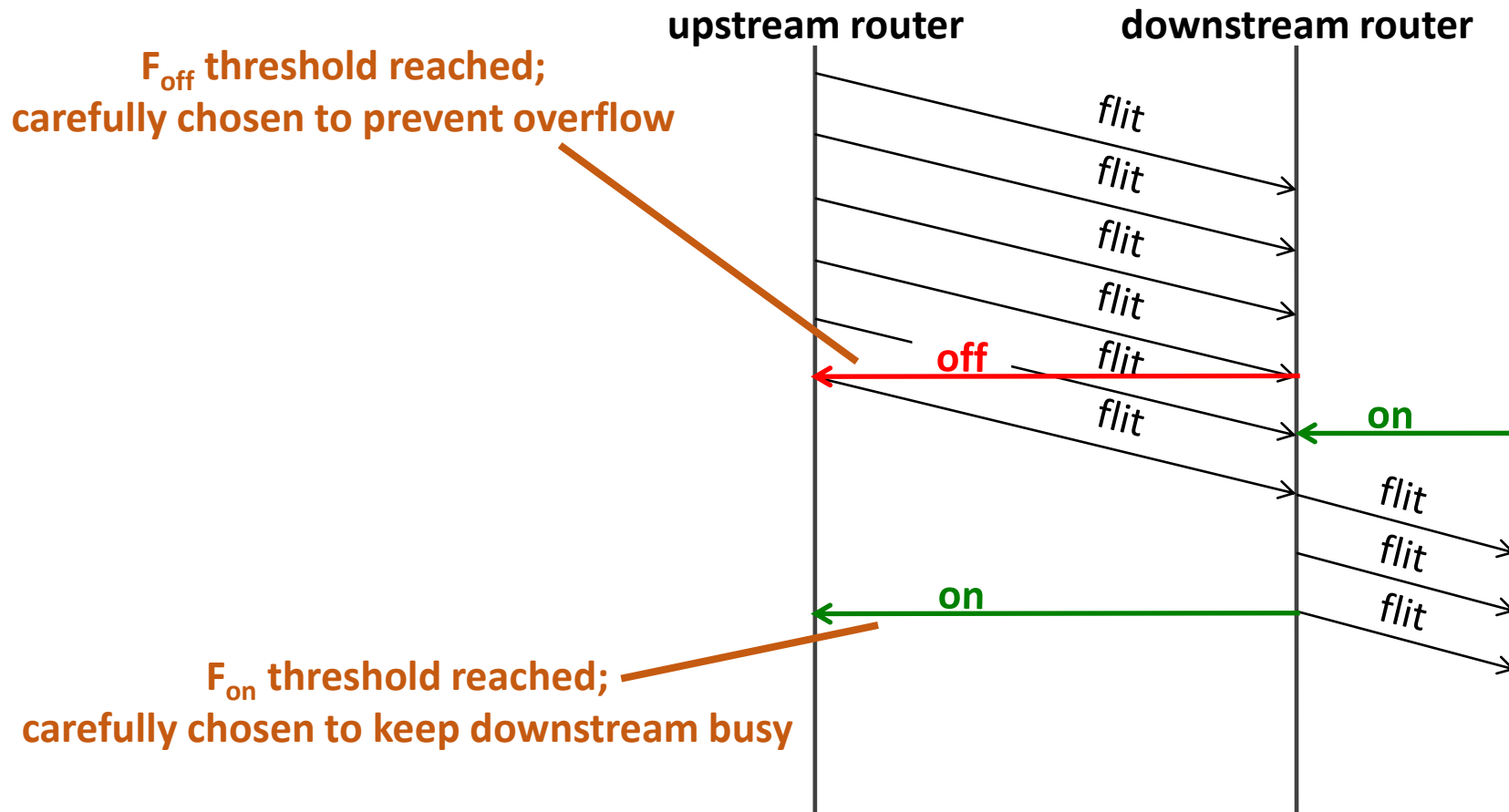
- Set thresholds to trade off between amount of signaling and buffer utilization



# Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

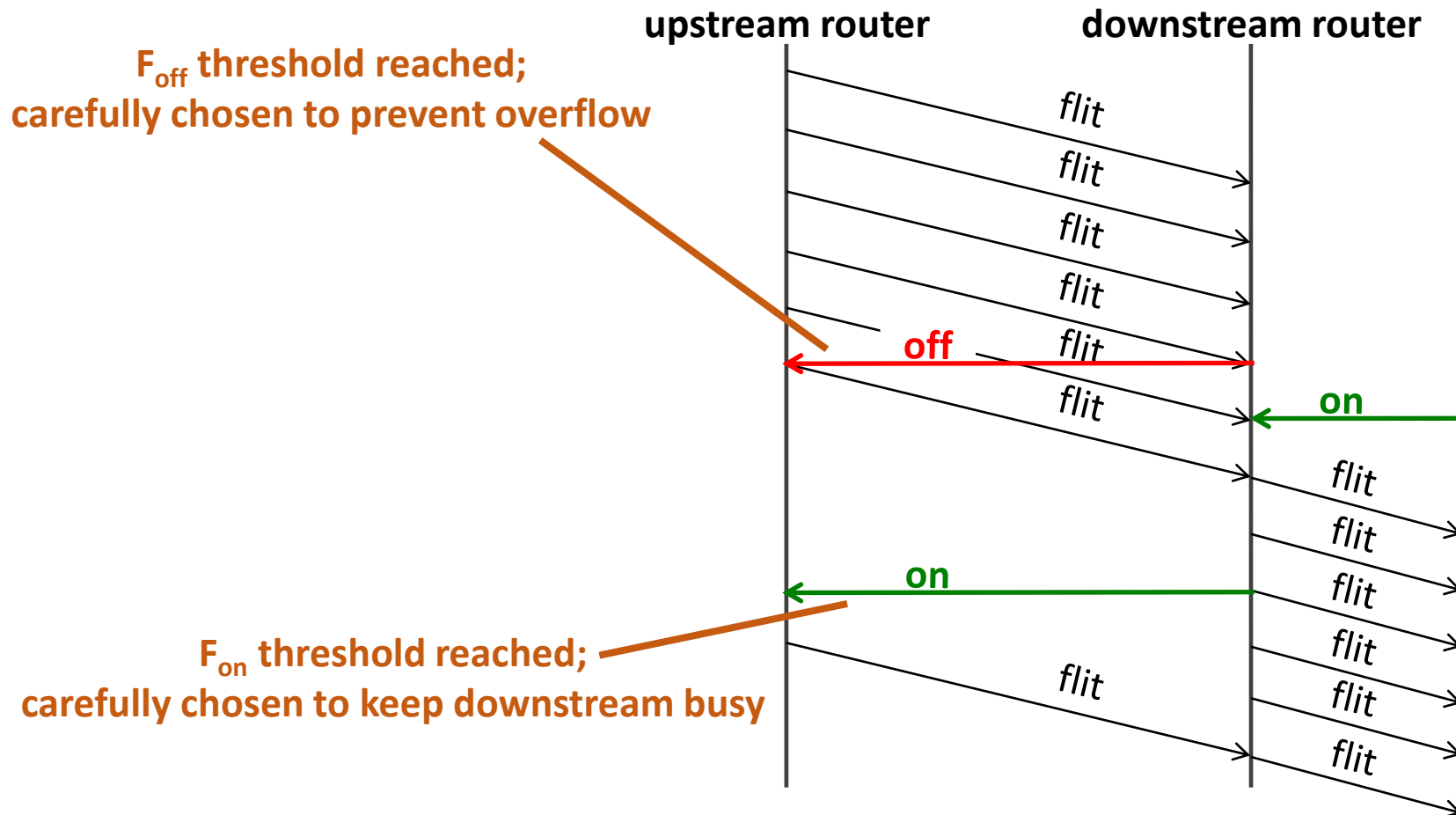
- Set thresholds to trade off between amount of signaling and buffer utilization



# Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

- Set thresholds to trade off between amount of signaling and buffer utilization



# Network-On-Chip

## Key design decisions:

### Topology

- Where to place routers/nodes and how to link them together

### Routing

- Which set of links to traverse between source-destination pairs

### Flow Control

- What granularity to move messages through links and routers

### Router Microarchitecture

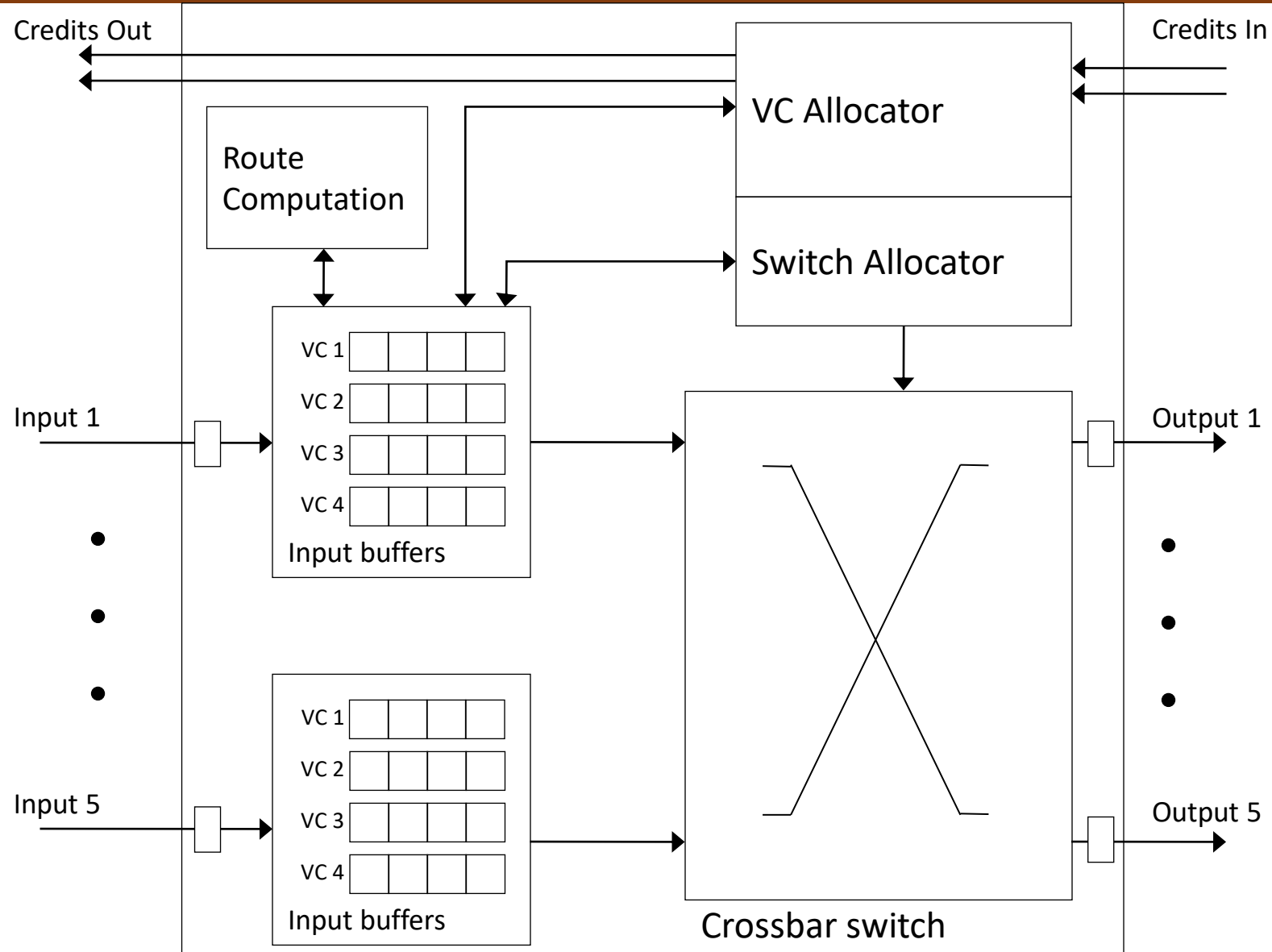
- How to pipeline and arbitrate incoming messages in routers

# Router Microarchitecture

Implementation of routing algorithm, flow control and router pipeline

- Impacts per-hop delay and energy
- Router complexity increases with bandwidth demands
- Focus on router microarchitecture with virtual channels

# Router Microarchitecture



# Router Components

Input buffers, route computation logic, virtual channel allocator, switch allocator, crossbar switch

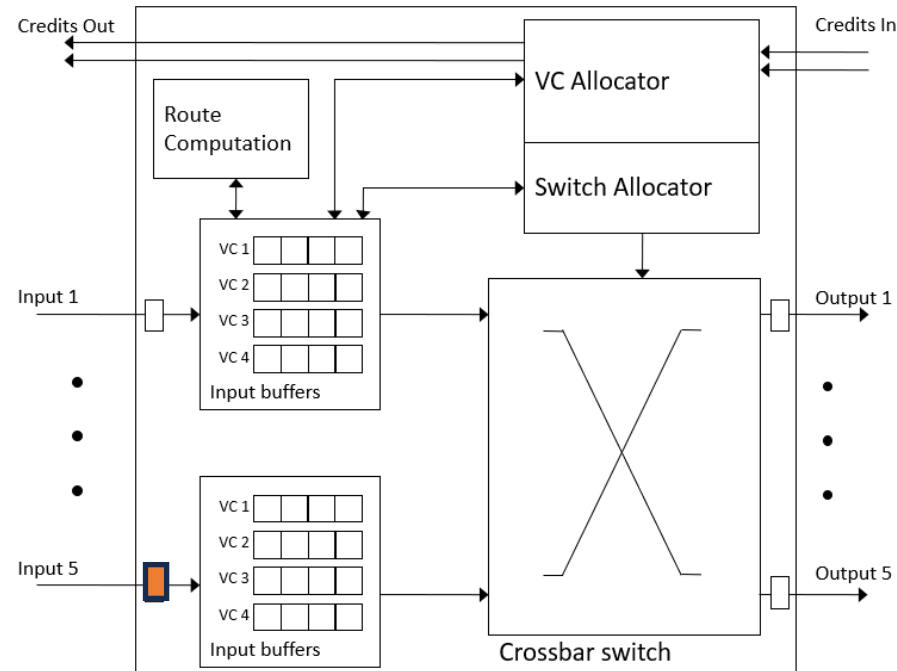
Most NoC routers are input-buffered

- Use single-ported memories
- Input buffer stores flits for duration in router

# Router Pipeline

## Canonical 5-stage pipeline

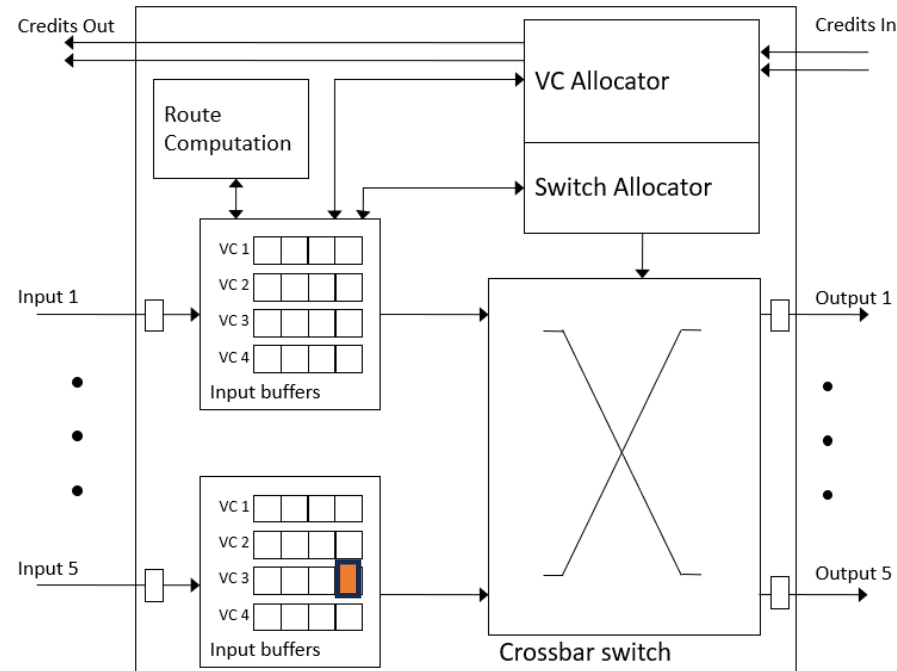
- BW: buffer write
- RC: routing computation
- VA: virtual channel allocation
- SA: switch allocation
- ST: switch traversal
- *LT: link traversal*



# Router Pipeline

## Canonical 5-stage pipeline

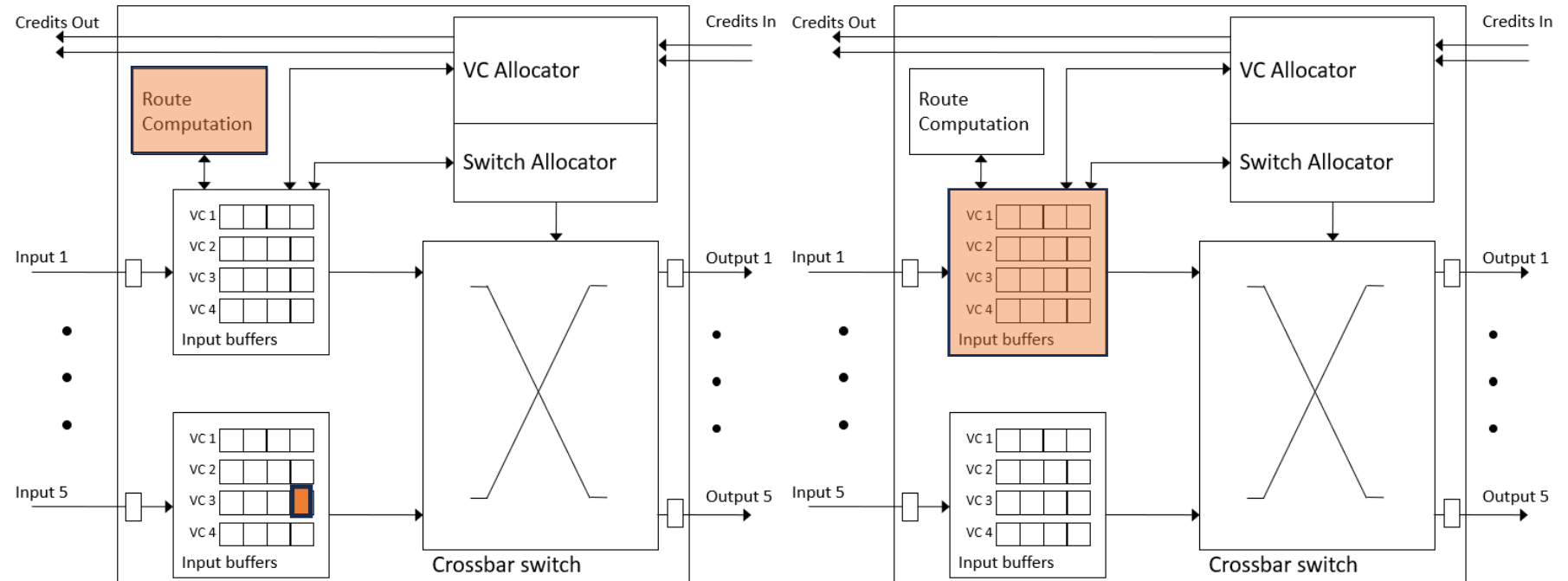
- **BW: buffer write**
- RC: routing computation
- VA: virtual channel allocation
- SA: switch allocation
- ST: switch traversal
- *LT: link traversal*



# Router Pipeline

## Canonical 5-stage pipeline

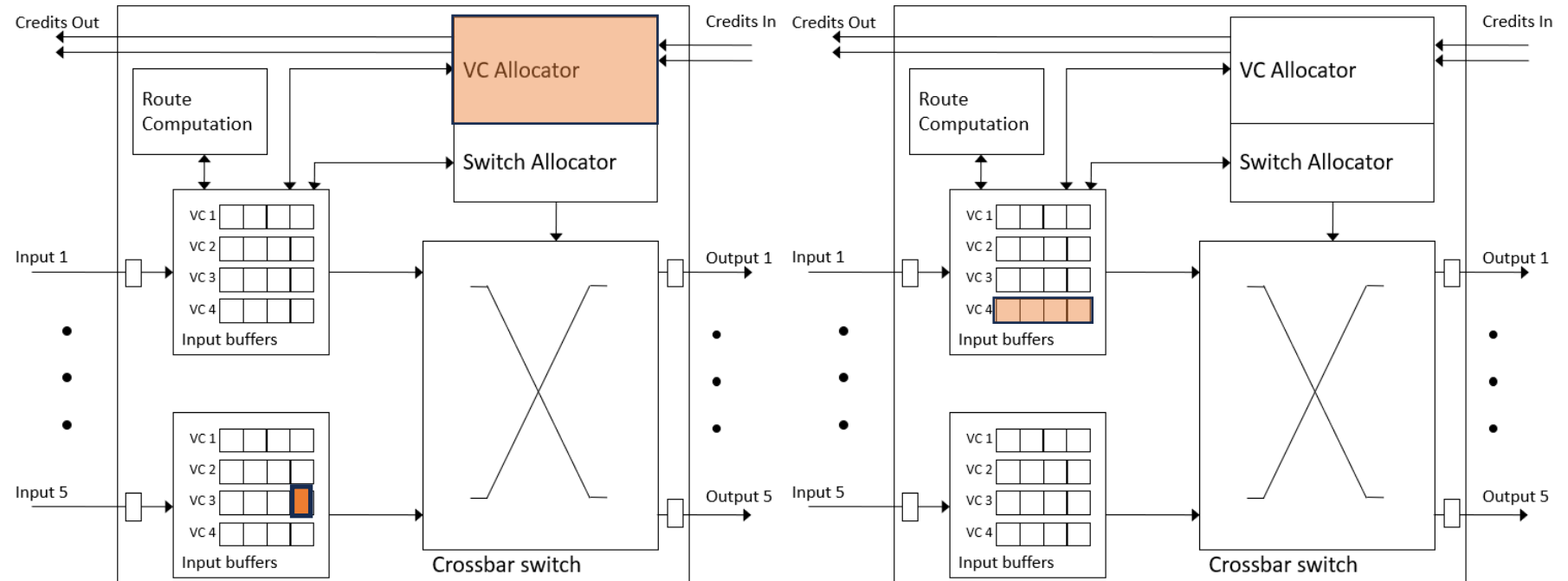
- BW: buffer write
- **RC: routing computation**
- VA: virtual channel allocation
- SA: switch allocation
- ST: switch traversal
- *LT: link traversal*



# Router Pipeline

## Canonical 5-stage pipeline

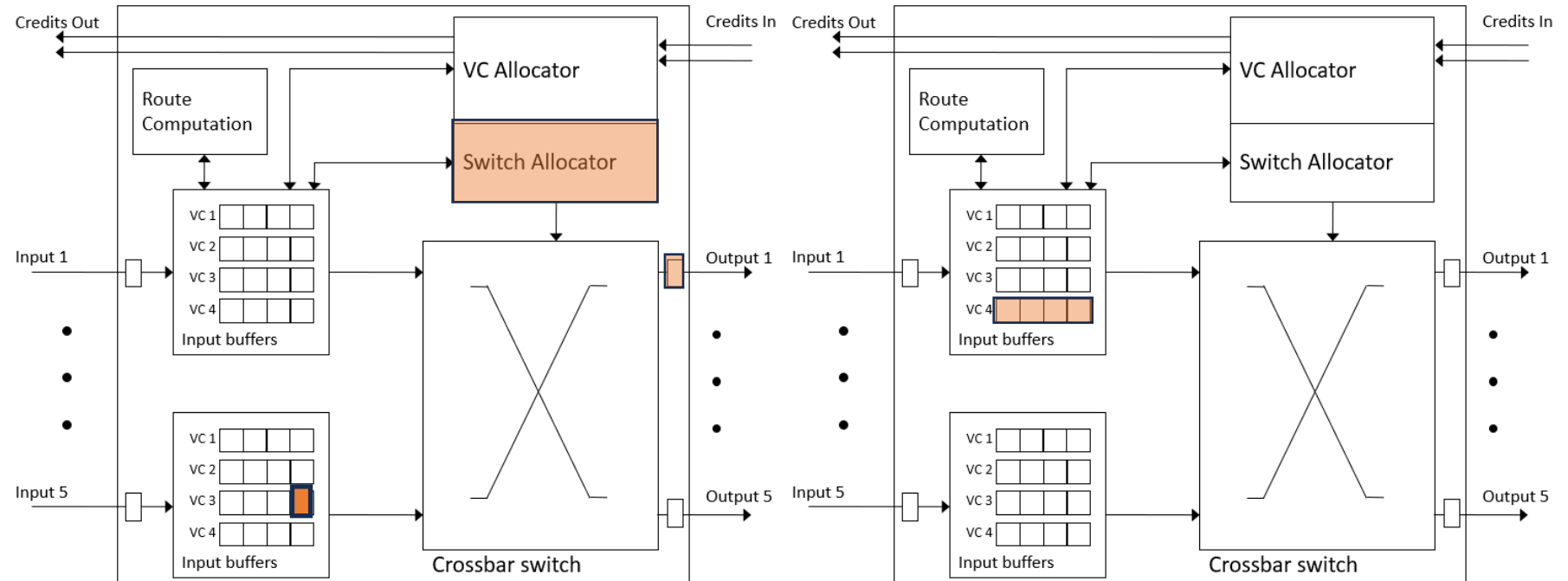
- BW: buffer write
- RC: routing computation
- **VA: virtual channel allocation**
- SA: switch allocation
- ST: switch traversal
- *LT: link traversal*



# Router Pipeline

## Canonical 5-stage pipeline

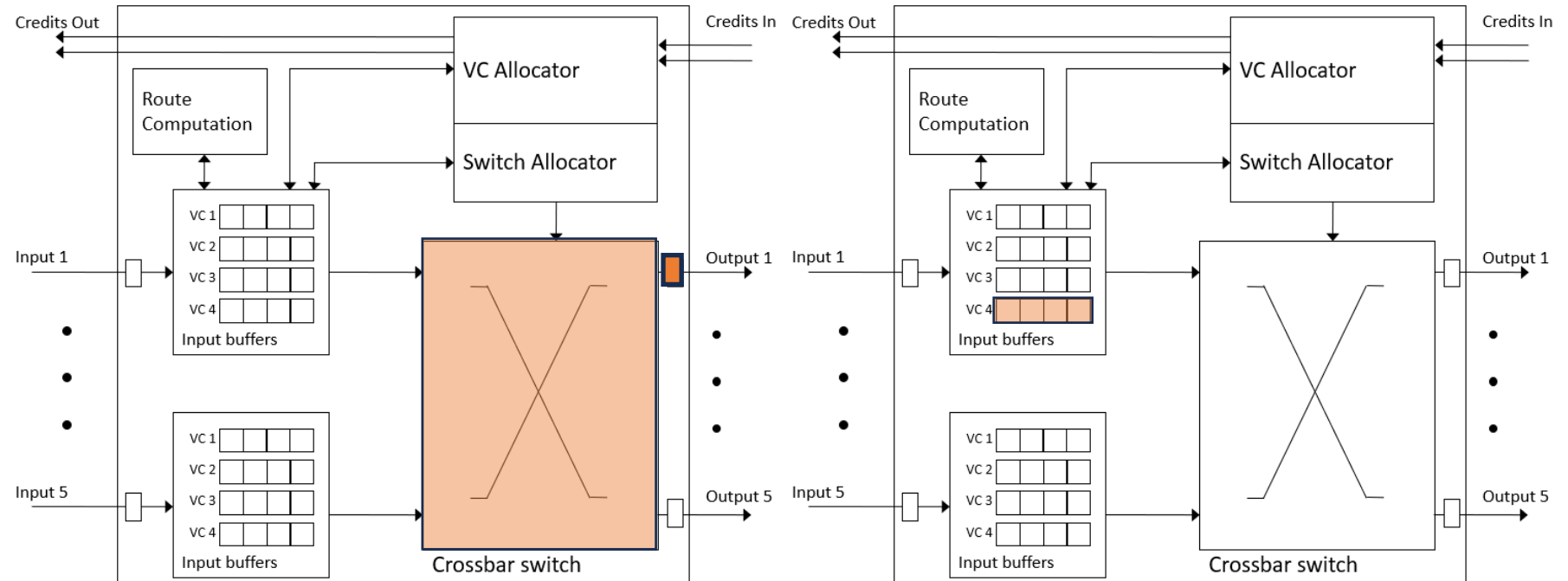
- BW: buffer write
- RC: routing computation
- VA: virtual channel allocation
- **SA: switch allocation**
- ST: switch traversal
- *LT: link traversal*



# Router Pipeline

## Canonical 5-stage pipeline

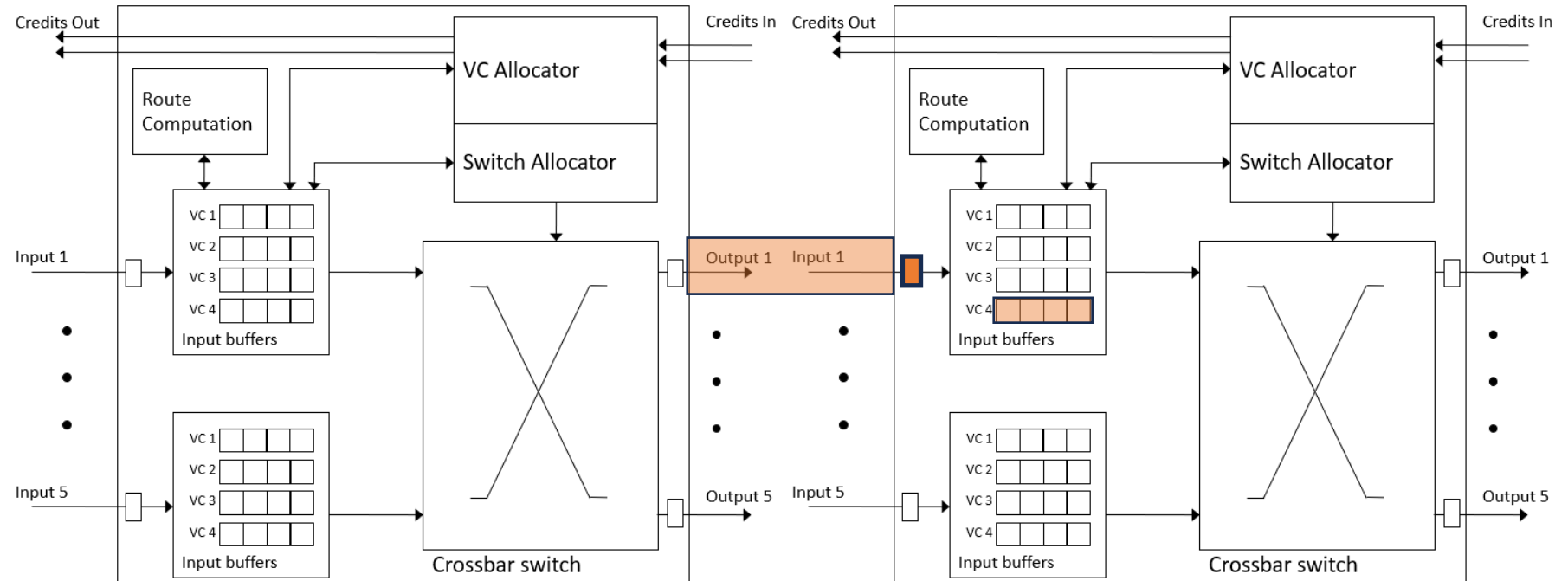
- BW: buffer write
- RC: routing computation
- VA: virtual channel allocation
- SA: switch allocation
- **ST: switch traversal**
- *LT: link traversal*



# Router Pipeline

## Canonical 5-stage pipeline

- BW: buffer write
- RC: routing computation
- VA: virtual channel allocation
- SA: switch allocation
- ST: switch traversal
- **LT: link traversal**



# Router Pipeline

Routing computation and virtual channel allocation are performed once per packet

- Body and tail flits inherit this info from head flit

| Cycle  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
|--------|----|----|----|----|----|----|----|----|----|
| Head   | BW | RC | VA | SA | ST | LT |    |    |    |
| Body 1 |    | BW |    |    | SA | ST | LT |    |    |
| Body 2 |    |    | BW |    |    | SA | ST | LT |    |
| Tail   |    |    |    | BW |    |    | SA | ST | LT |

# Router Pipeline

Zero-load packet latency with baseline wormhole VC router

- Ideally, only pay link delay

$$= (5 \text{ cycles} + \text{link delay}) \times \text{hops} + t_{\text{serialization}}$$

# Pipeline Optimizations – Lookahead Routing

At current router, perform routing computation for next router

- Overlap RC (for next hop) with one of the stages in the current hop
- Pre-computing route allows flits to compete for VCs immediately after BW

|    |          |    |    |    |
|----|----------|----|----|----|
| BW | RC<br>VA | SA | ST | LT |
|----|----------|----|----|----|

# Pipeline Optimizations – Speculation

Assume virtual channel allocation will be successful

- Typically true under low to moderate network loads
- Perform VA and SA in parallel
- If VA unsuccessful (i.e., no VC returned), try again in next cycle
- Prioritize non-speculative requests

|    |                |    |    |
|----|----------------|----|----|
| BW | RC<br>VA<br>SA | ST | LT |
|----|----------------|----|----|

# Pipeline Optimizations – Bypassing

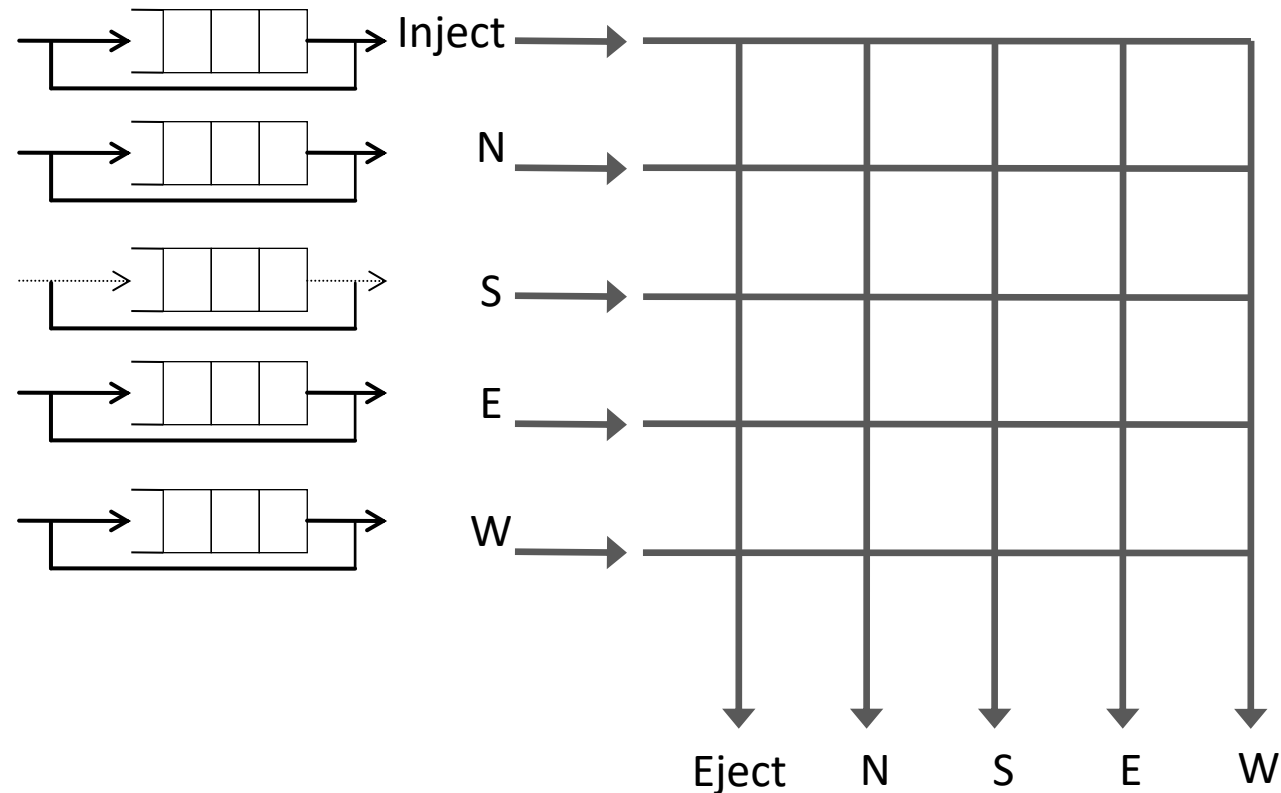
When no flits in input buffer, speculatively try to enter ST

- In the first stage, a free VC is allocated, then routing is performed and the crossbar is set up
- On port conflict, speculation canceled

|                |    |    |
|----------------|----|----|
| RC<br>VA<br>SA | ST | LT |
|----------------|----|----|

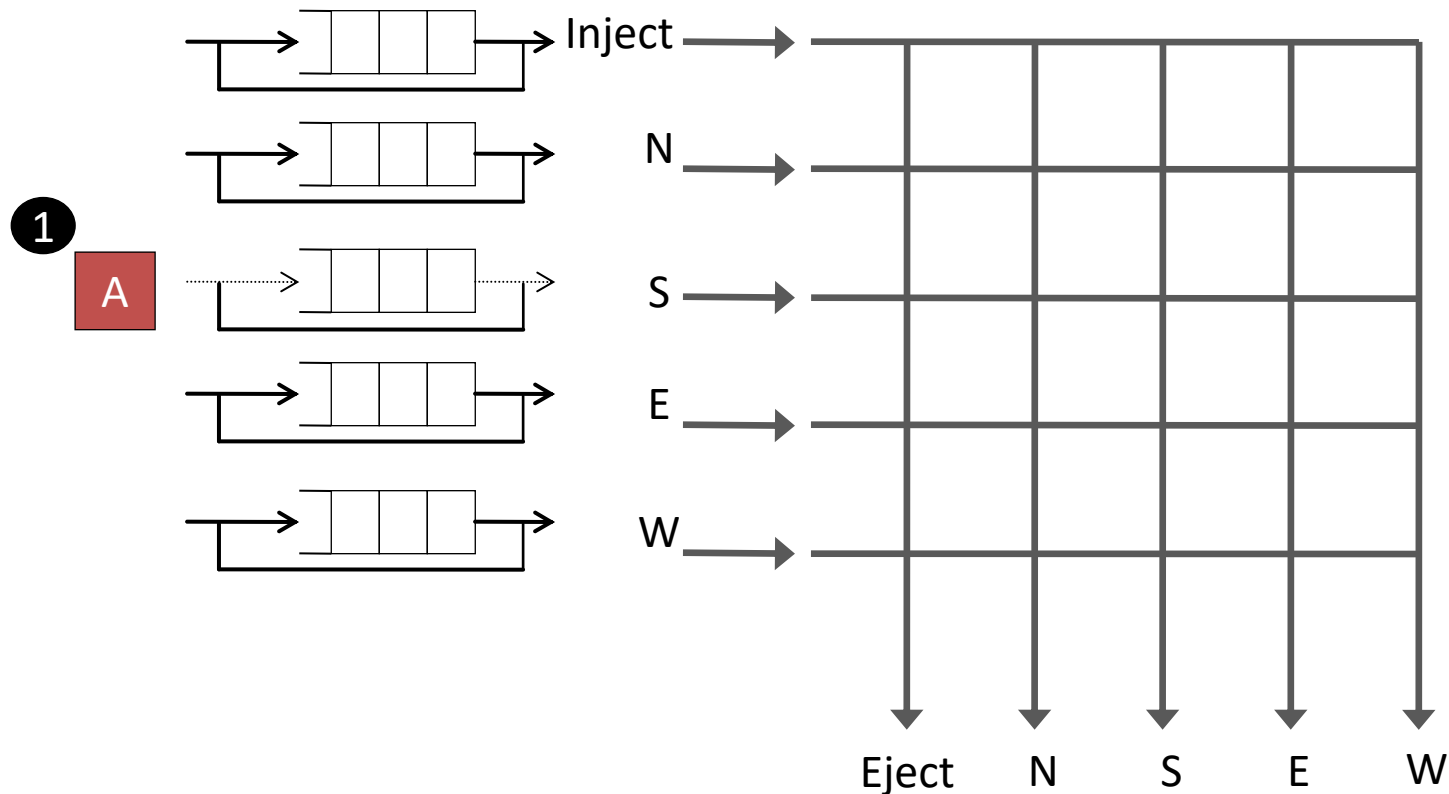
# Pipeline Optimizations – Example 1

e.g., No buffered flits when packet A arrives



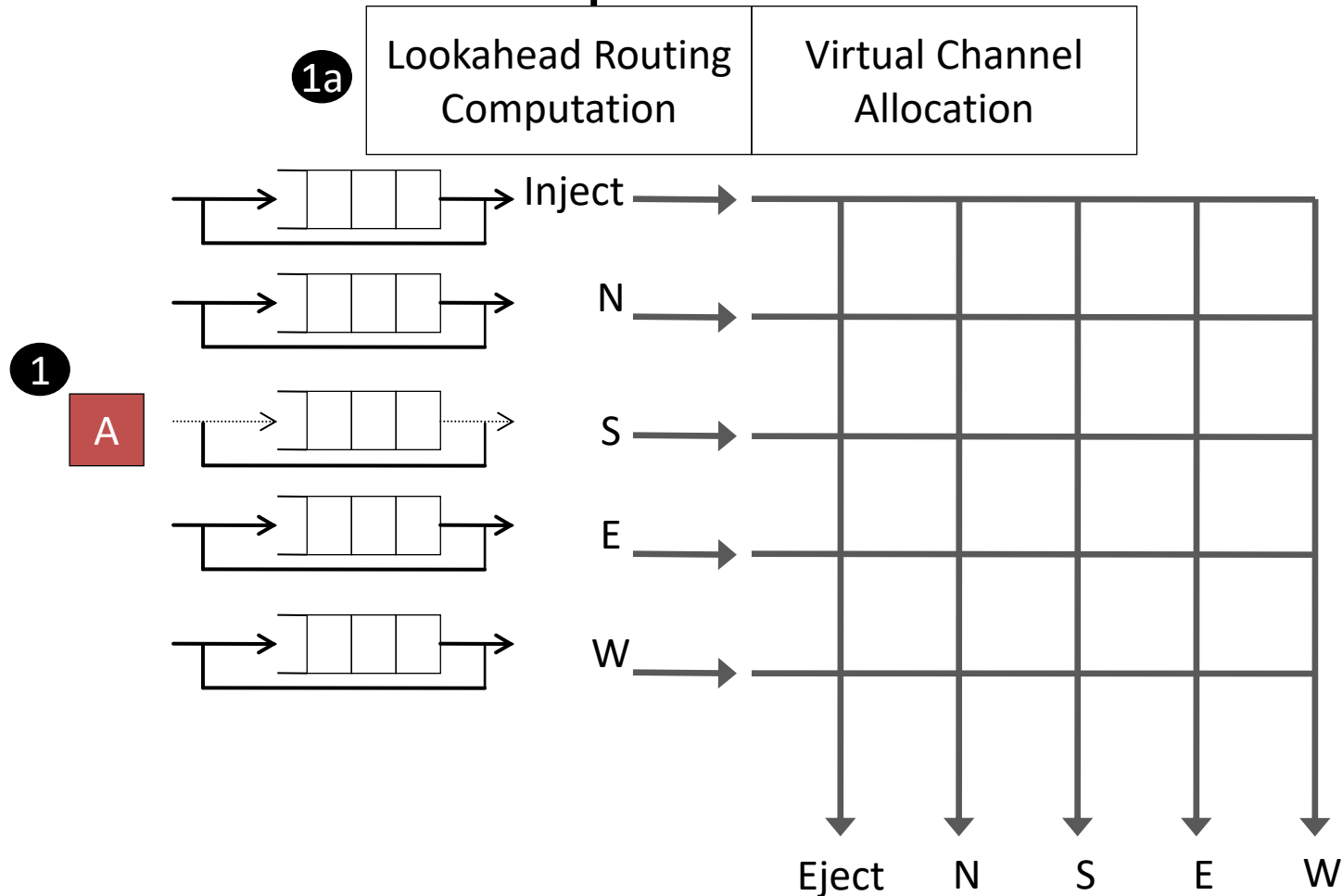
# Pipeline Optimizations – Example 1

e.g., No buffered flits when packet A arrives



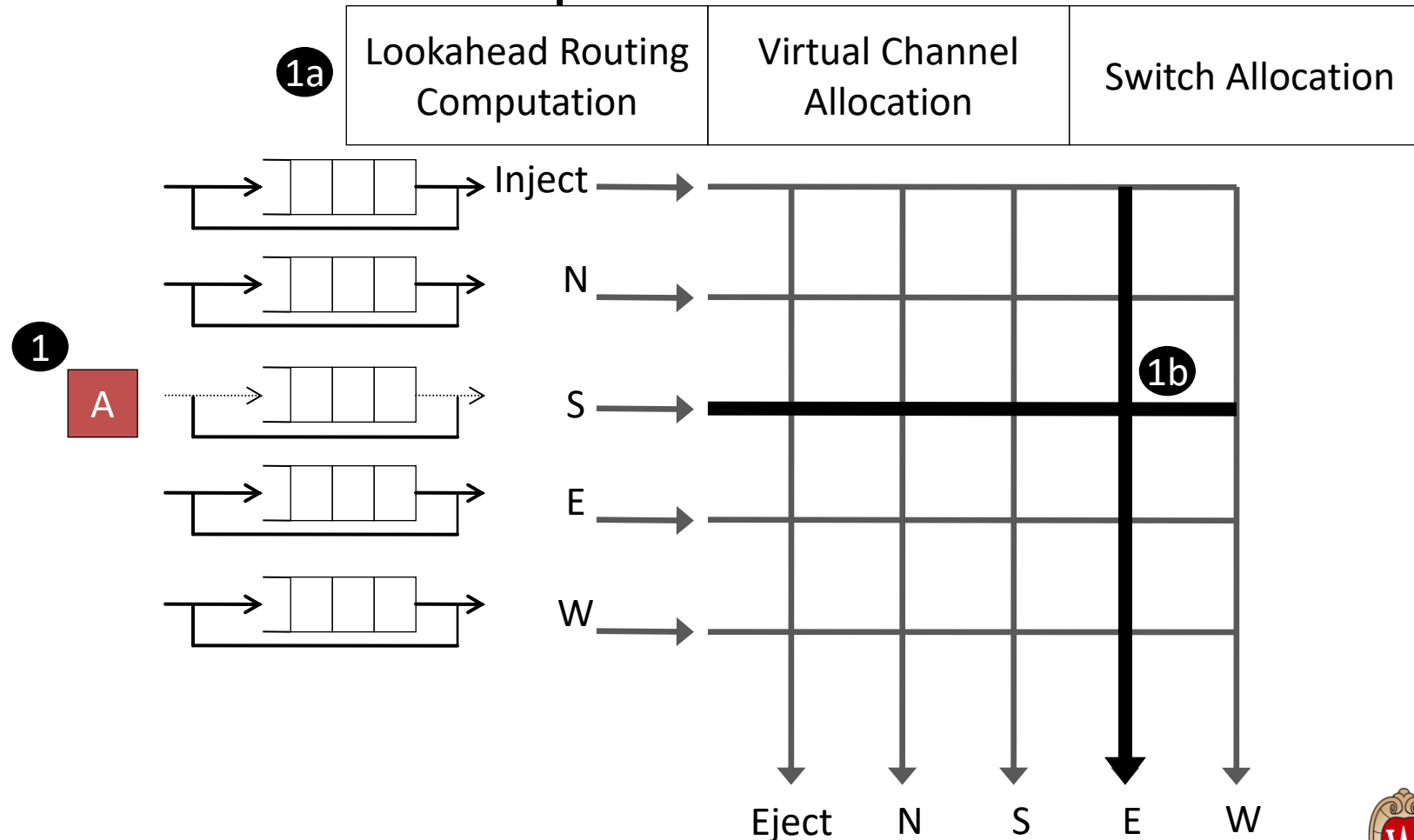
# Pipeline Optimizations – Example 1

e.g., No buffered flits when packet A arrives



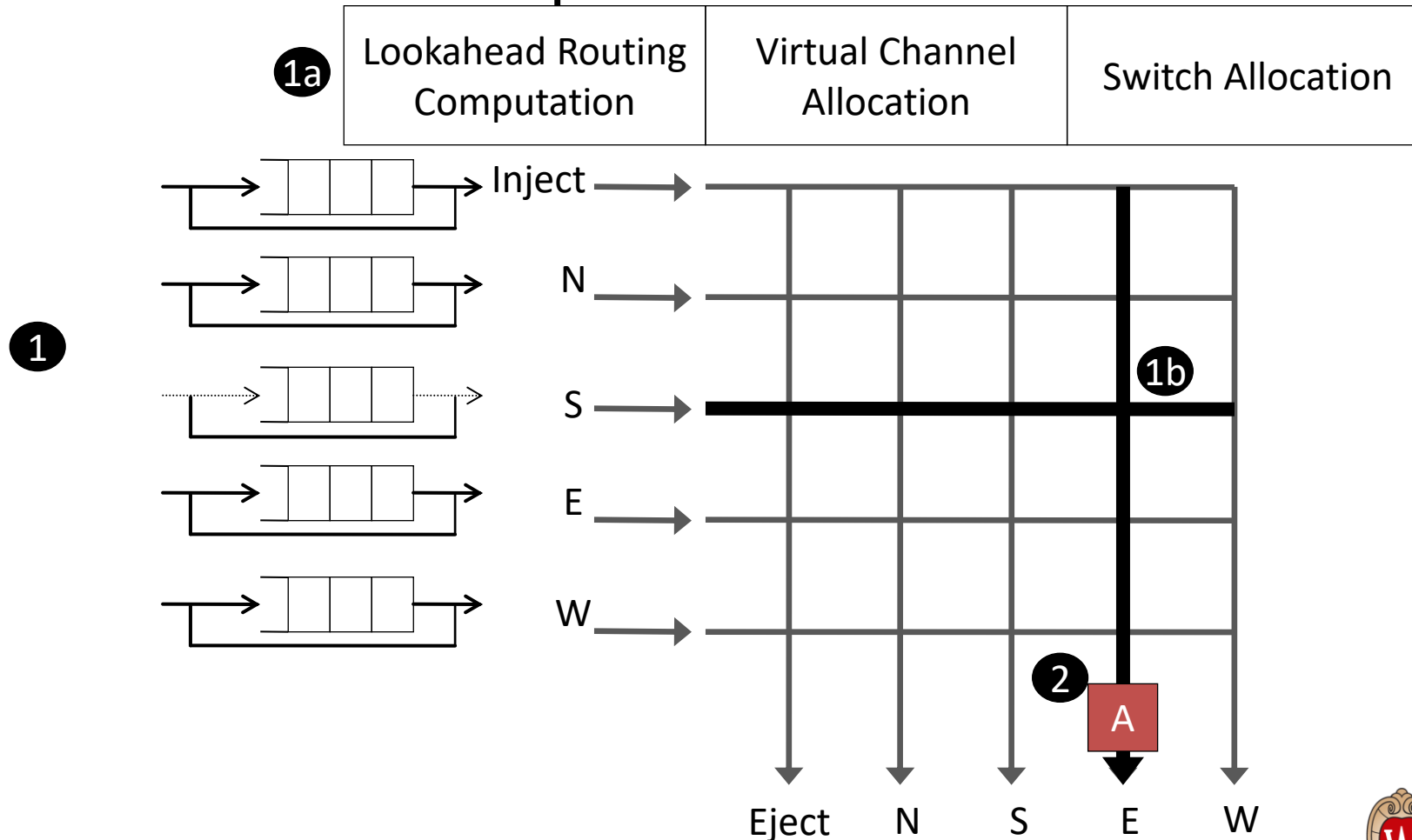
# Pipeline Optimizations – Example 1

e.g., No buffered flits when packet A arrives



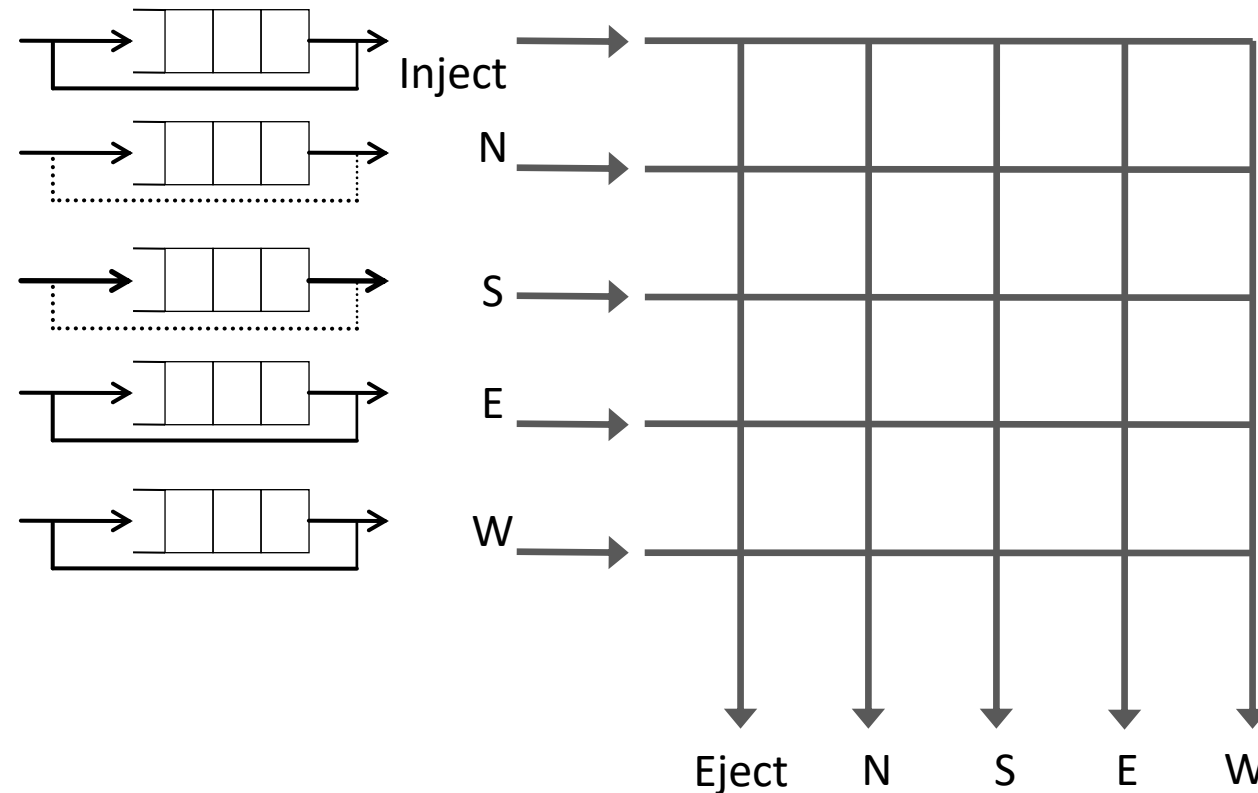
# Pipeline Optimizations – Example 1

e.g., No buffered flits when packet A arrives



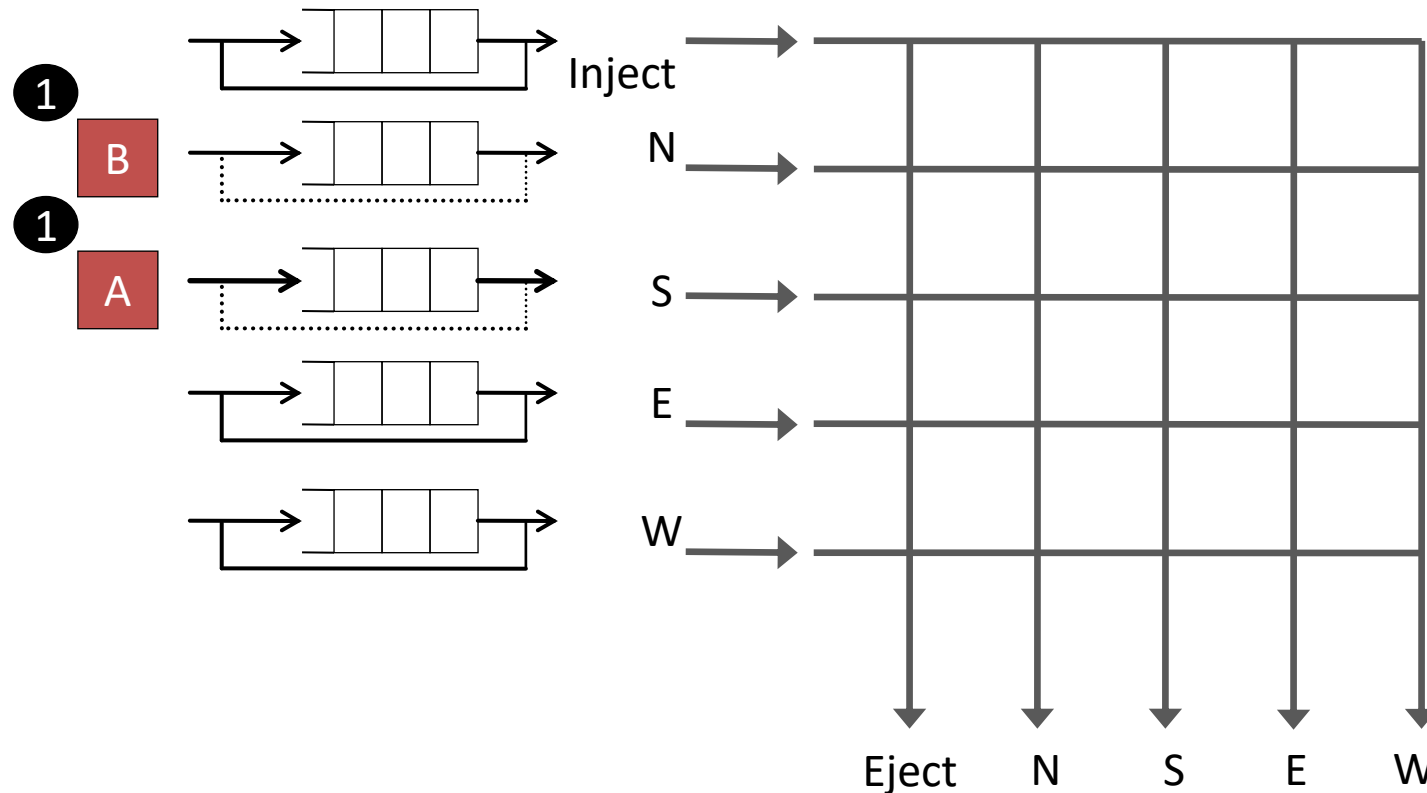
# Pipeline Optimizations – Example 2

e.g., No buffered flits when packets A and B arrive



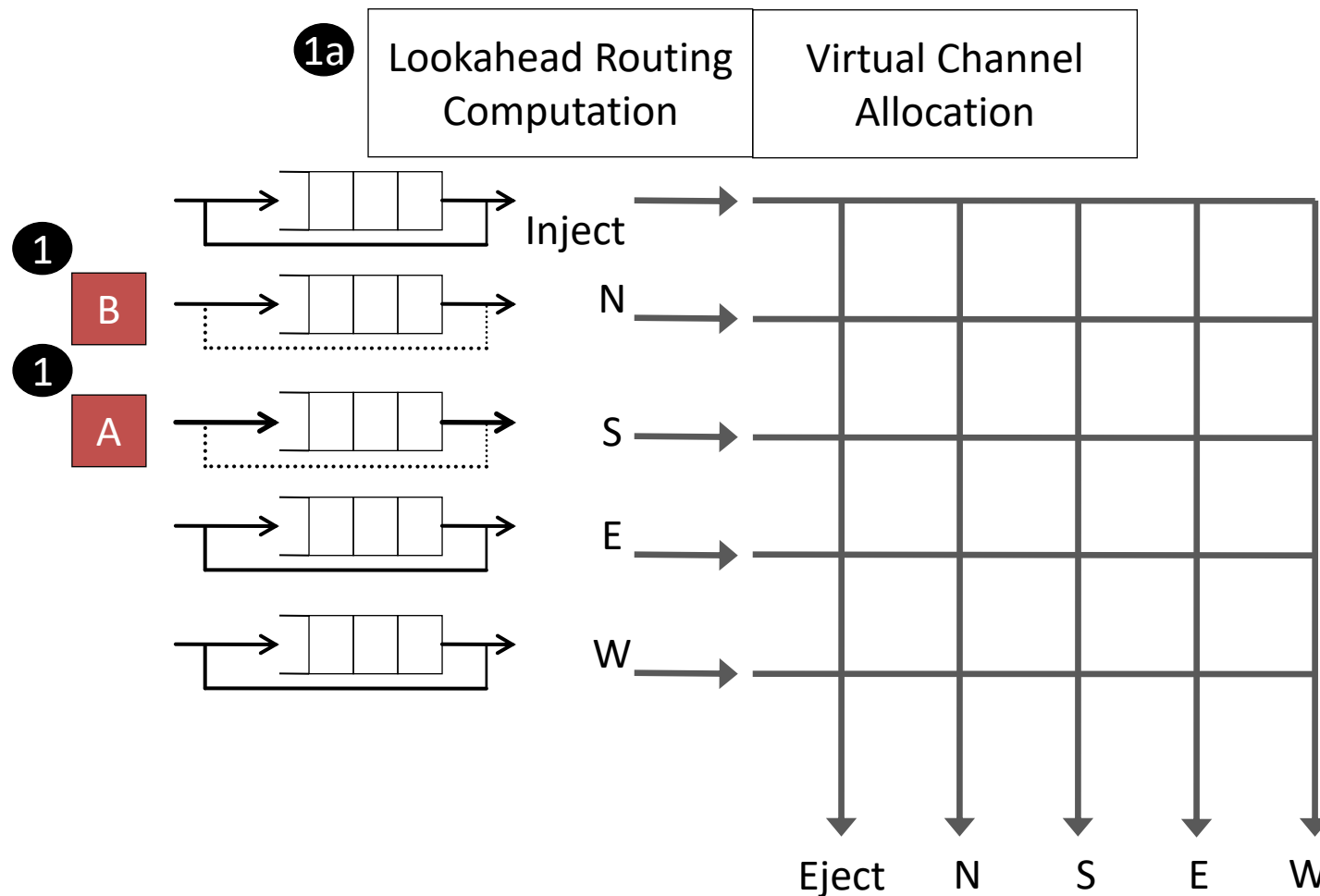
# Pipeline Optimizations – Example 2

e.g., No buffered flits when packets A and B arrive



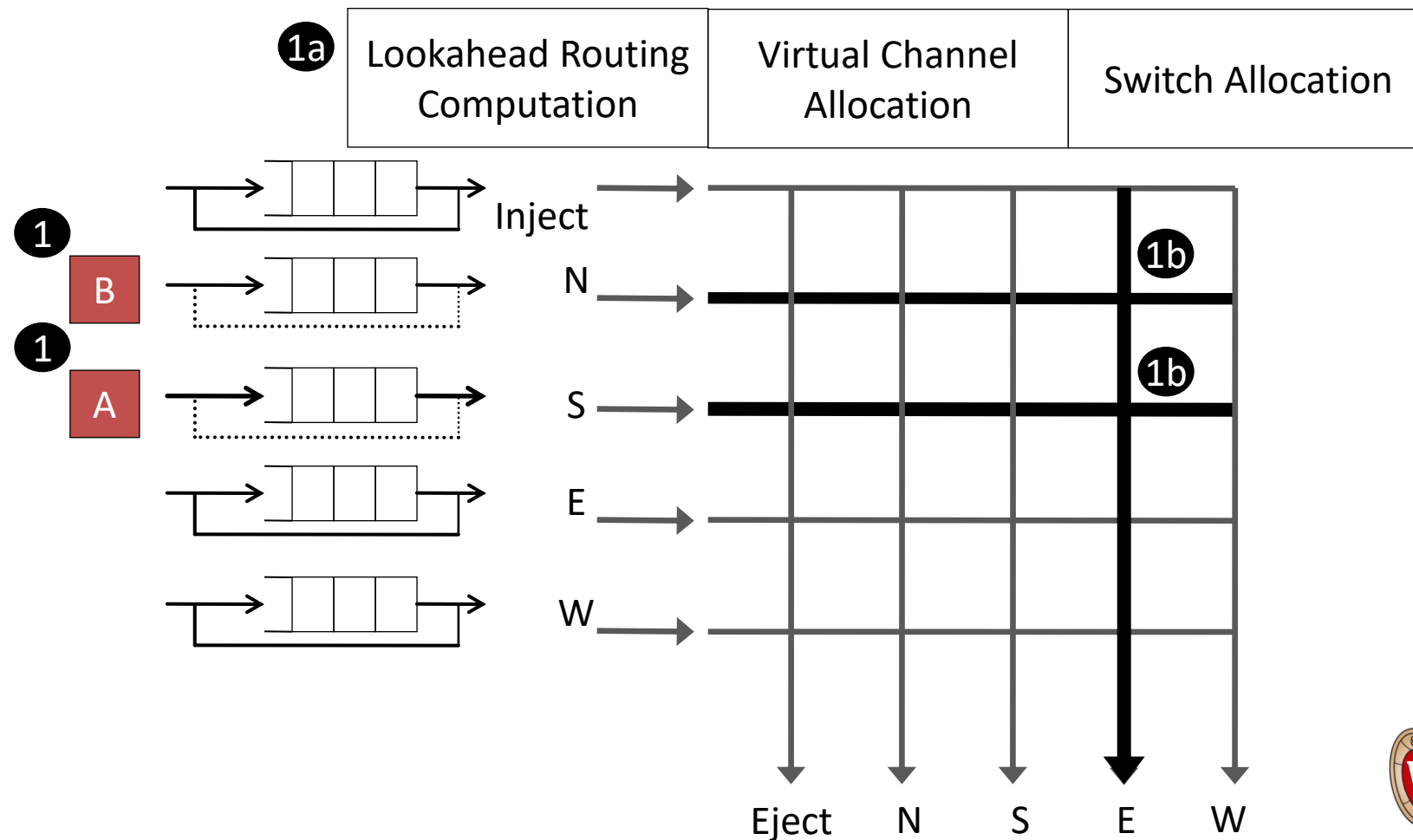
# Pipeline Optimizations – Example 2

e.g., No buffered flits when packets A and B arrive



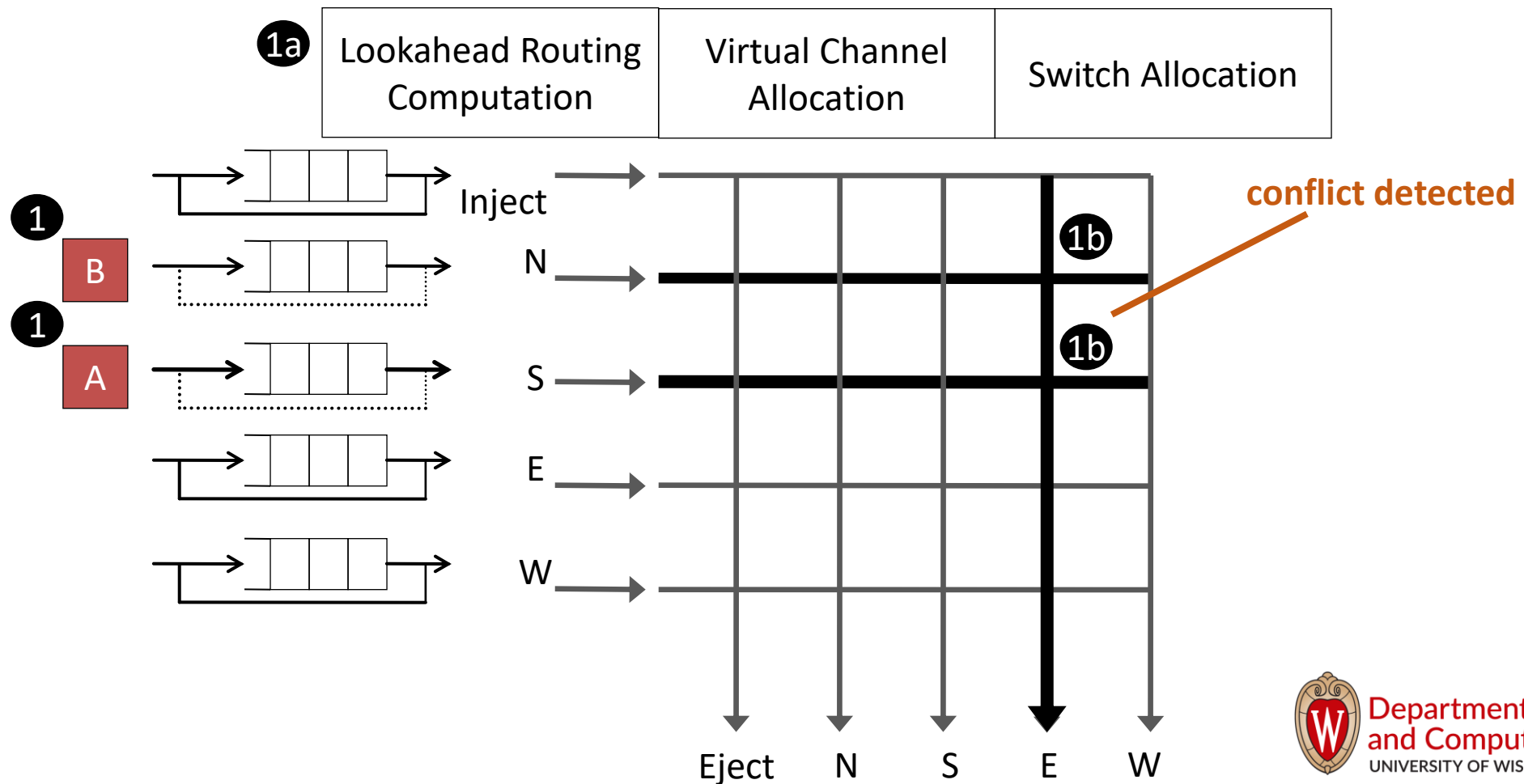
# Pipeline Optimizations – Example 2

e.g., No buffered flits when packets A and B arrive



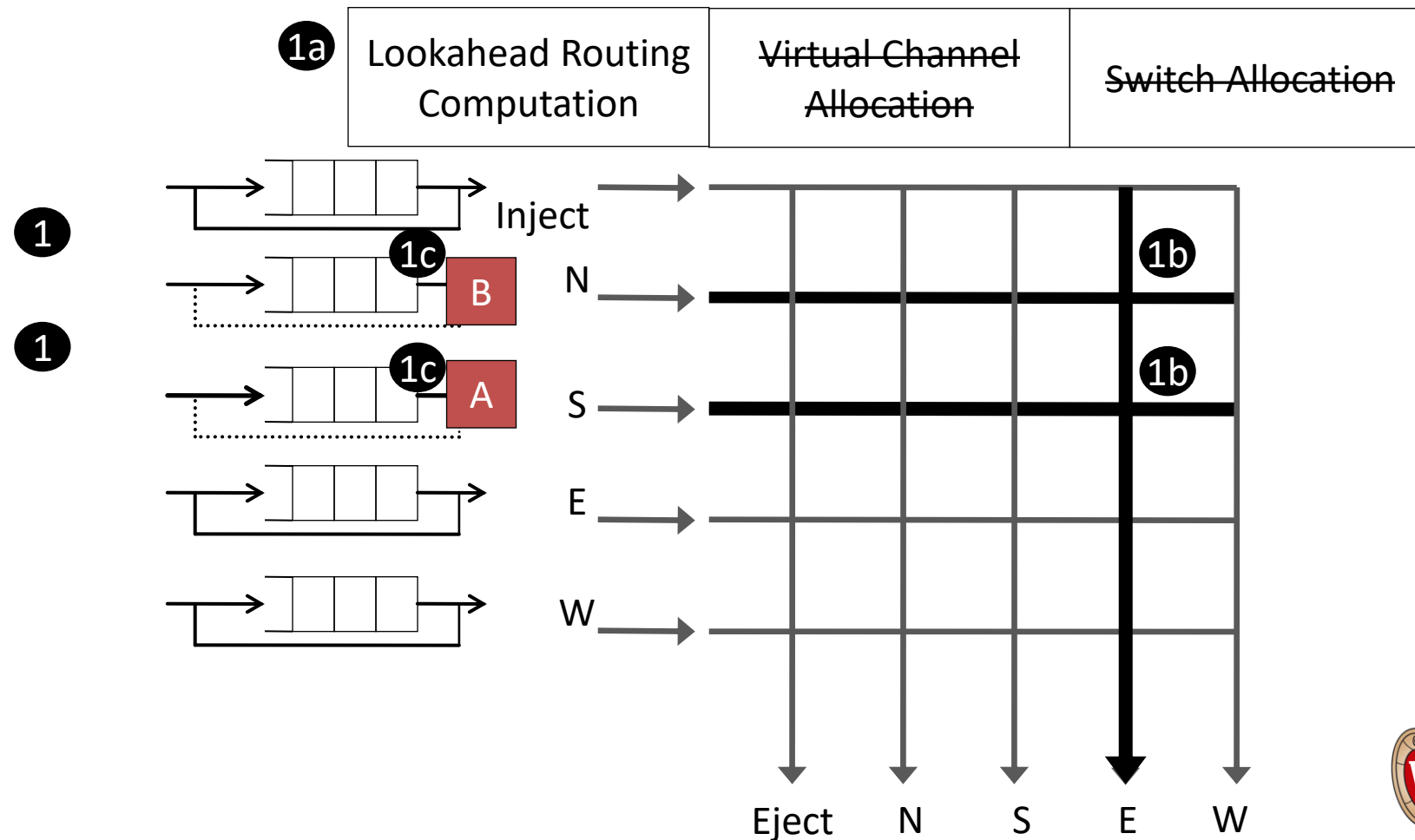
# Pipeline Optimizations – Example 2

e.g., No buffered flits when packets A and B arrive



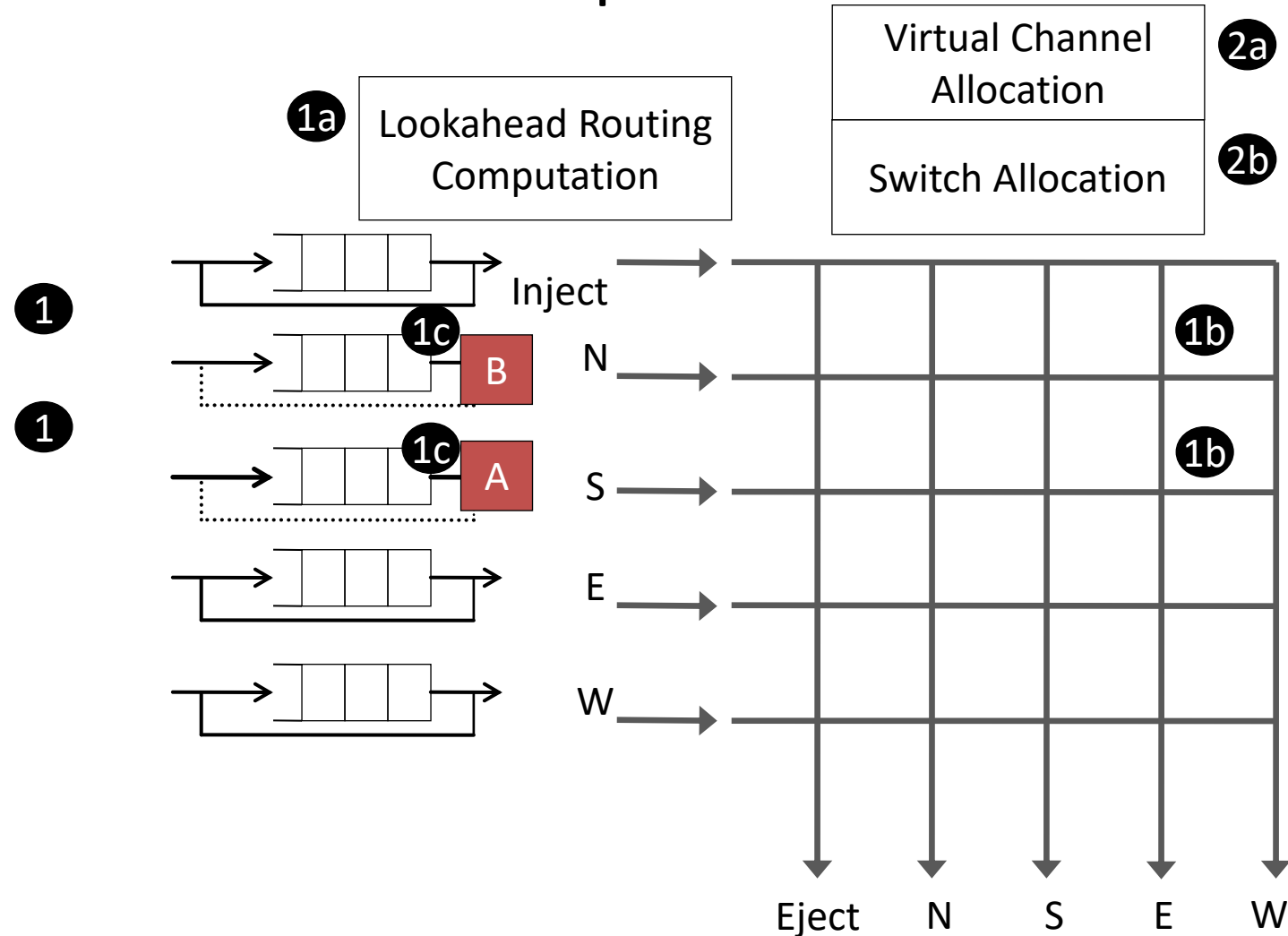
# Pipeline Optimizations – Example 2

e.g., No buffered flits when packets A and B arrive



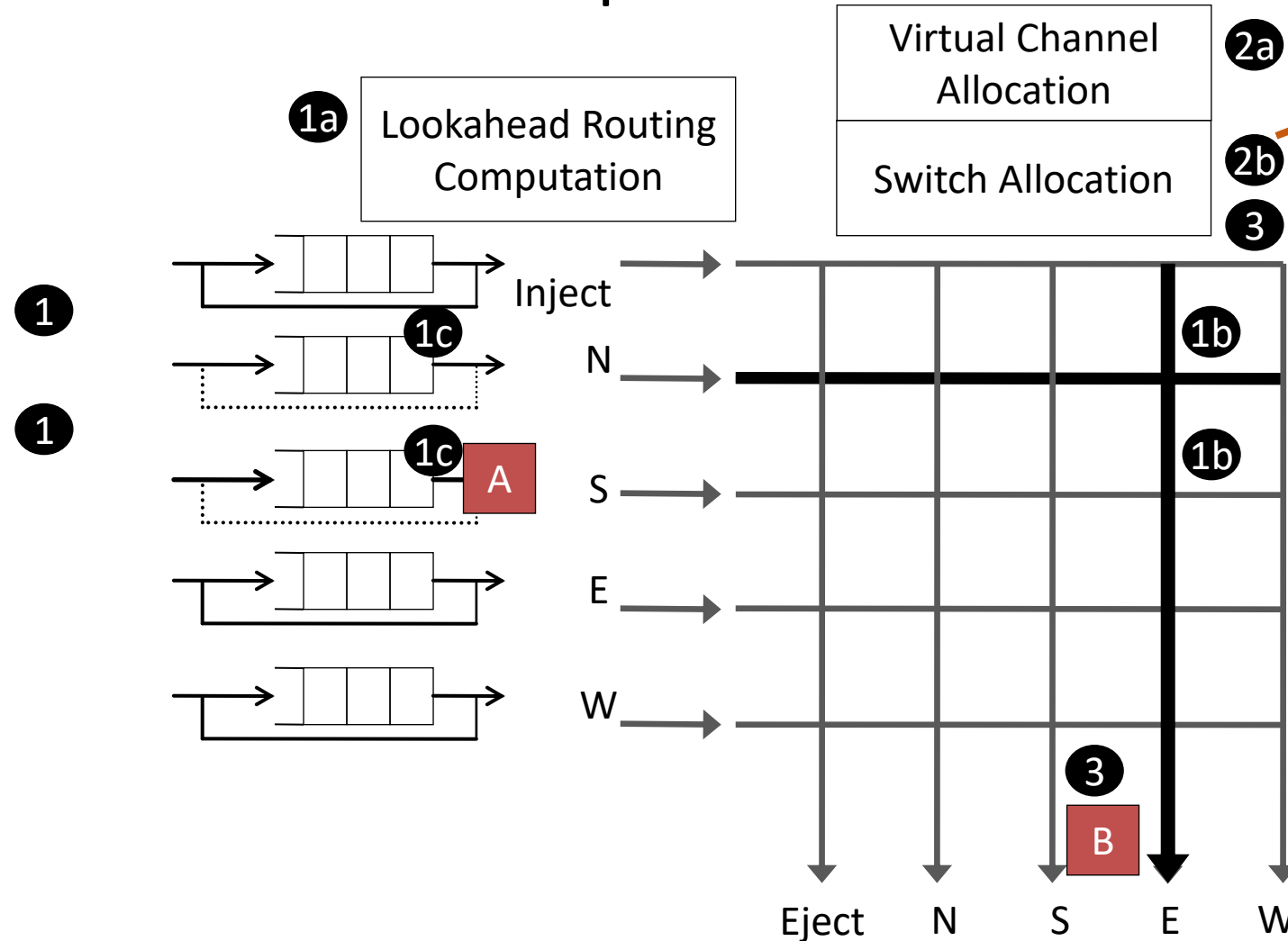
# Pipeline Optimizations – Example 2

e.g., No buffered flits when packets A and B arrive



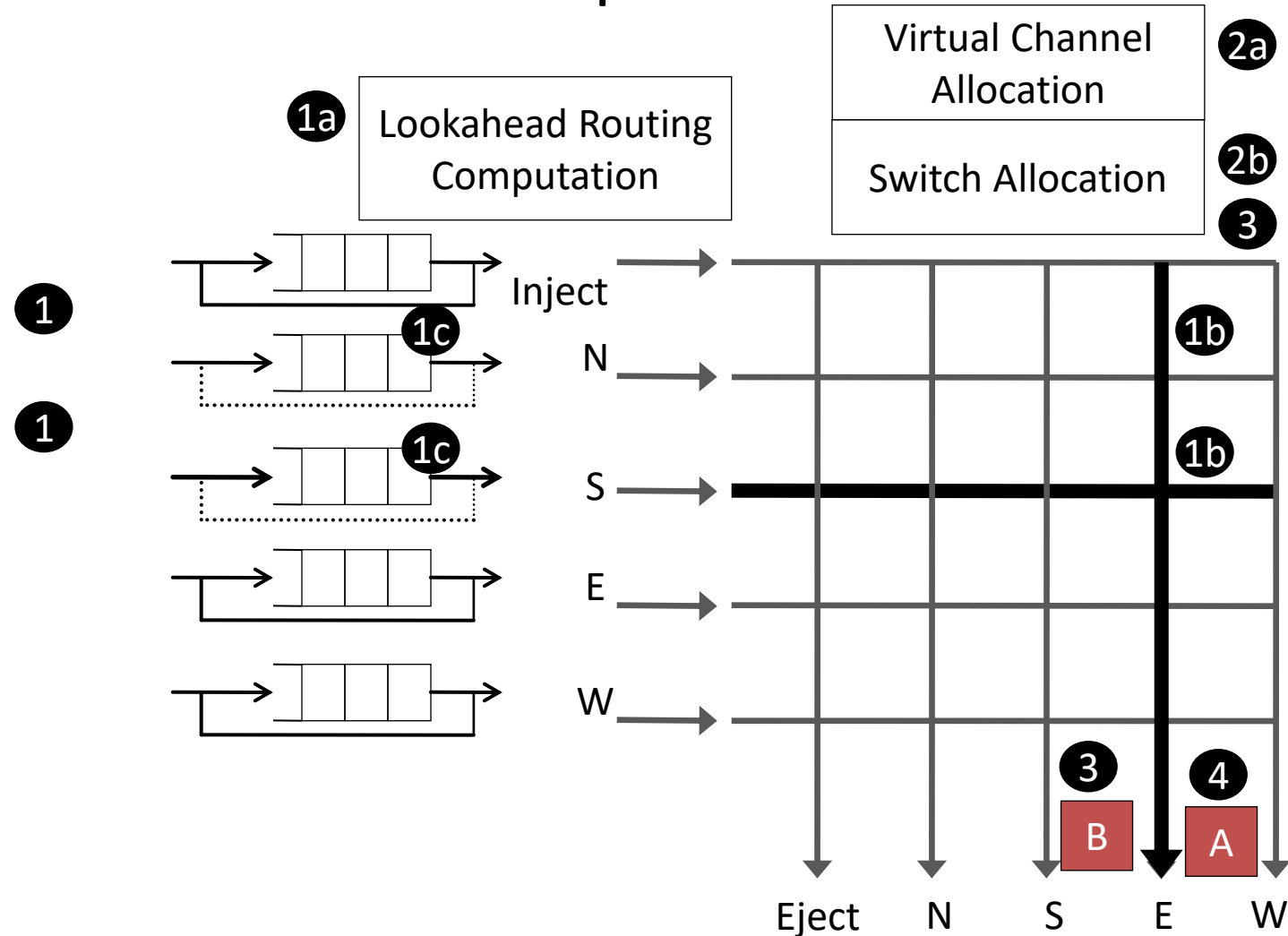
# Pipeline Optimizations – Example 2

e.g., No buffered flits when packets A and B arrive



# Pipeline Optimizations – Example 2

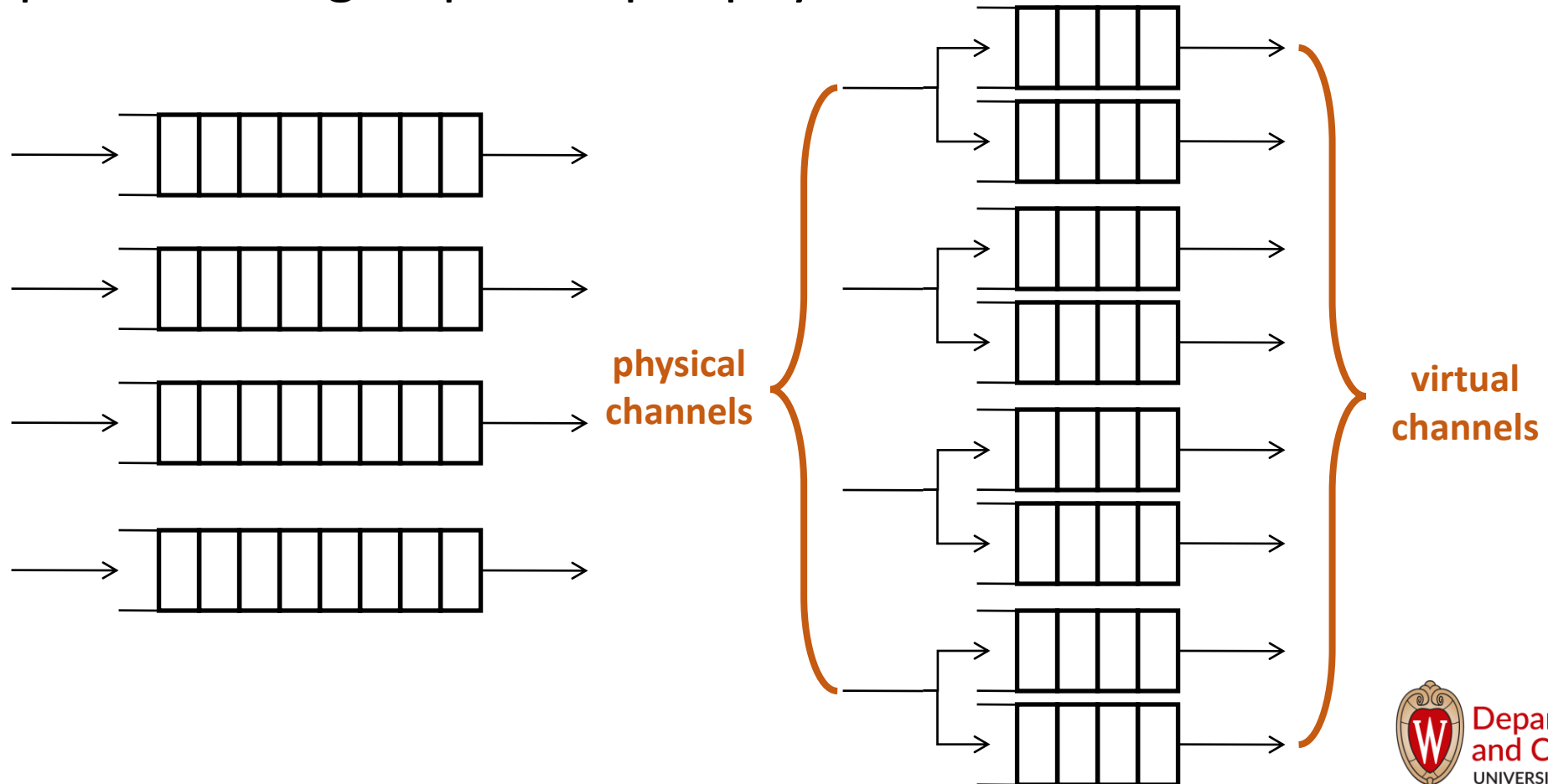
e.g., No buffered flits when packets A and B arrive



# Buffer Organization

## Single buffer per input

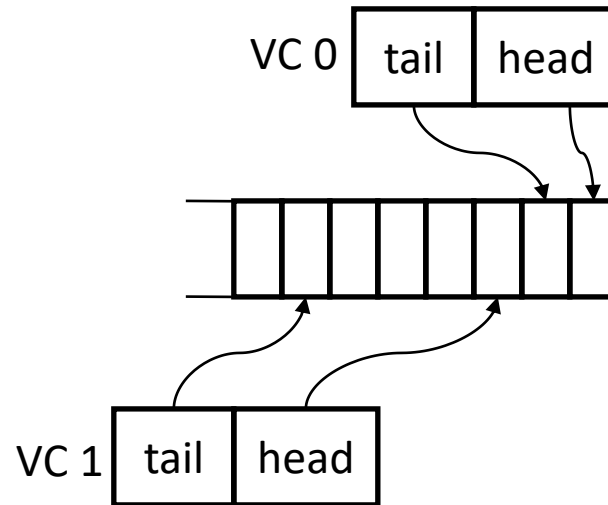
- Multiple fixed length queues per physical channel



# Buffer Organization

## Multiple variable length queues

- Multiple VCs share a large buffer
- Each VC must have minimum 1 flit buffer to prevent deadlock
- More complex circuitry



# Buffer Organization

## Many shallow VCs

- + Better addresses head-of-line blocking
- + Good for heavy traffic

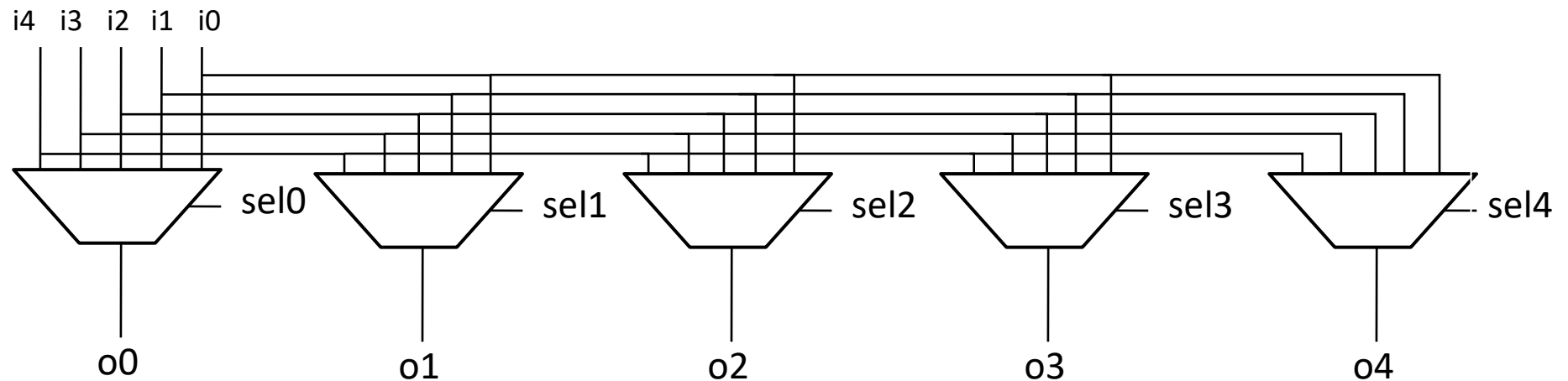
## Few deep VCs

- + Less complex VC allocator
- + Smoother flow control

# Switch Organization – Mux Crossbar

Crossbar composed of many multiplexers

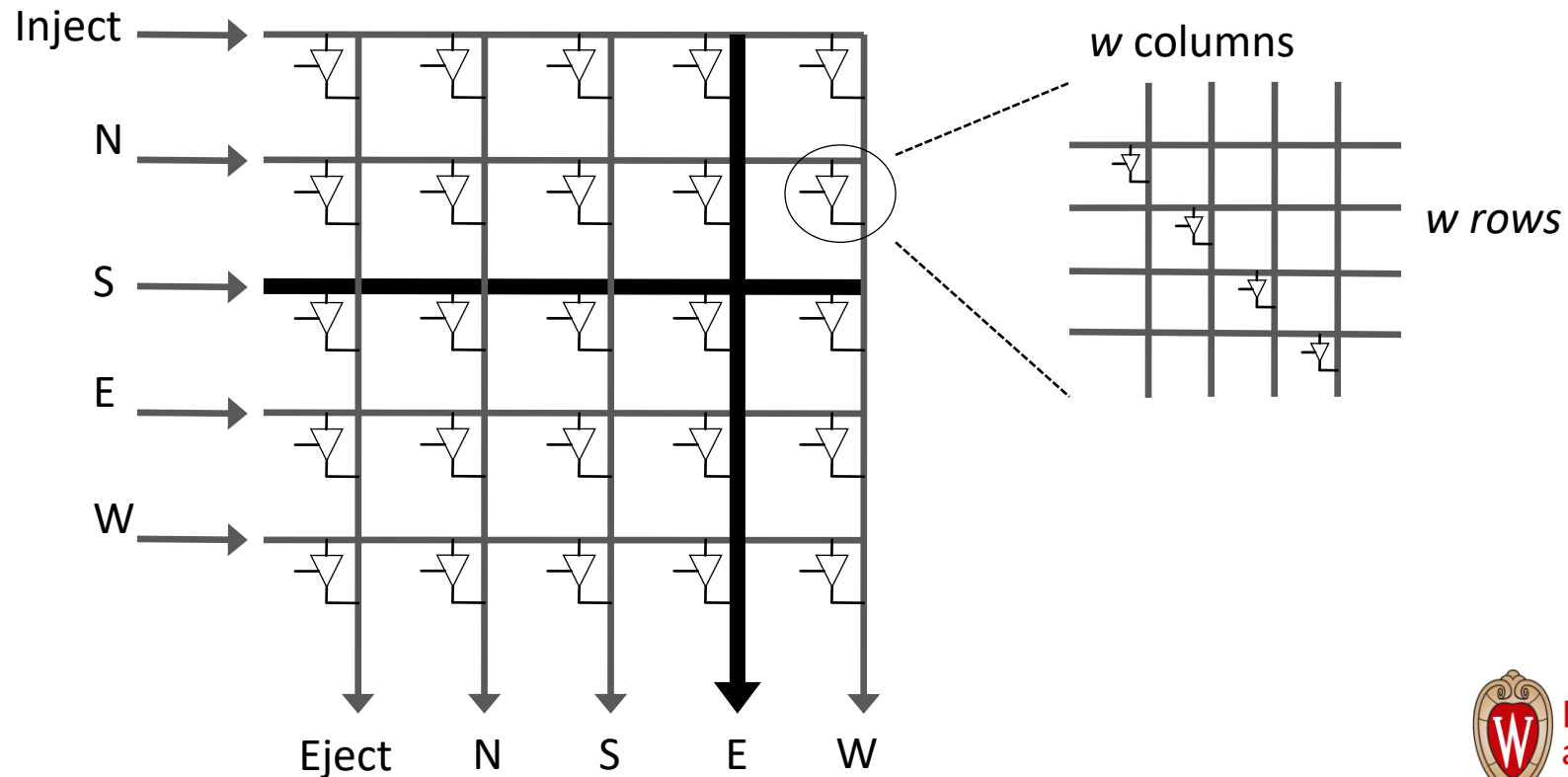
- Common in low-frequency router designs



# Switch Organization – Crosspoint

## Grid of tristate buffers

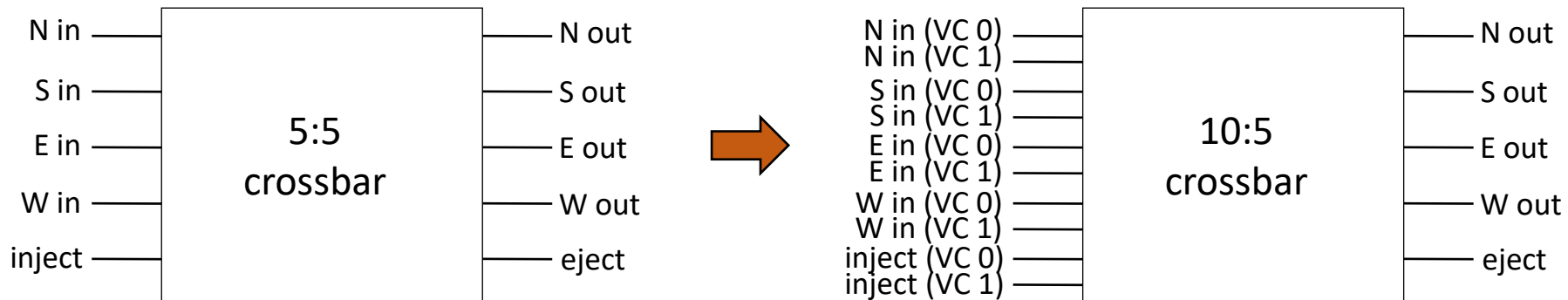
- $w$ : port width in bits, determines phit size



# Switch Organization – Crossbar Speedup

Design crossbar with more ports

- Increases internal switch bandwidth
- Allows VCs of the same input port to use crossbar at the same time



# Allocators

**Allocator:** matches  $N$  requests to  $M$  resources

**Arbiter:** matches  $N$  requests to 1 resource

- Requesters are packets
- Resources are VCs and crossbar switch ports

# Allocators

## **VC allocator (VA)**

- Grants output VCs to packets in input VCs

## **Switch allocator (SA)**

- Grants crossbar switch output ports to packets at input ports/VCs

Goal of allocator/arbiter is to deliver high matching probability

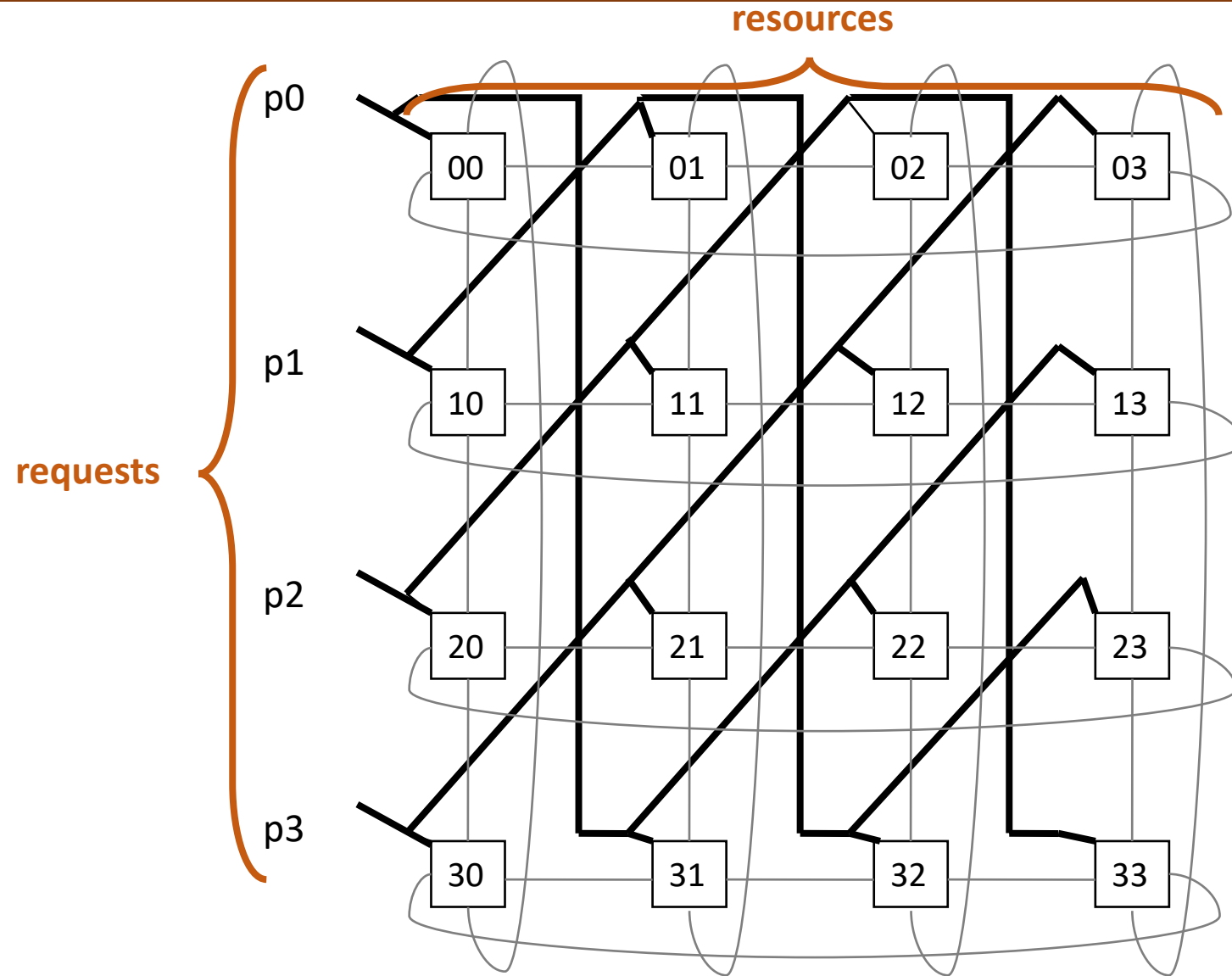
- Translates to high network throughput
- Ideally also fast and/or able to be pipelined

# Wavefront Allocator

Arbitrates among requests for inputs and outputs simultaneously

- Concept: try different request-to-resource mappings in a diagonal pattern
- Row and column tokens granted to diagonal group of cells
- If a cell is requesting a resource, it will consume row and column tokens (i.e., request is granted)
- Cells that cannot use tokens pass row tokens to right and column tokens down

# Wavefront Allocator



# Wavefront Allocator

A requesting  
resources 0,1,2

p0

B requesting  
resources 0,1

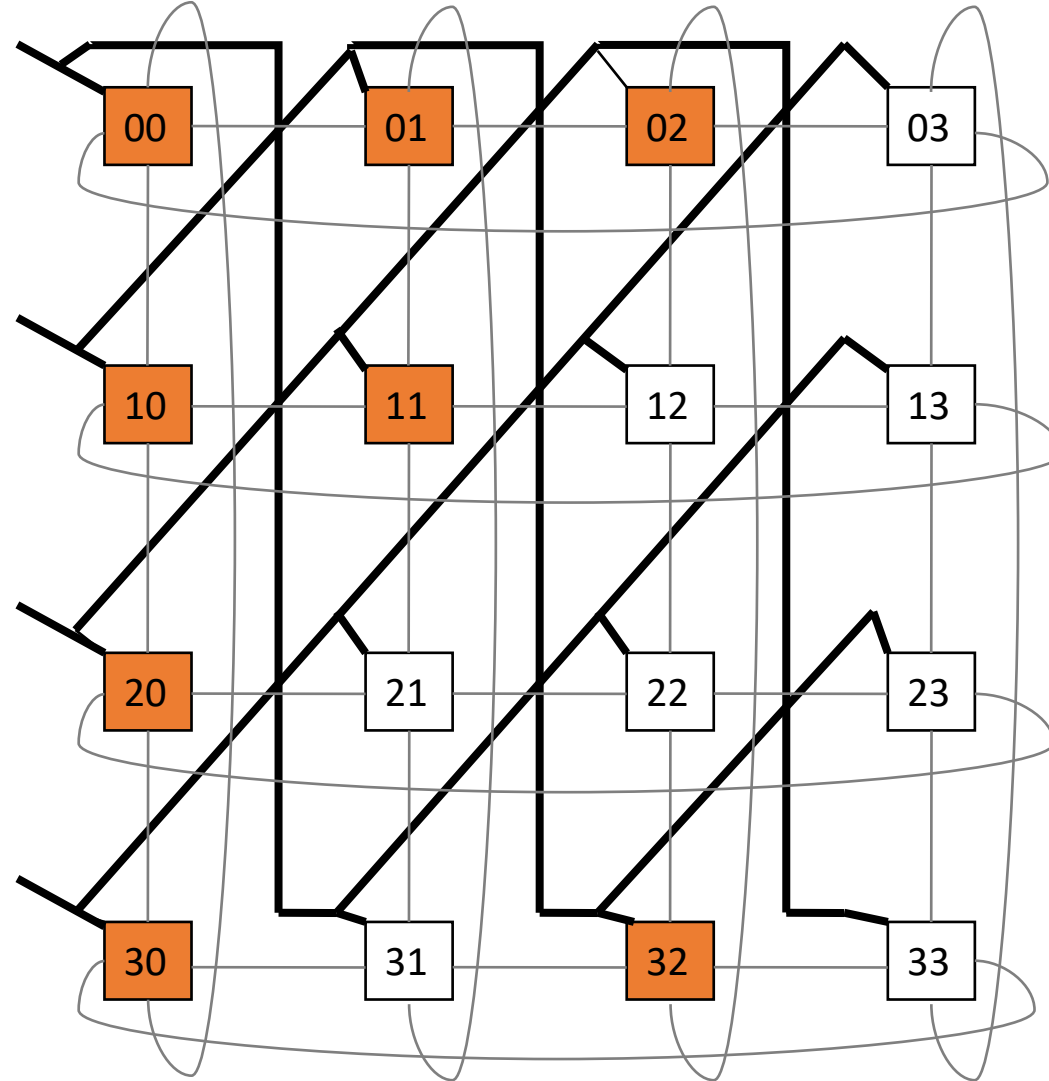
p1

C requesting  
resource 0

p2

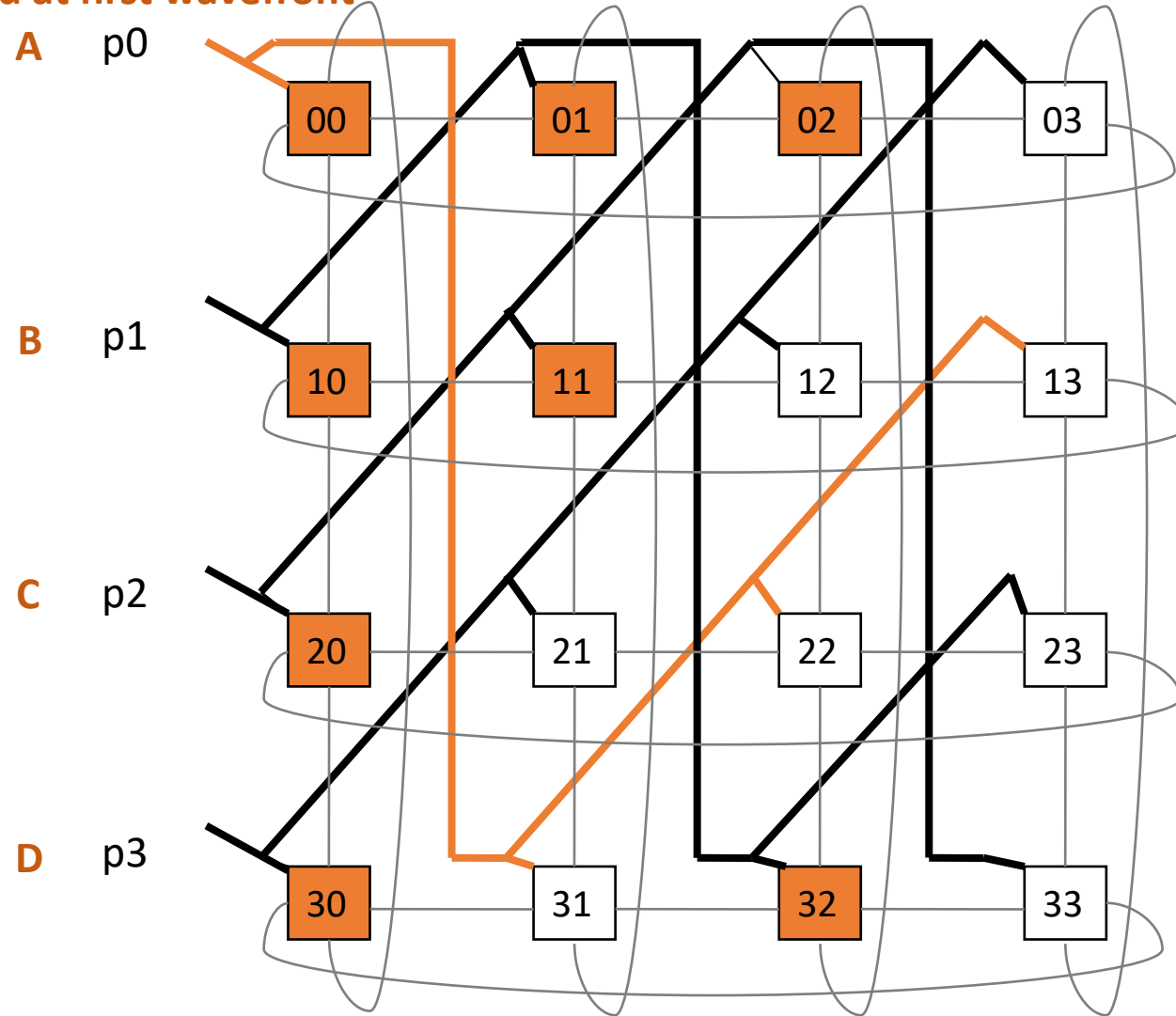
D requesting  
resources 0,2

p3



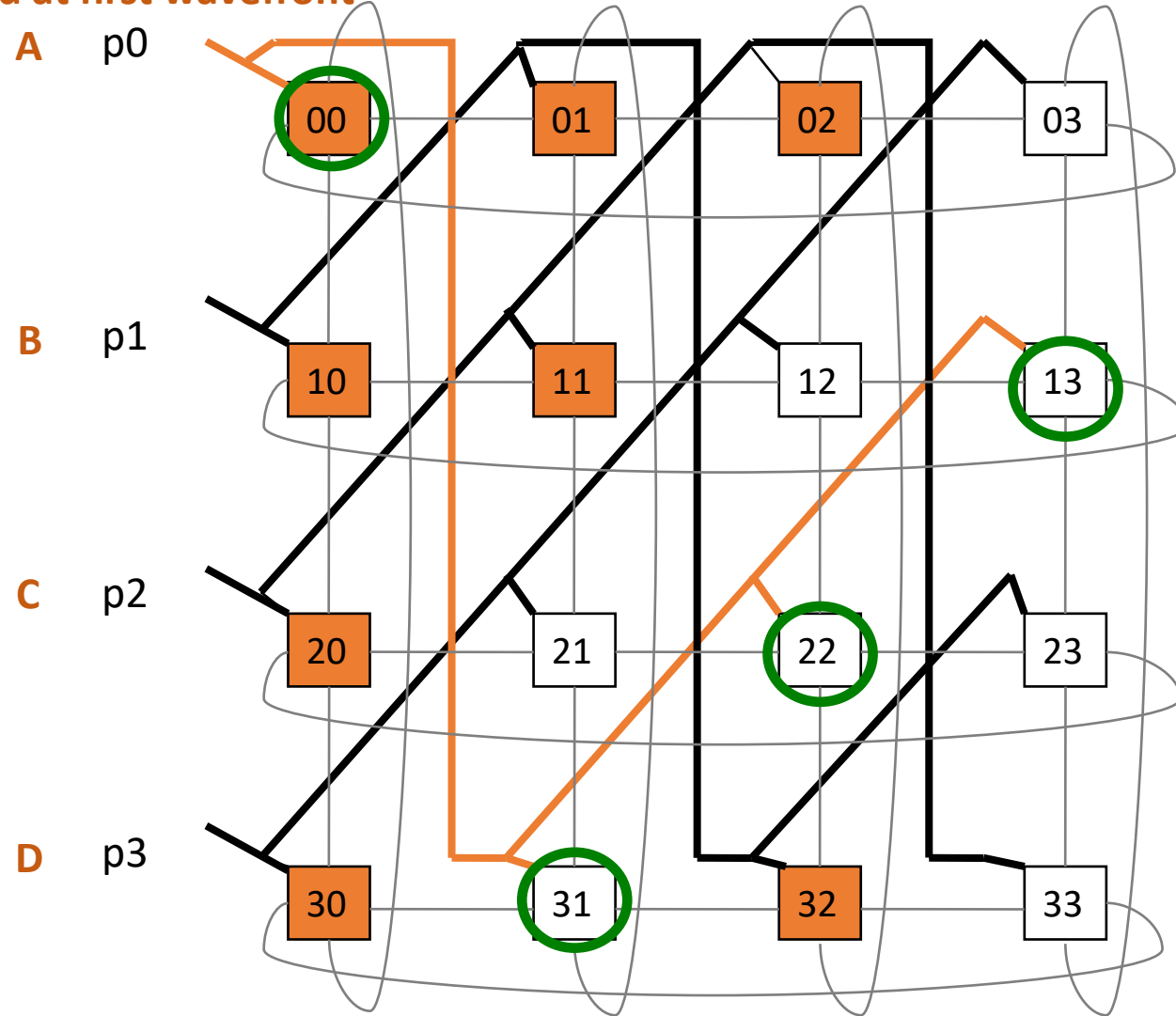
# Wavefront Allocator

tokens inserted at first wavefront

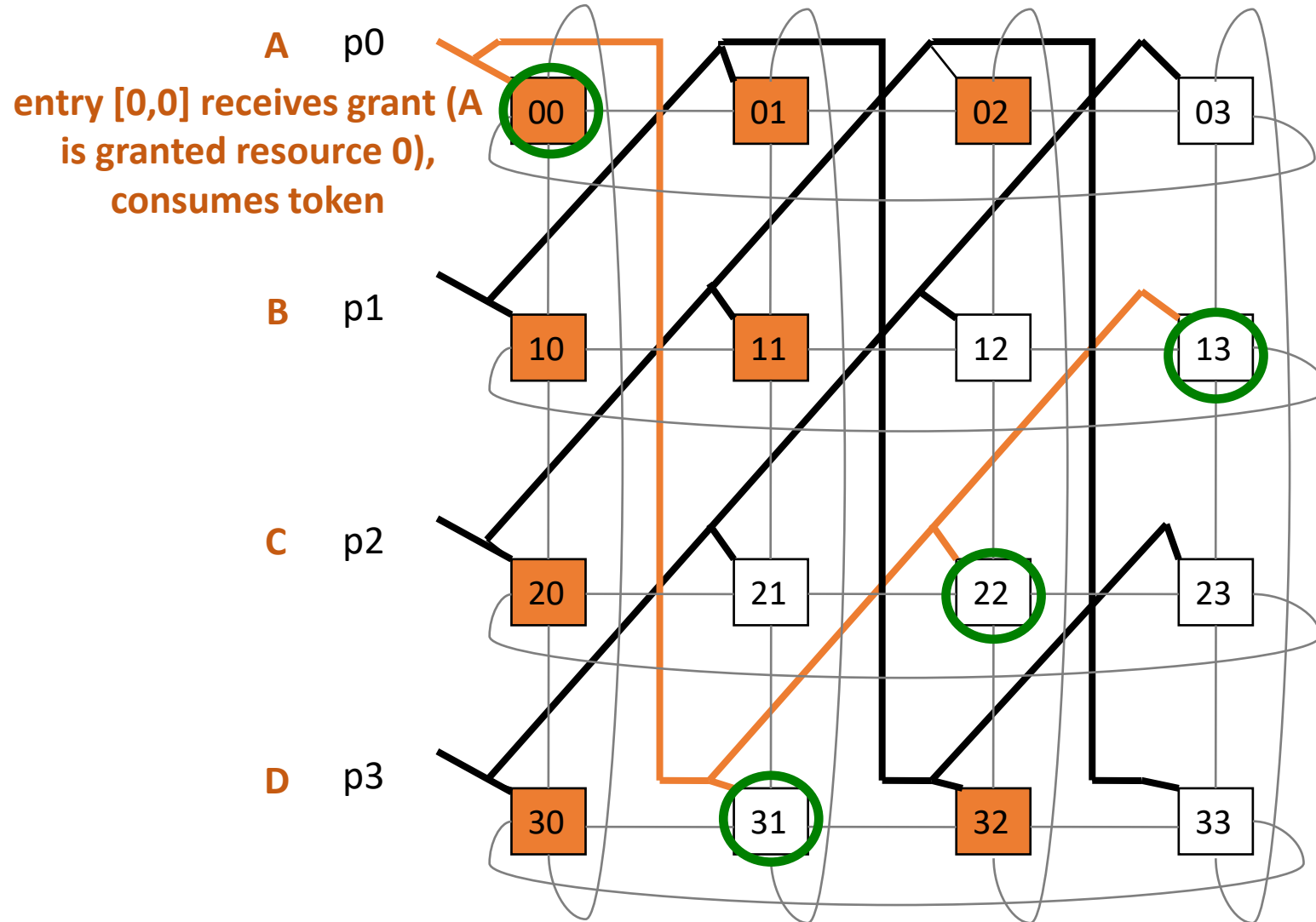


# Wavefront Allocator

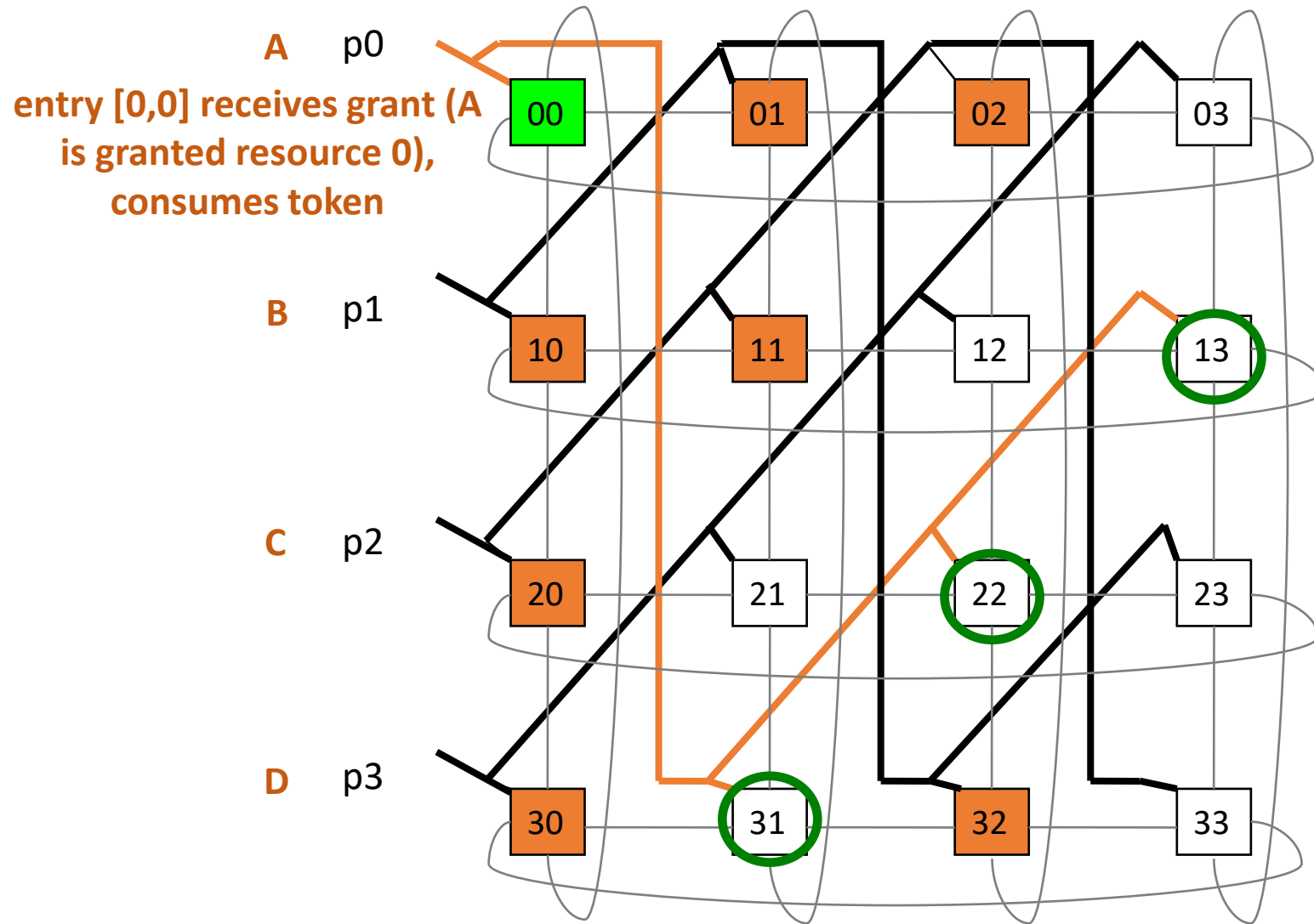
tokens inserted at first wavefront



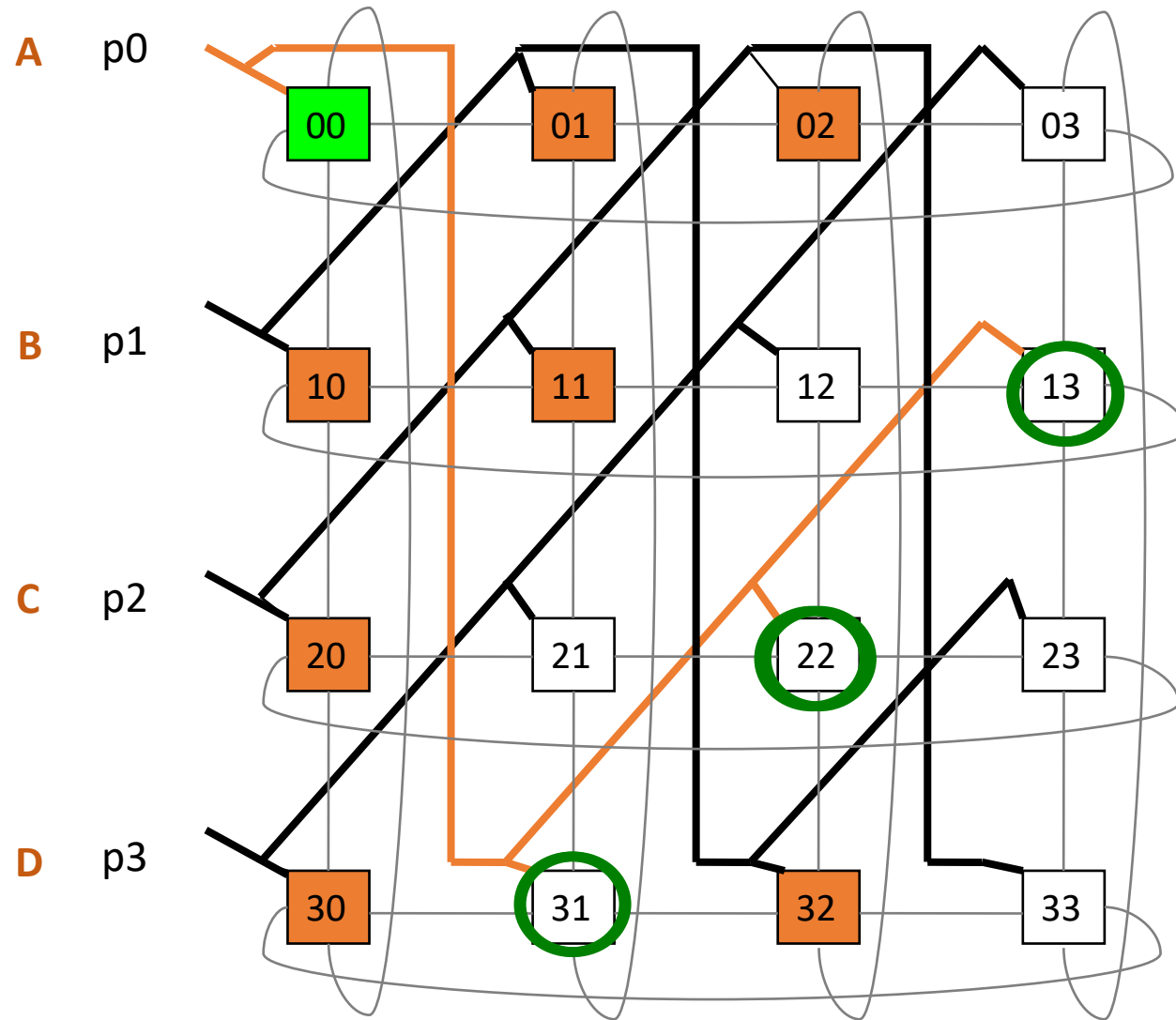
# Wavefront Allocator



# Wavefront Allocator



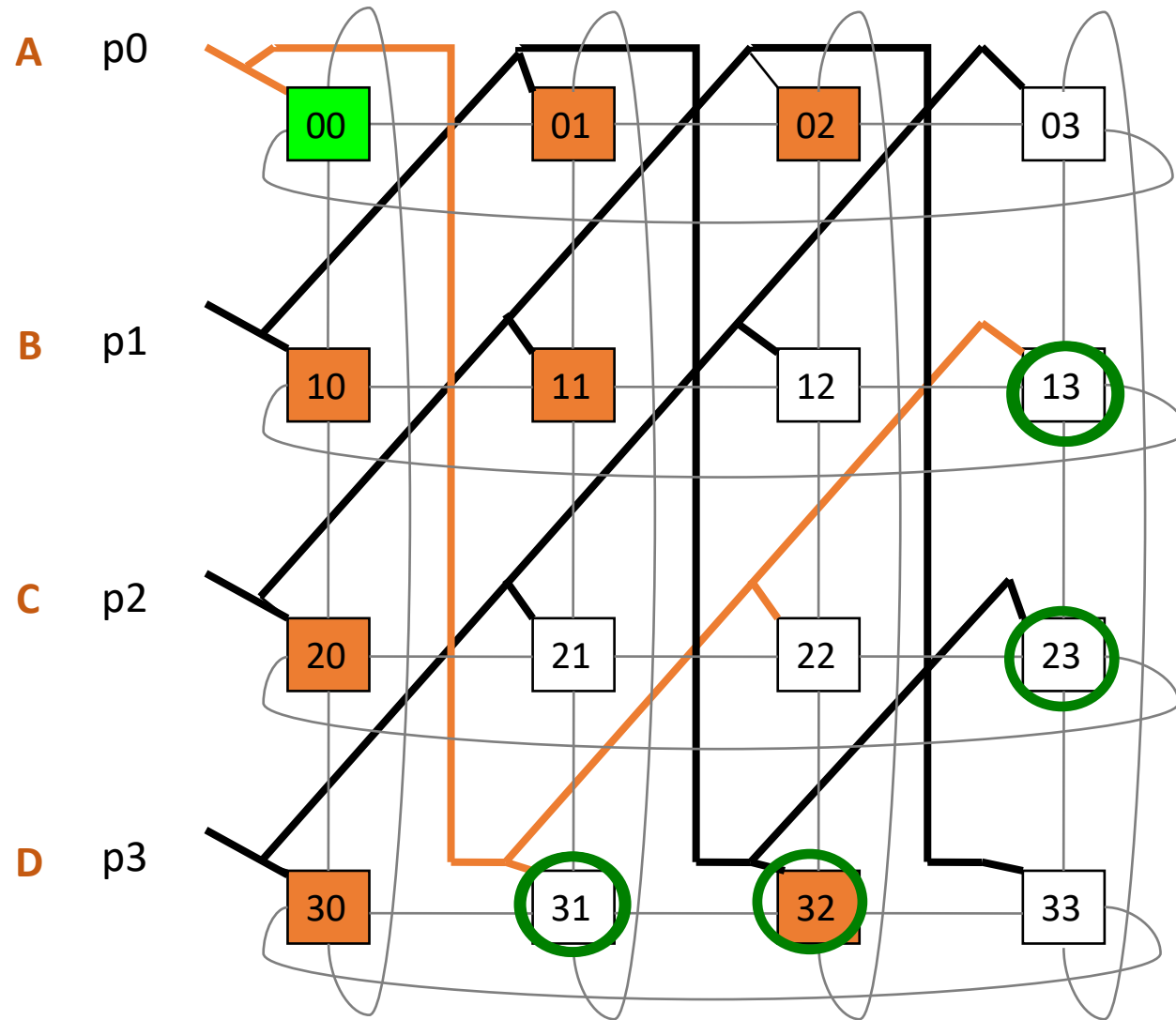
# Wavefront Allocator



remaining tokens pass  
down and right



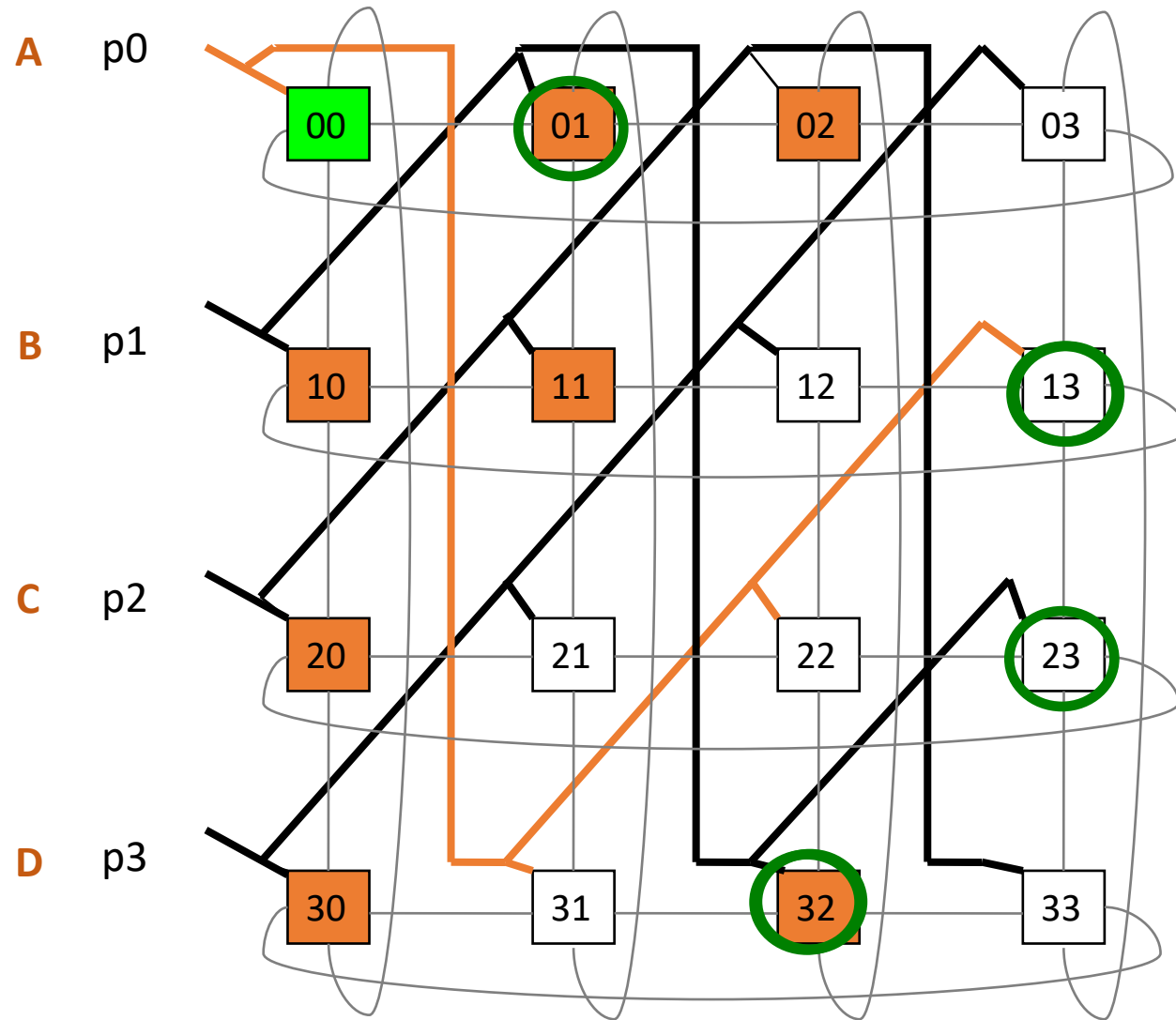
# Wavefront Allocator



remaining tokens pass  
down and right

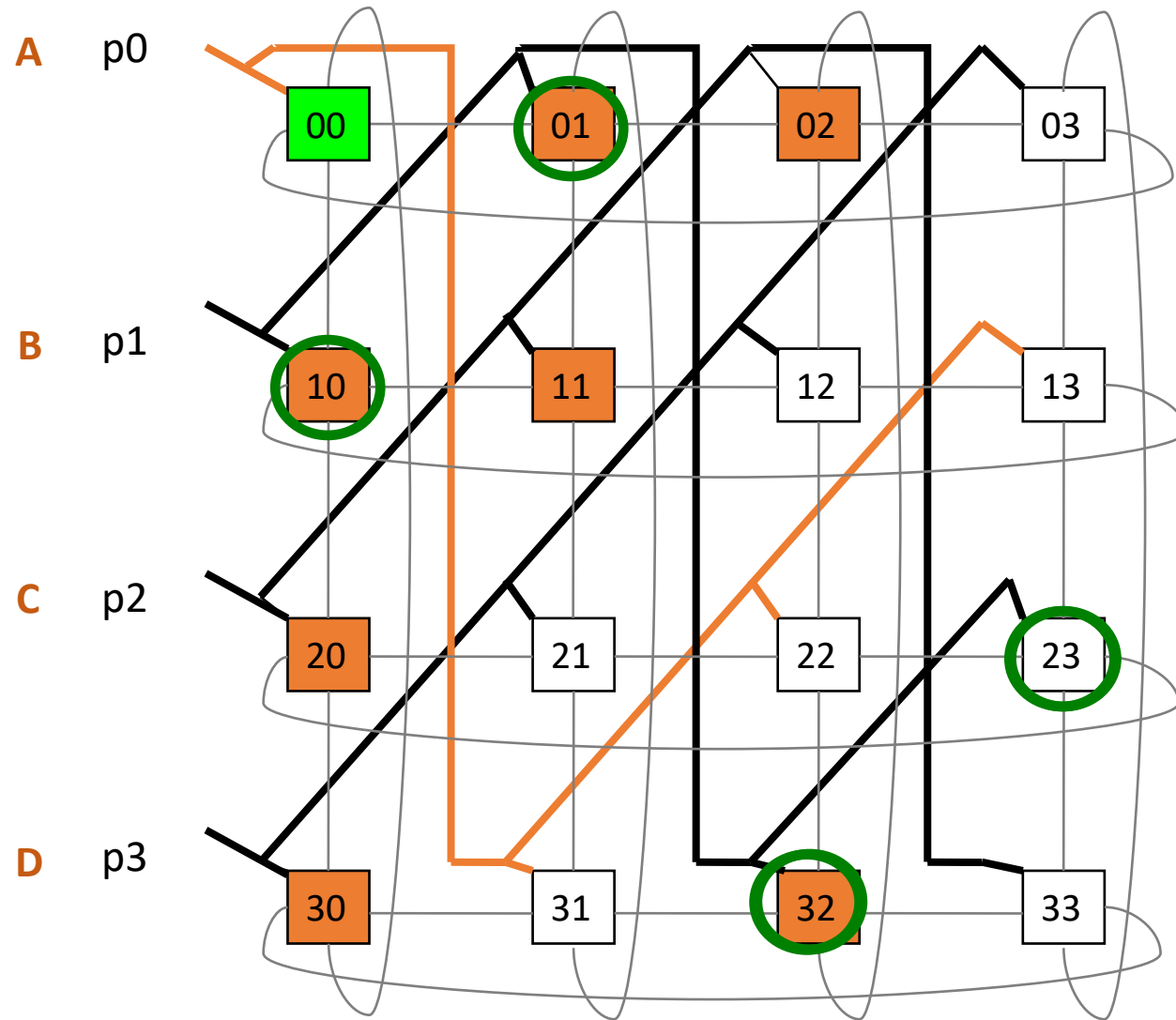


# Wavefront Allocator



remaining tokens pass  
down and right

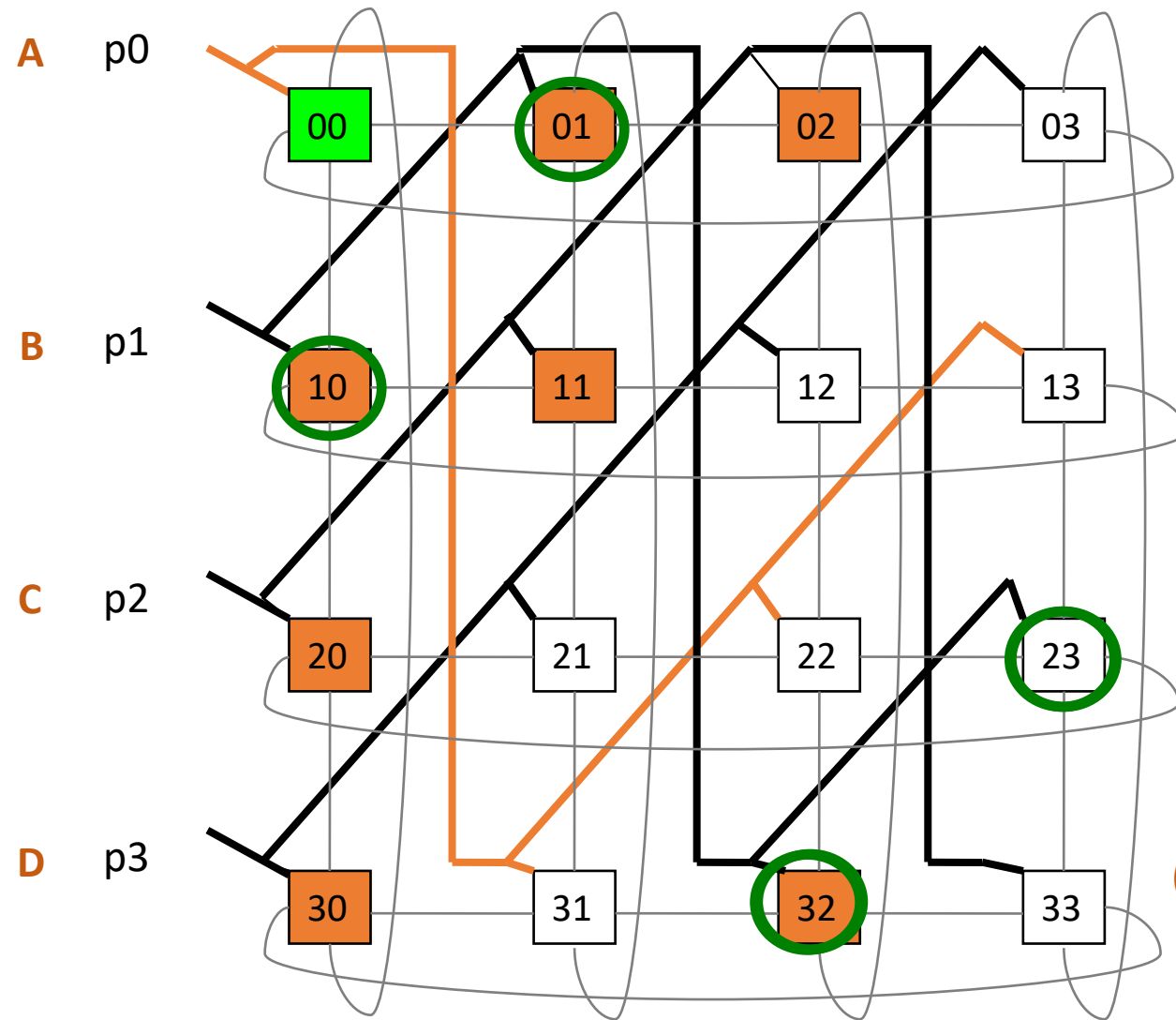
# Wavefront Allocator



remaining tokens pass  
down and right

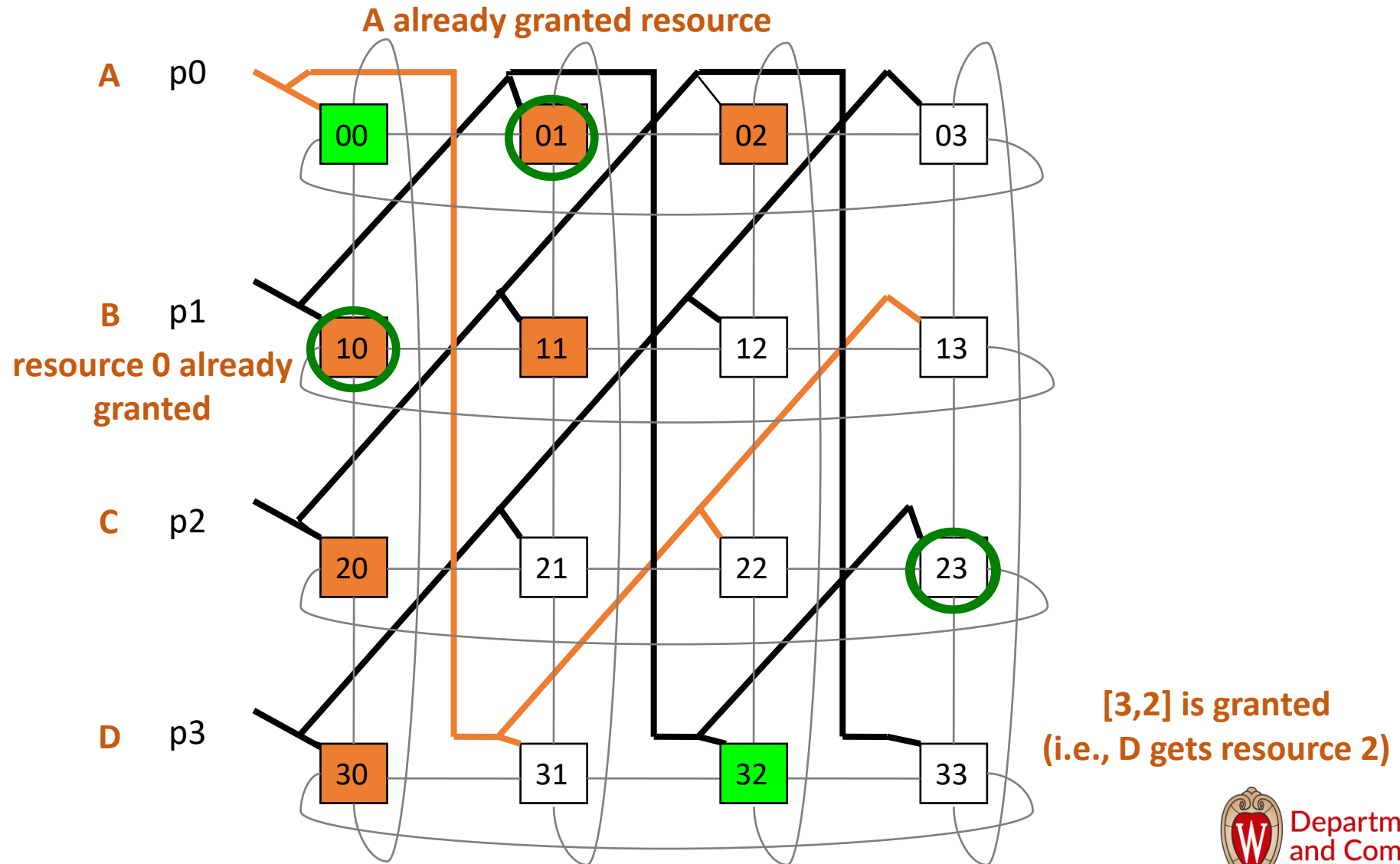


# Wavefront Allocator

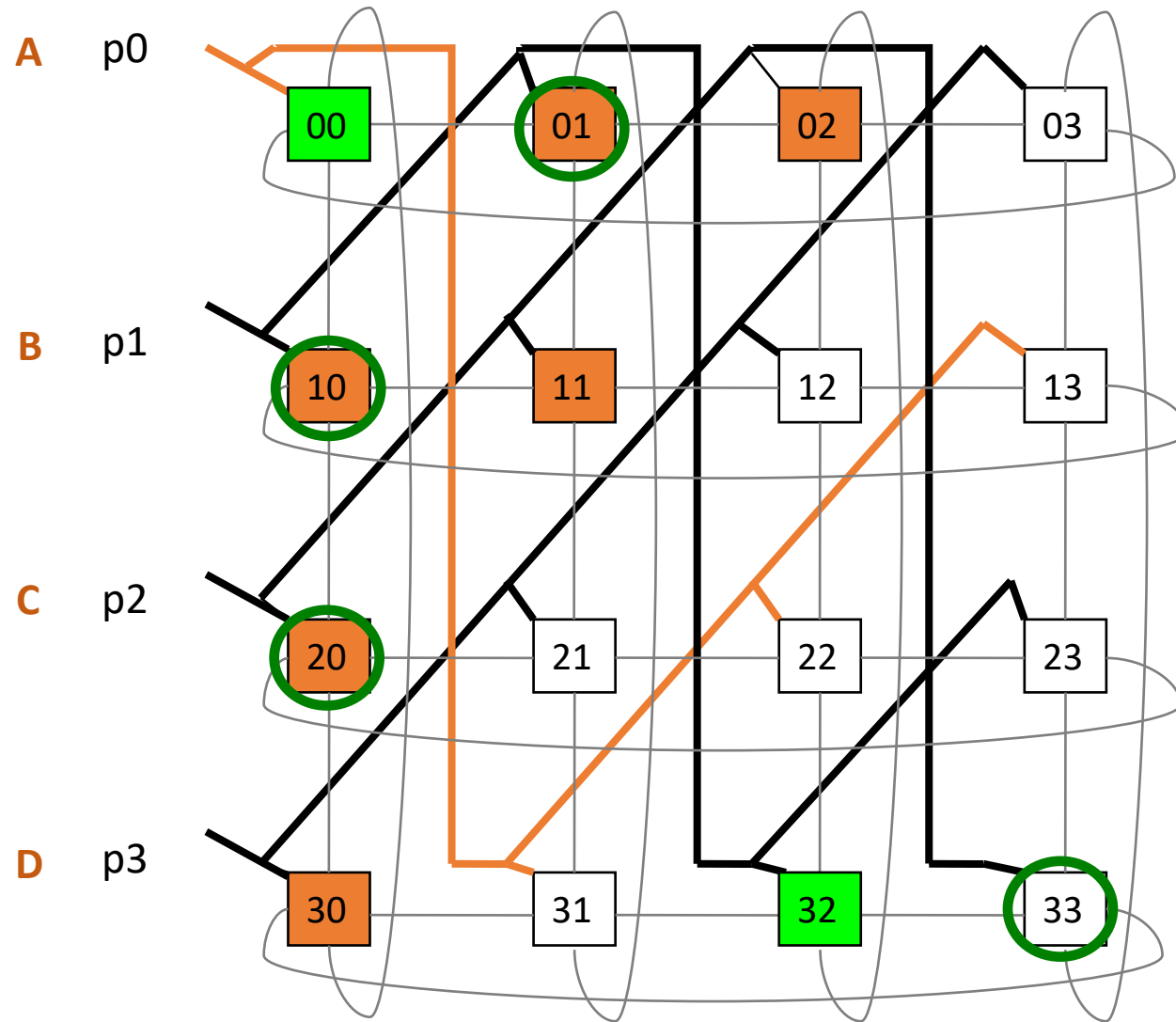


[3,2] is granted  
(i.e., D gets resource 2)

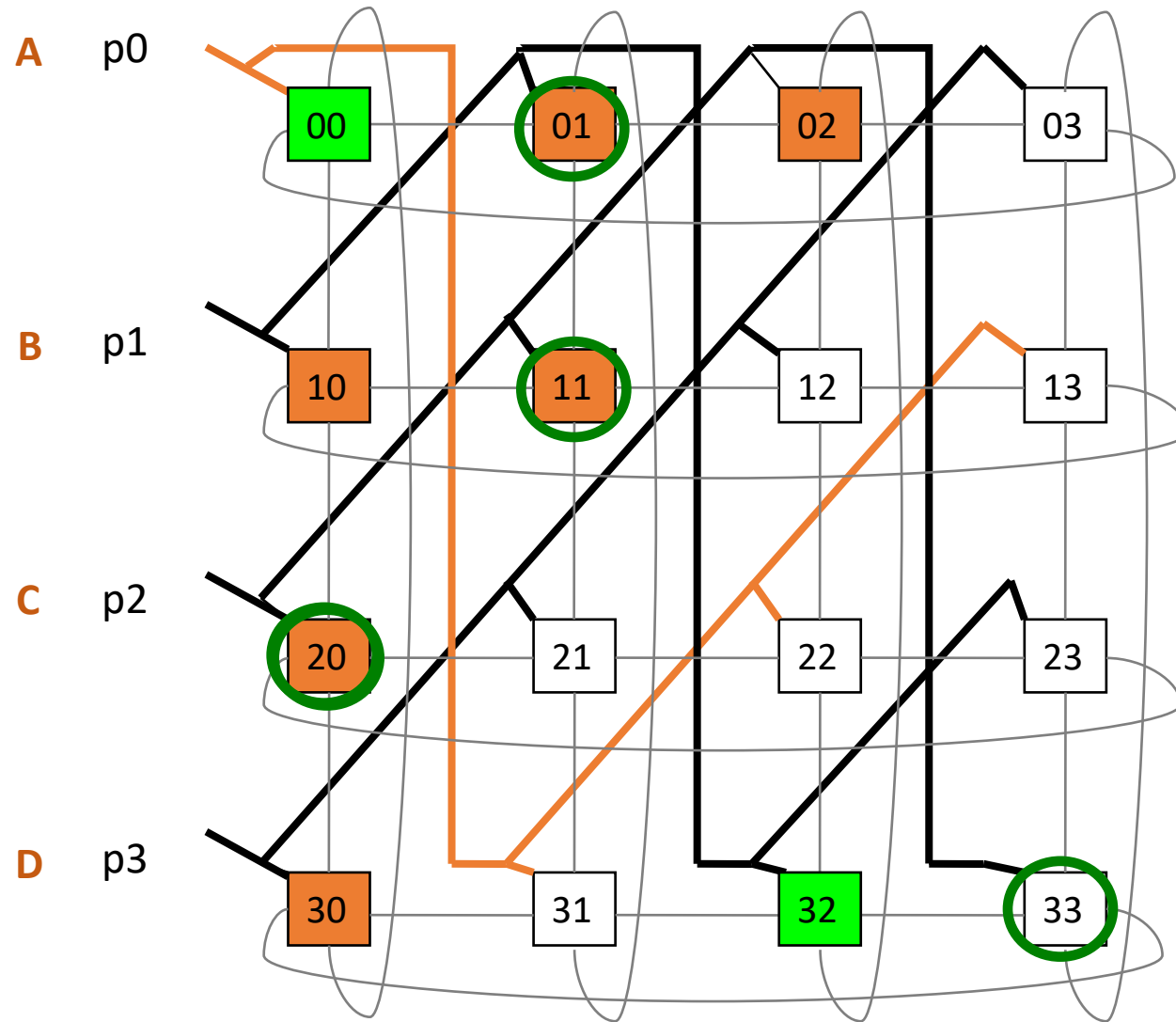
# Wavefront Allocator



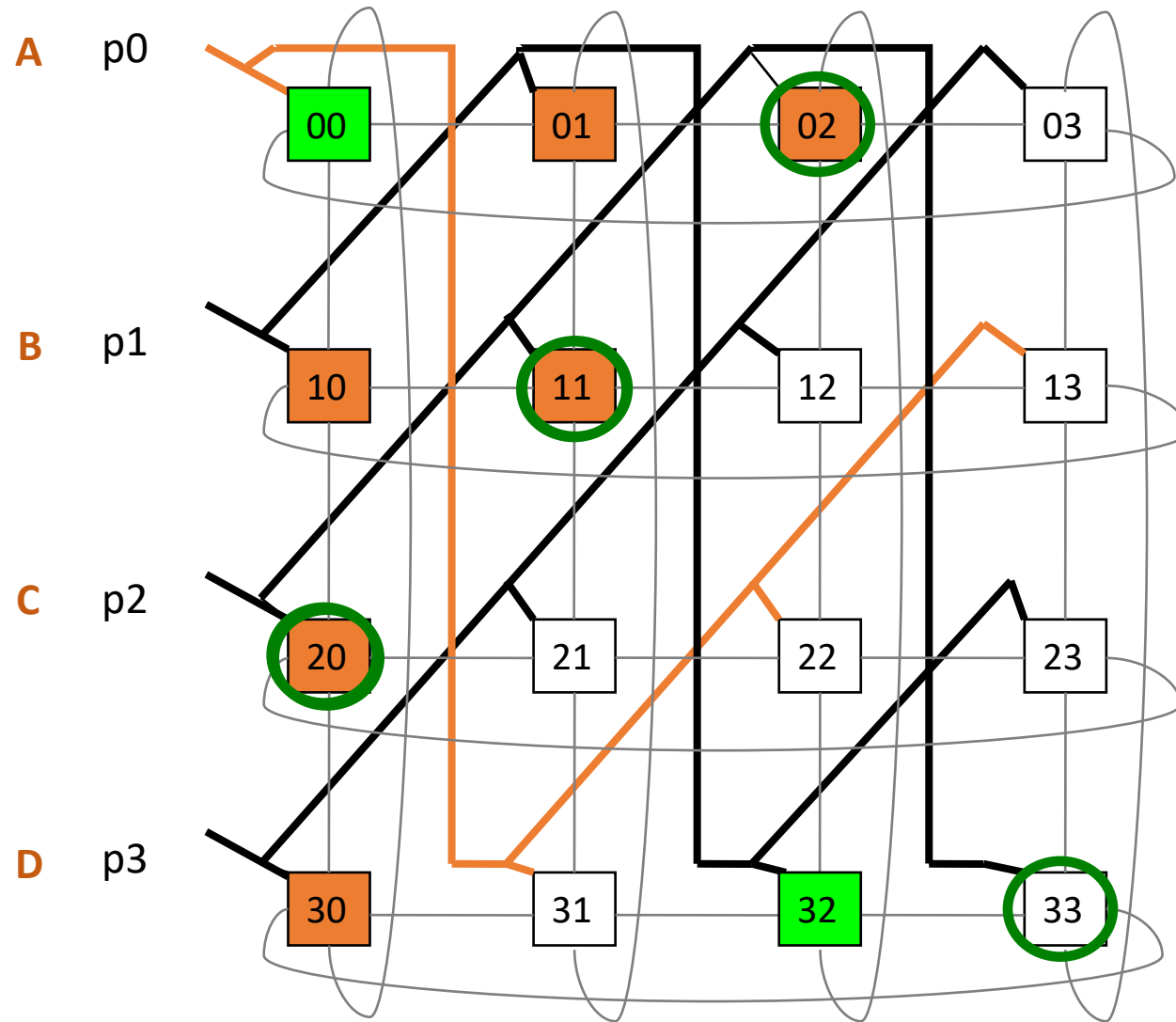
# Wavefront Allocator



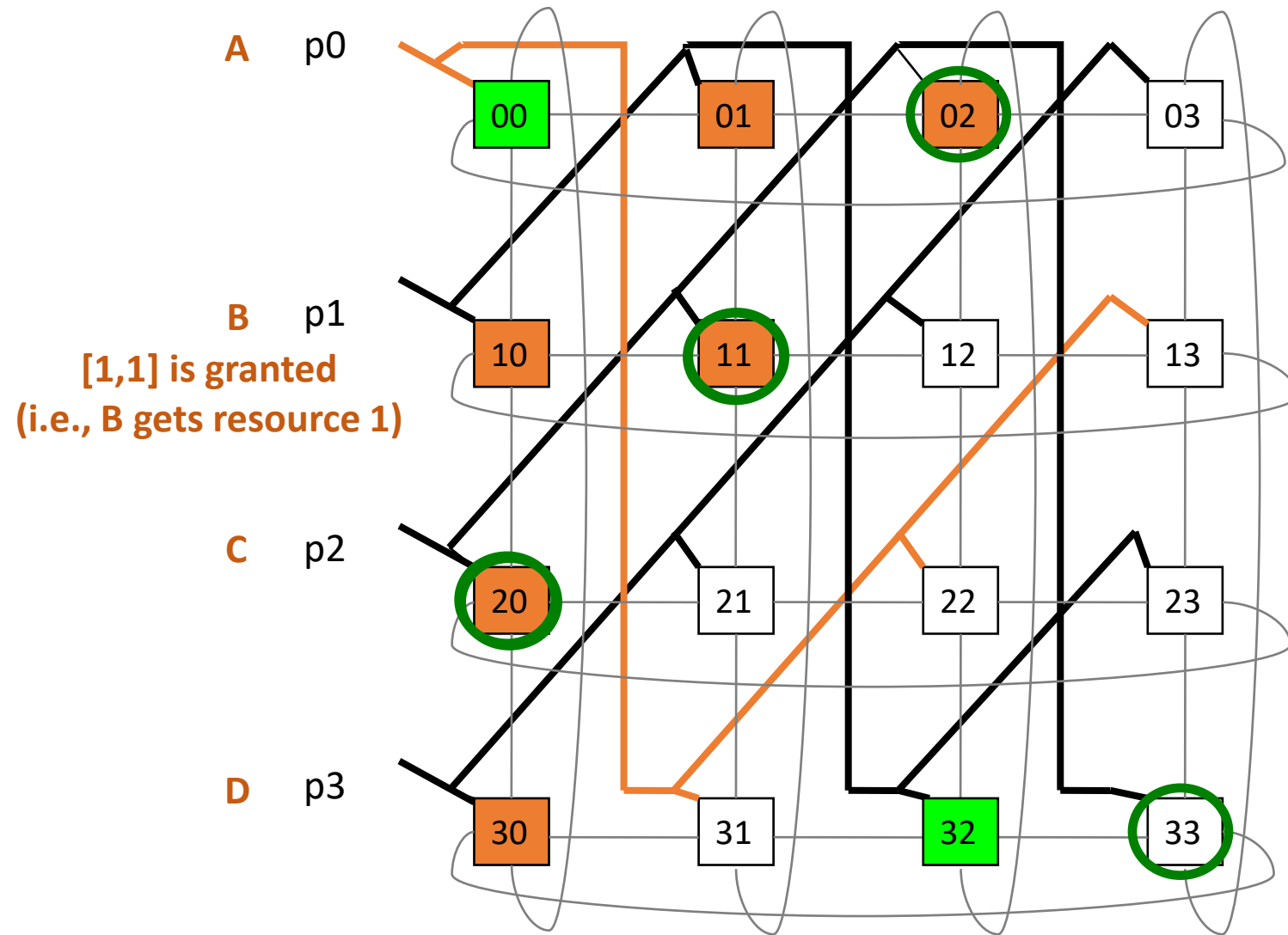
# Wavefront Allocator



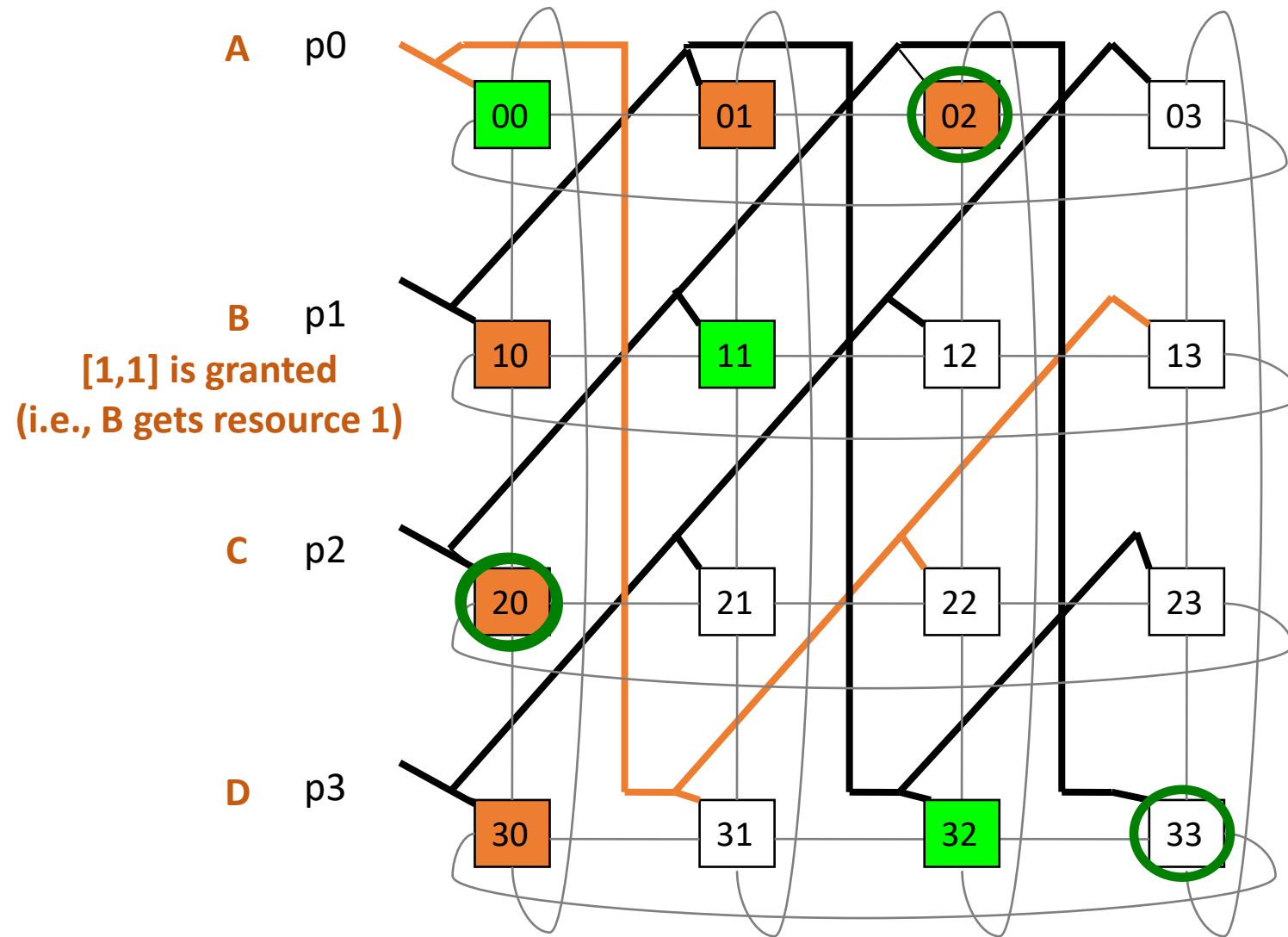
# Wavefront Allocator



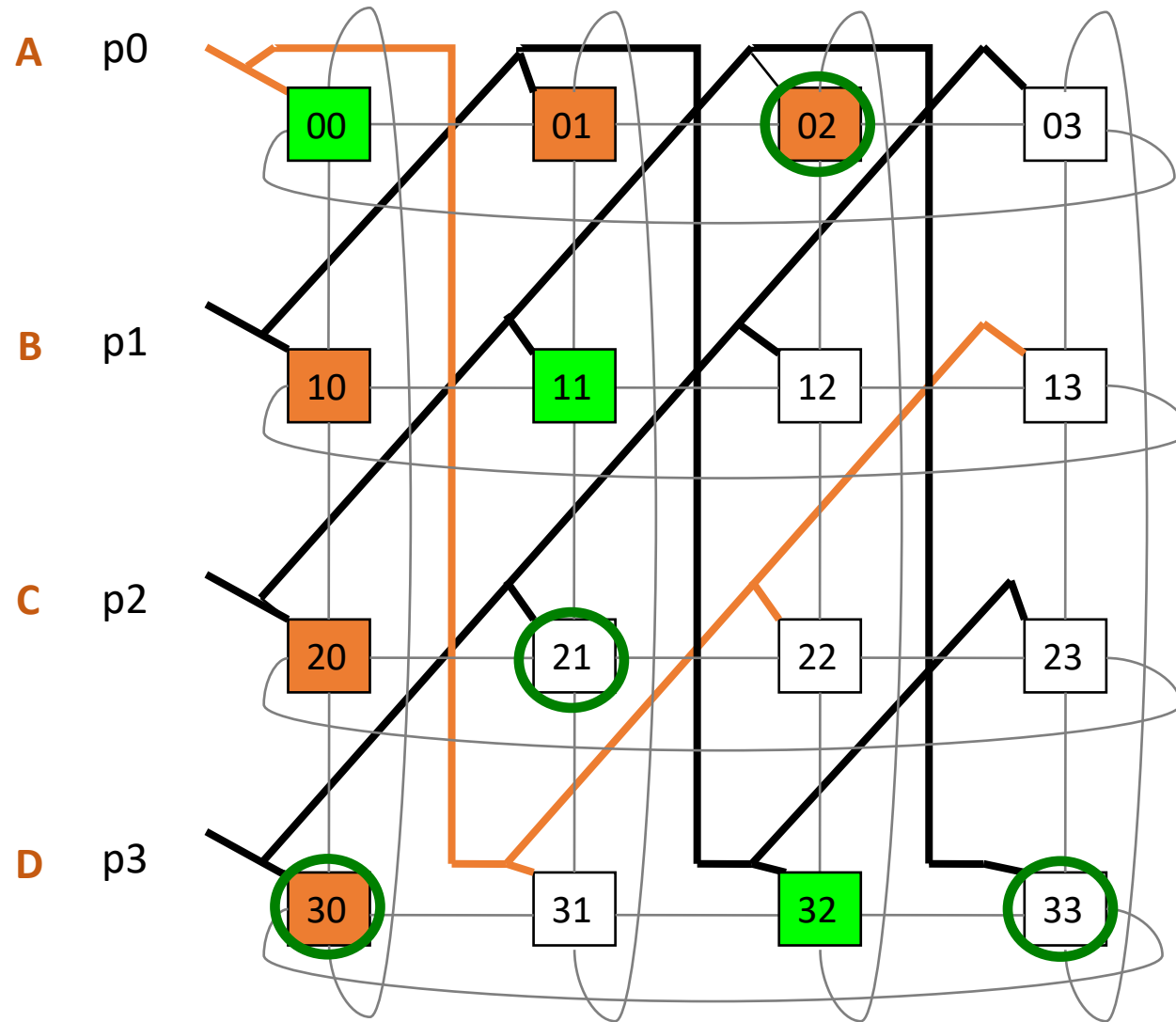
# Wavefront Allocator



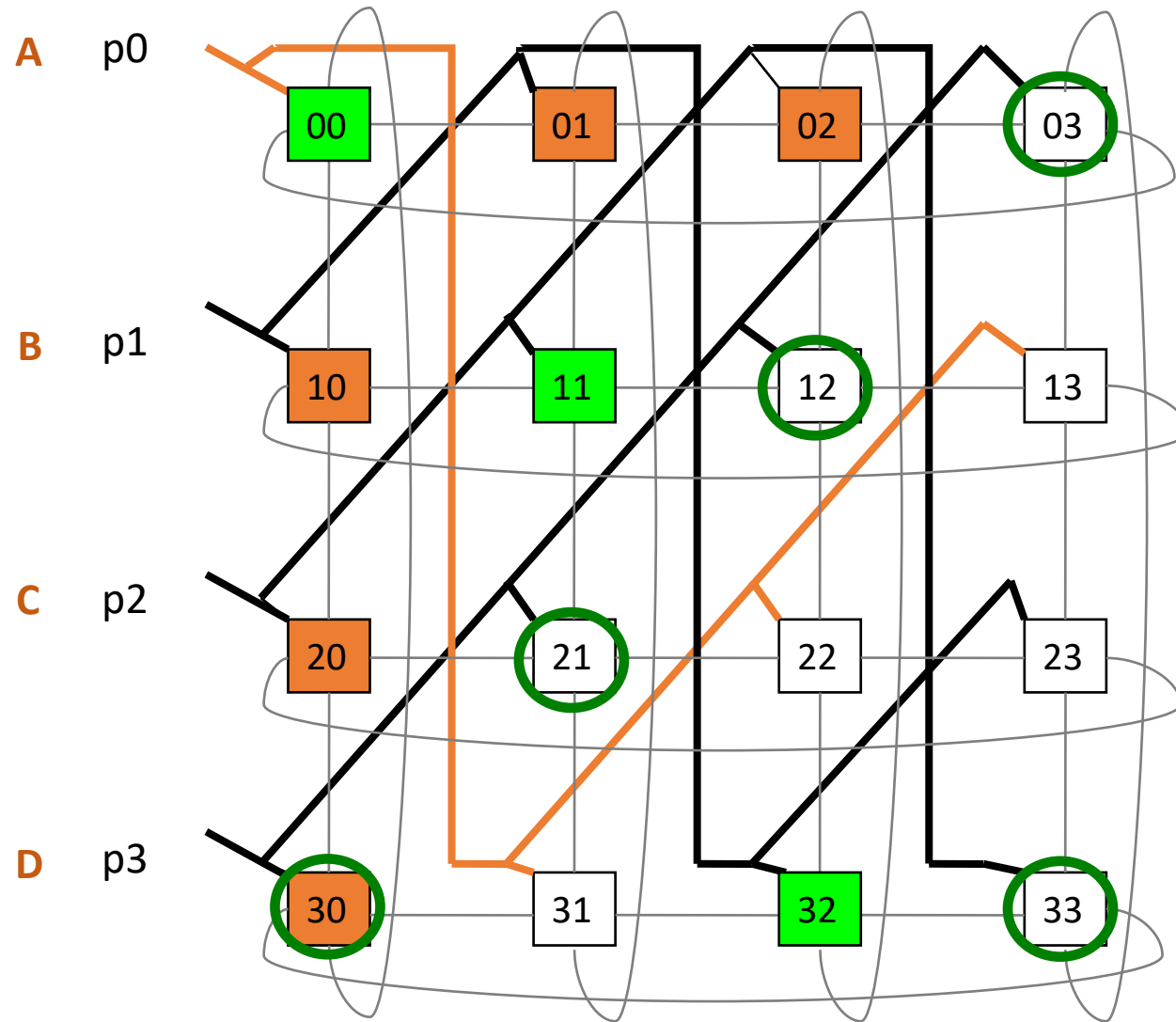
# Wavefront Allocator



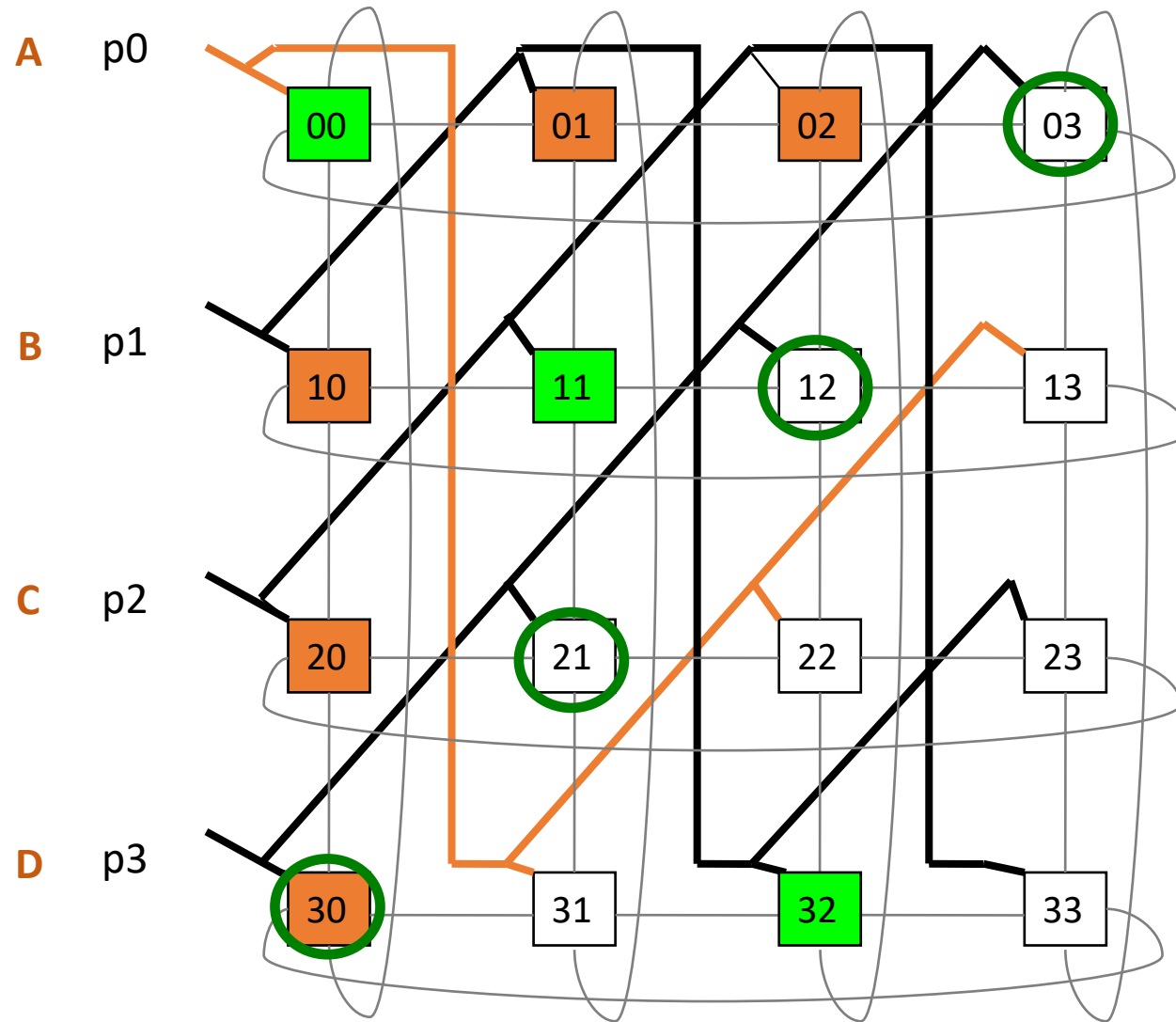
# Wavefront Allocator



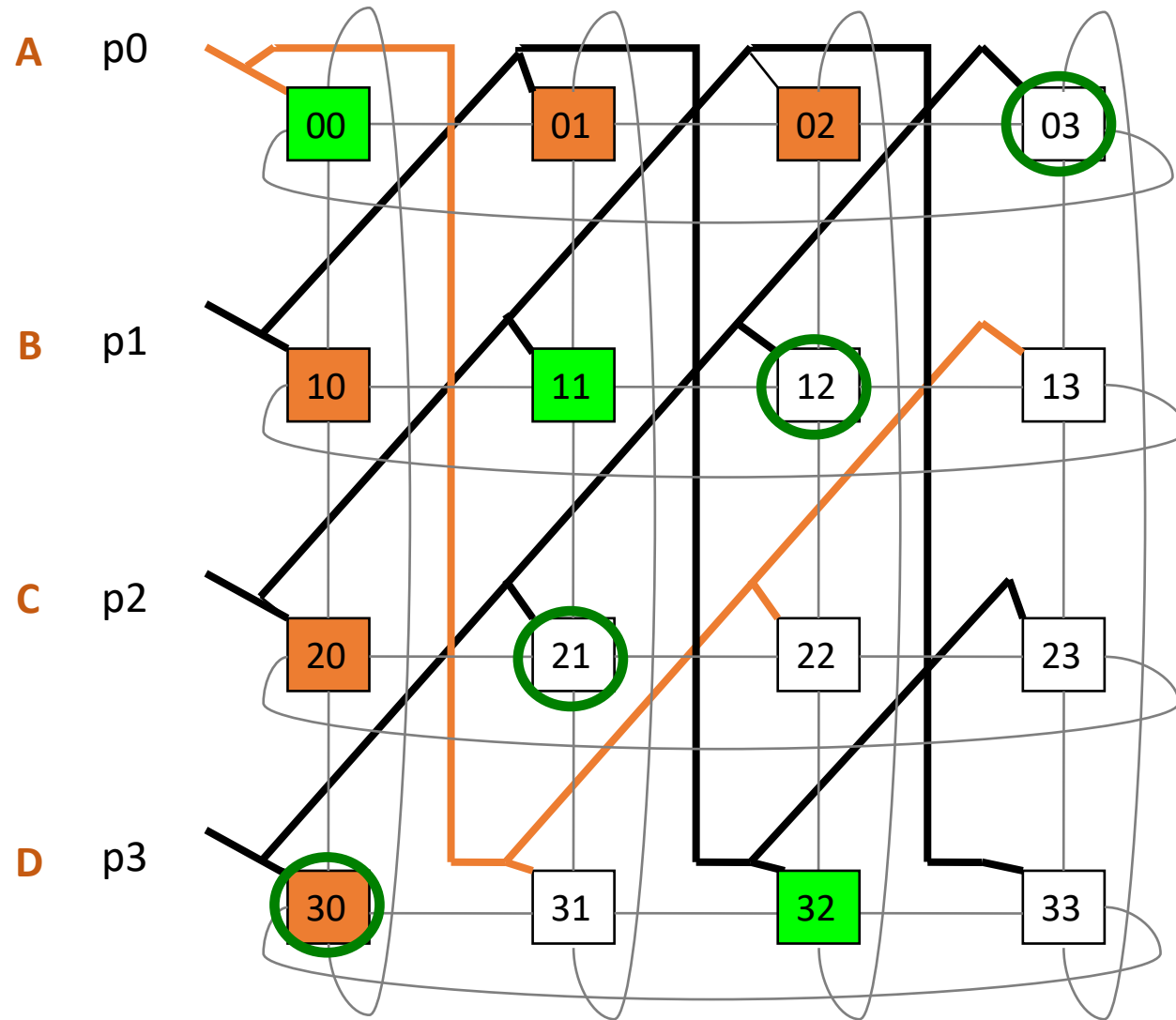
# Wavefront Allocator



# Wavefront Allocator



# Wavefront Allocator



# Separable Allocator

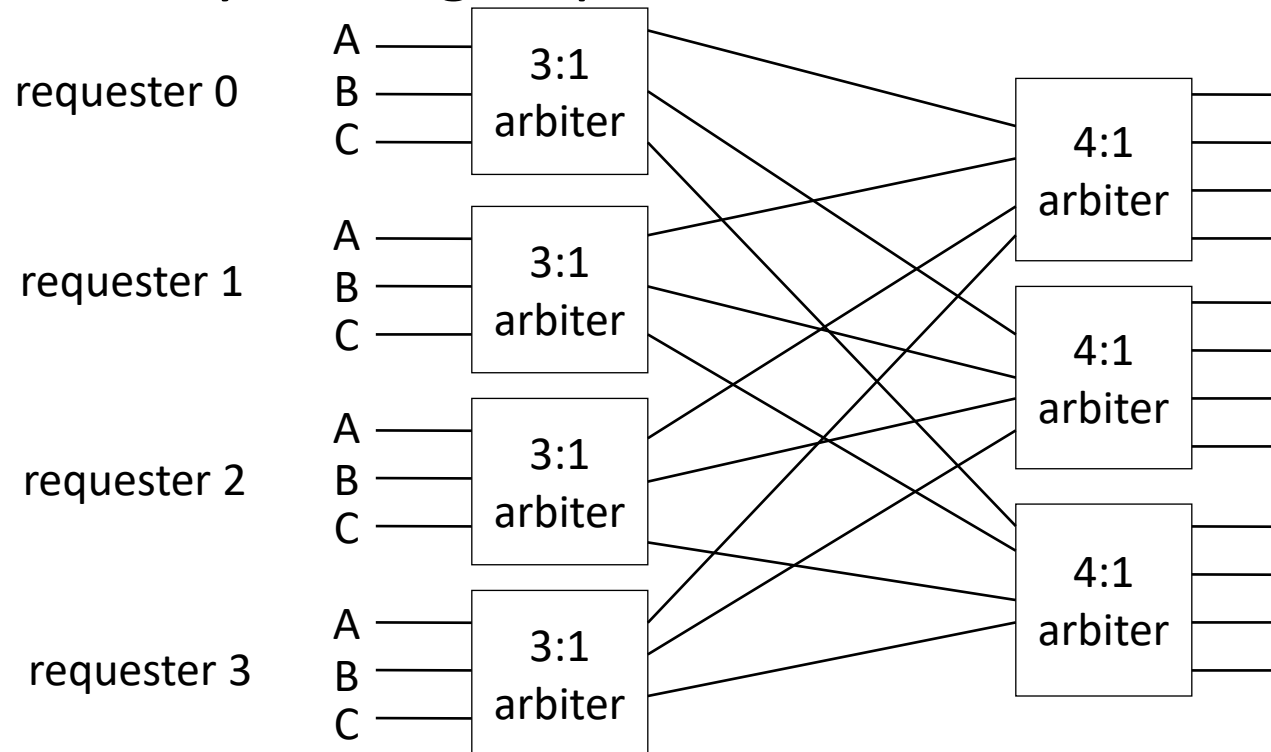
## Separable switch allocator

- Need for pipelineable allocators
- Allocator composed of  $N:1$  arbiters
- **First stage:** select single request at each input port
- **Second stage:** select single request for each output port

# Separable Allocator

e.g., 4:3 allocator = 4 requesters (0,1,2,3) and 3 resources (A,B,C)

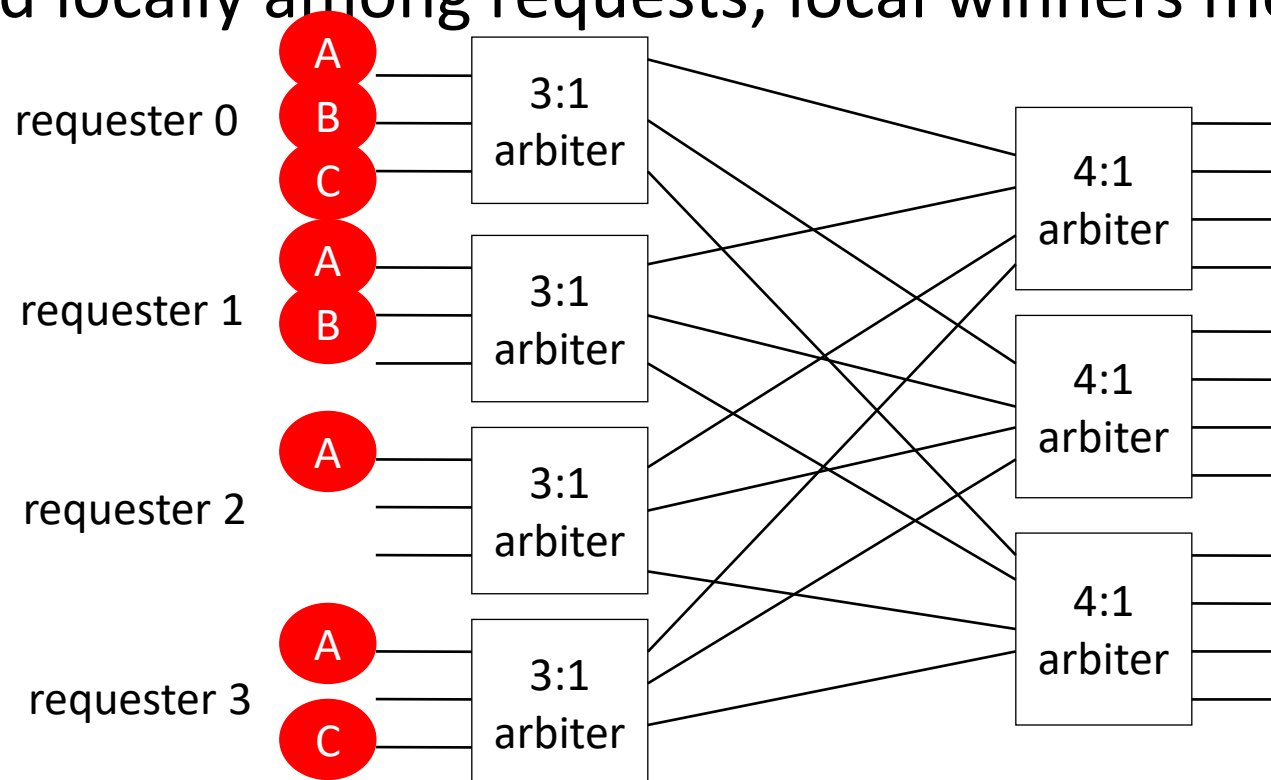
- **First stage:** four 3:1 arbiters, reduces to one resource for each requester
- **Second stage:** three 4:1 arbiters, selects one requester for each resource
- Arbitrated locally among requests; local winners move on to second stage



# Separable Allocator

e.g., 4:3 allocator = 4 requesters (0,1,2,3) and 3 resources (A,B,C)

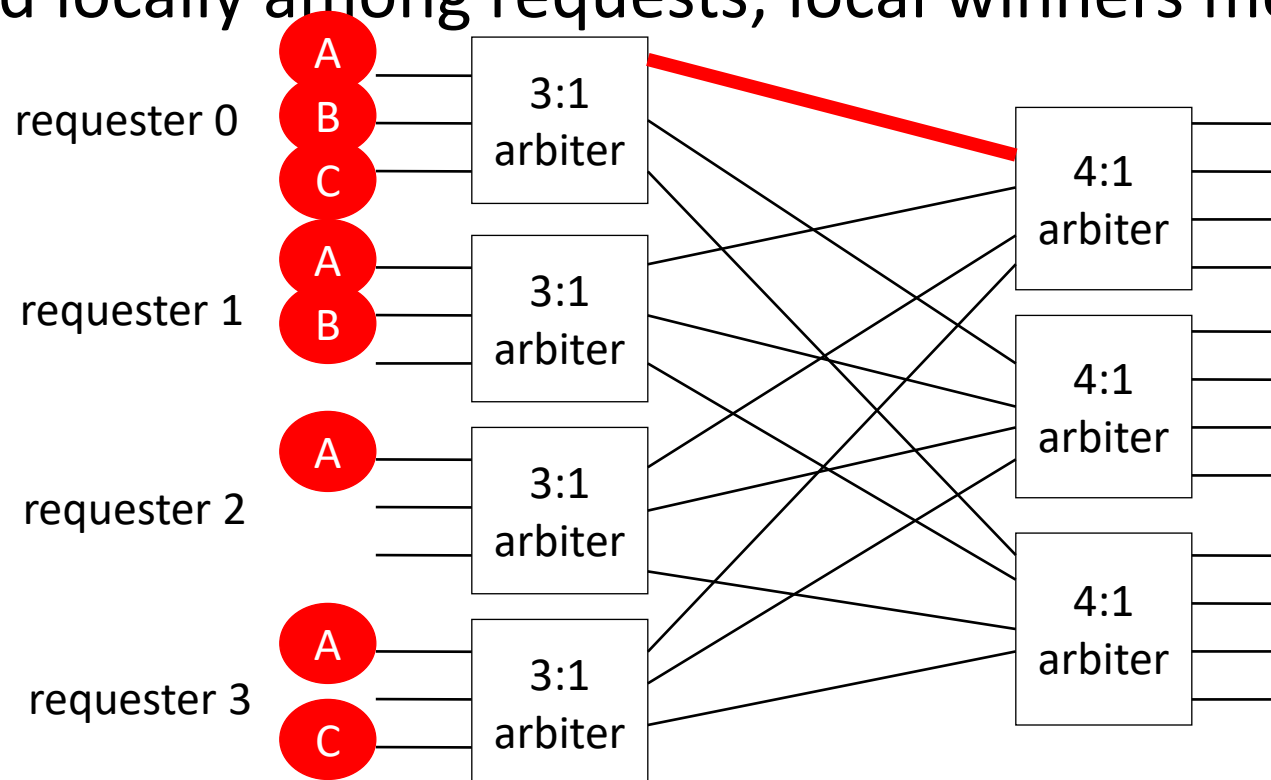
- **First stage:** four 3:1 arbiters, reduces to one resource for each requester
- **Second stage:** three 4:1 arbiters, selects one requester for each resource
- Arbitrated locally among requests; local winners move on to second stage



# Separable Allocator

e.g., 4:3 allocator = 4 requesters (0,1,2,3) and 3 resources (A,B,C)

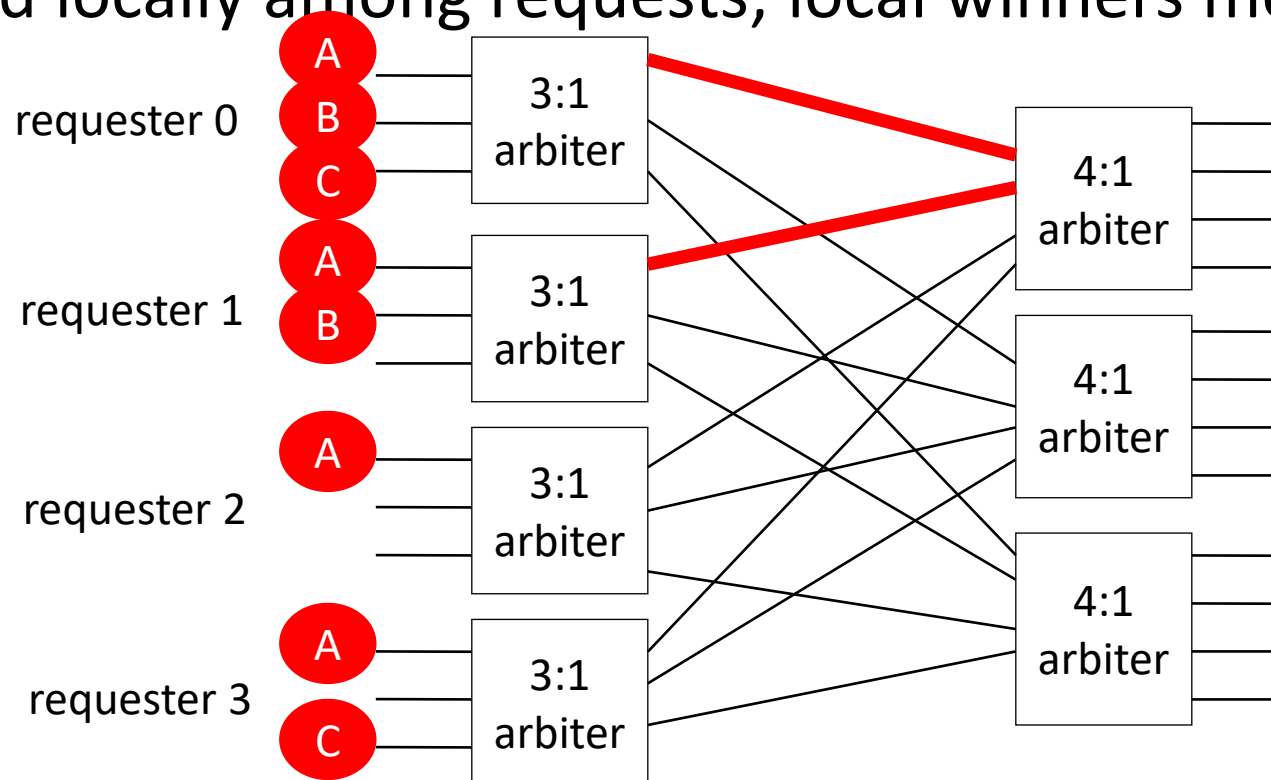
- **First stage:** four 3:1 arbiters, reduces to one resource for each requester
- **Second stage:** three 4:1 arbiters, selects one requester for each resource
- Arbitrated locally among requests; local winners move on to second stage



# Separable Allocator

e.g., 4:3 allocator = 4 requesters (0,1,2,3) and 3 resources (A,B,C)

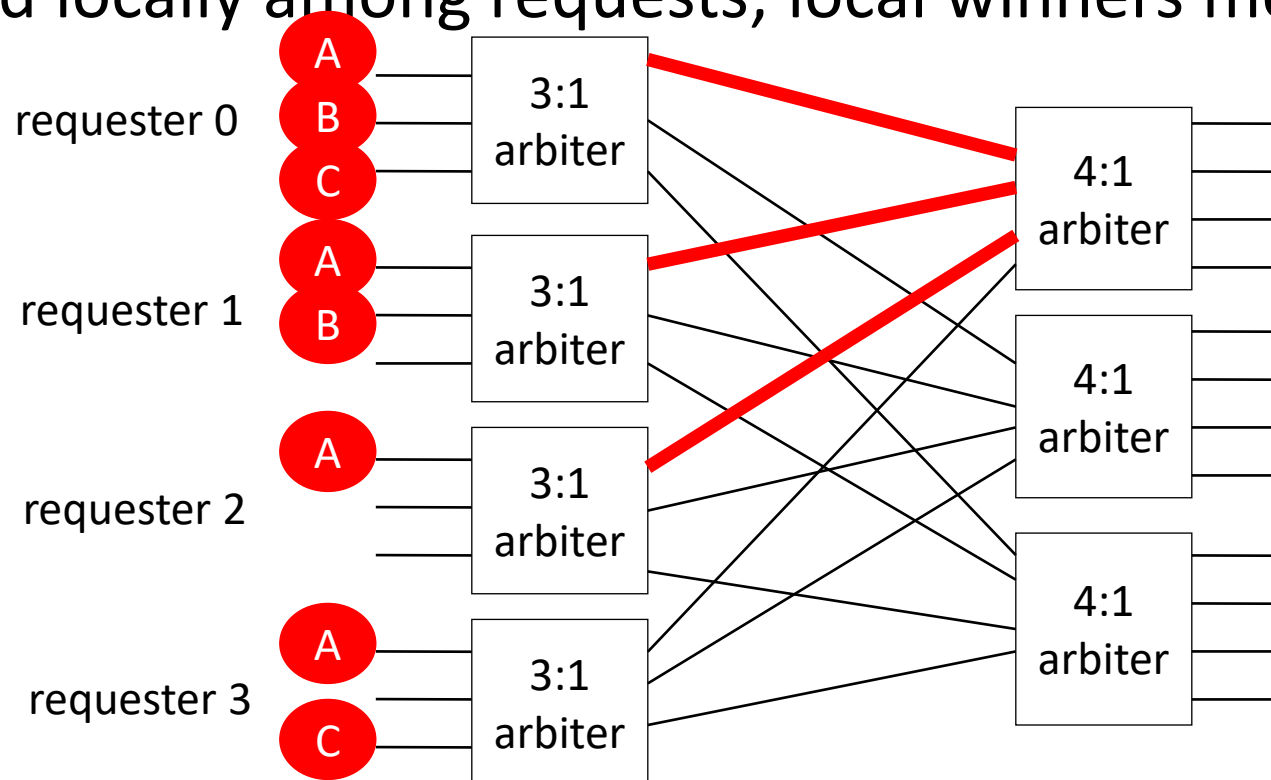
- **First stage:** four 3:1 arbiters, reduces to one resource for each requester
- **Second stage:** three 4:1 arbiters, selects one requester for each resource
- Arbitrated locally among requests; local winners move on to second stage



# Separable Allocator

e.g., 4:3 allocator = 4 requesters (0,1,2,3) and 3 resources (A,B,C)

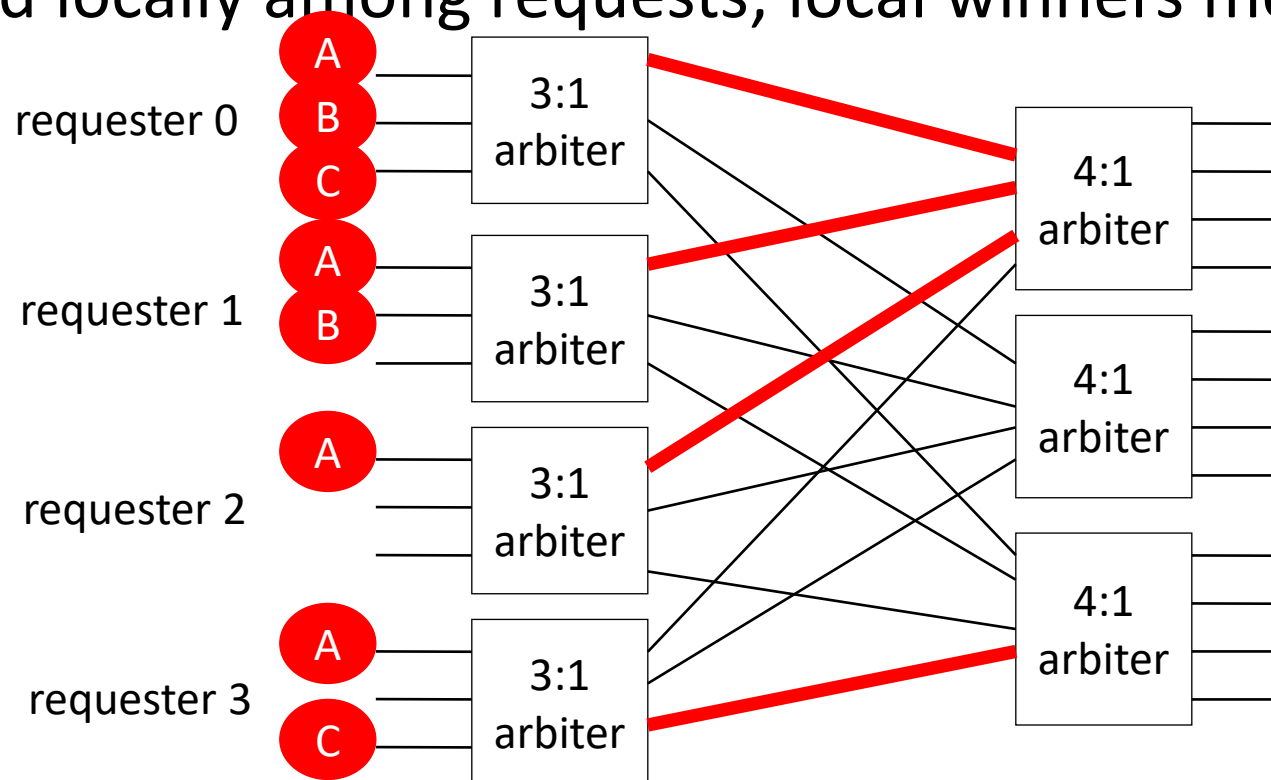
- **First stage:** four 3:1 arbiters, reduces to one resource for each requester
- **Second stage:** three 4:1 arbiters, selects one requester for each resource
- Arbitrated locally among requests; local winners move on to second stage



# Separable Allocator

e.g., 4:3 allocator = 4 requesters (0,1,2,3) and 3 resources (A,B,C)

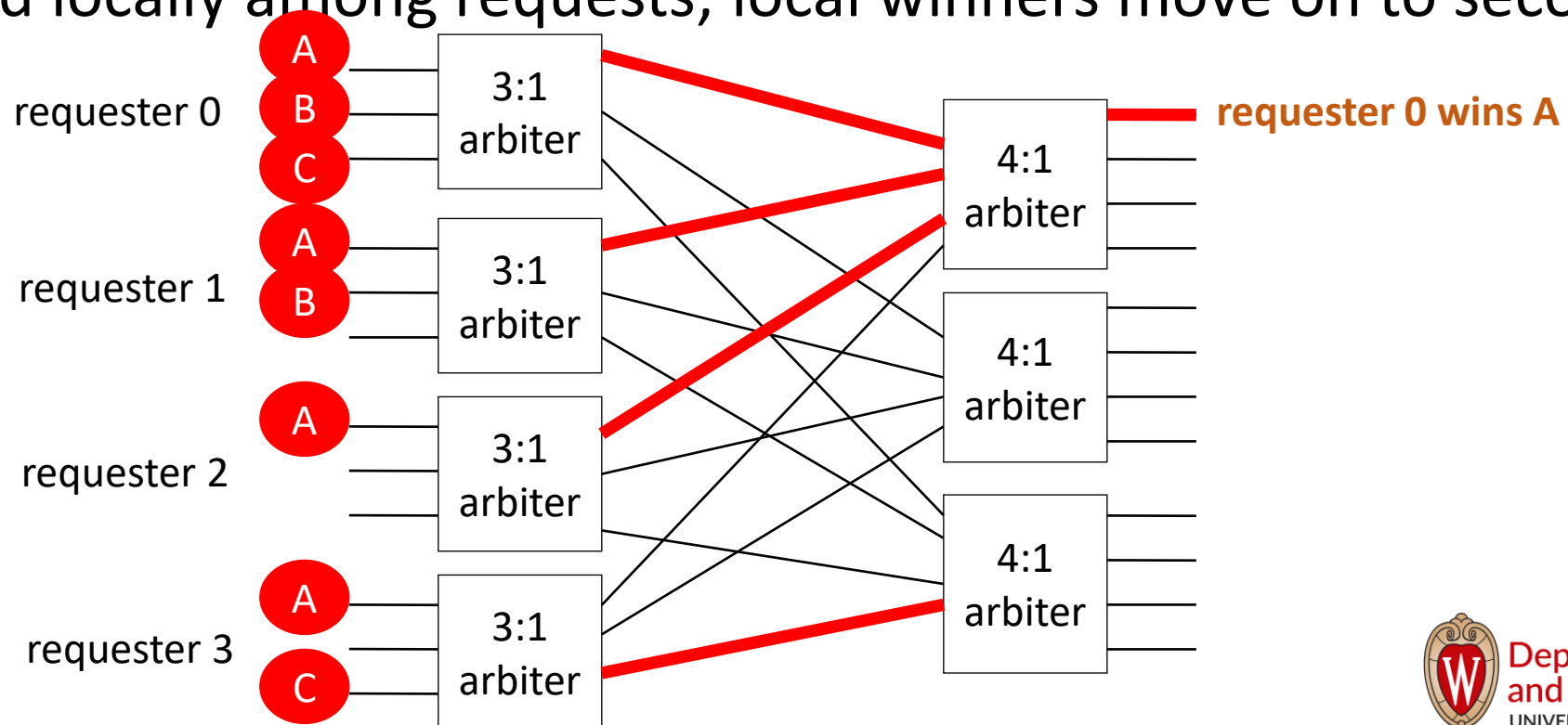
- **First stage:** four 3:1 arbiters, reduces to one resource for each requester
- **Second stage:** three 4:1 arbiters, selects one requester for each resource
- Arbitrated locally among requests; local winners move on to second stage



# Separable Allocator

e.g., 4:3 allocator = 4 requesters (0,1,2,3) and 3 resources (A,B,C)

- **First stage:** four 3:1 arbiters, reduces to one resource for each requester
- **Second stage:** three 4:1 arbiters, selects one requester for each resource
- Arbitrated locally among requests; local winners move on to second stage



# Separable Allocator

e.g., 4:3 allocator = 4 requesters (0,1,2,3) and 3 resources (A,B,C)

- **First stage:** four 3:1 arbiters, reduces to one resource for each requester
- **Second stage:** three 4:1 arbiters, selects one requester for each resource
- Arbitrated locally among requests; local winners move on to second stage

